

Artificial Intelligence

UNIT-2

Search Algorithms: State space representation, Search graph and Search tree. Random search, Search with closed and open list, Depth first and Breadth first search. Heuristic search, Best first search. A* algorithm, problem reduction, constraint satisfaction, Game Search, minmax algorithm, alpha beta pruning, constraint satisfaction problems.

Basic Contexts in Problem solving

- Effective and efficient solutions can be formulated for real-world problems with the help of AI search algorithms.
- Problem-solving is one direct application of search.
- **Agents** are entities that perceive their environment and take actions to achieve specific goals.
- **Problem-solving agents** analyze situations and employ various techniques to achieve desired outcomes.

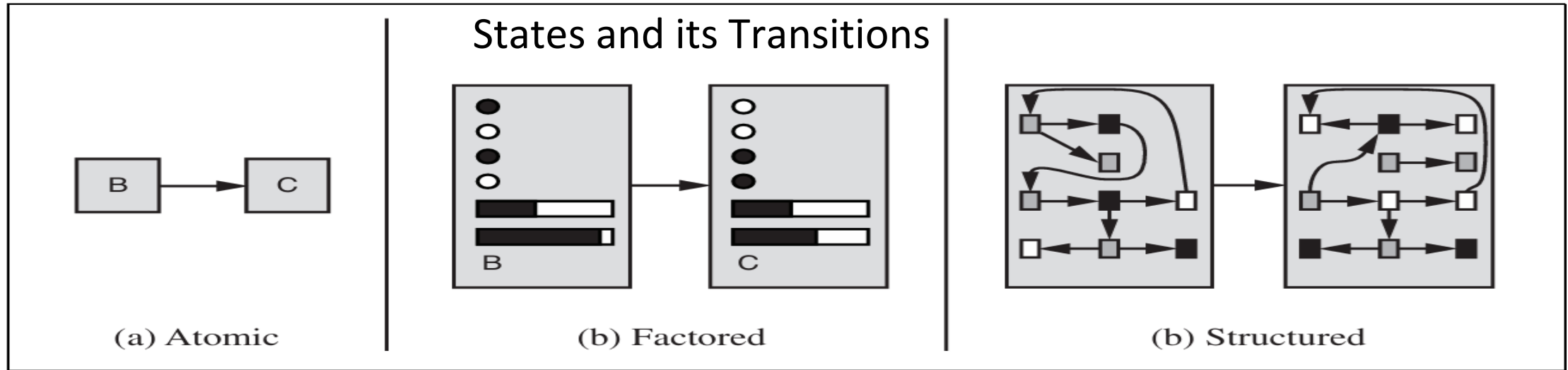
Types of Problems in AI

- **Ignorable Problems:** Can be overlooked (Minor data preprocessing errors)
- **Recoverable Problems:** Can recover from the error (Incorrect or biased training data)
- **Irrecoverable Problems:** Permanent failure of system (Security attacks)

Steps in Problem Solving in Artificial Intelligence (AI)

- Problem definition
- Problem analysis
- Knowledge Representation
- Problem solving
- Agent can follow this **four-phase** problem-solving process.
- **Goal formulation:** Goals organize behavior by limiting the **objectives** and hence the actions to be considered.
- **Problem formulation:** Problem formulation is the process of **deciding what actions and states to consider** for a given a goal.
- **Search:** Before taking any action in the real world, the agent simulates **sequences of actions** in its model, searching until it finds a sequence of actions that reaches the goal. **Such a sequence is called a solution.**
- **Execution:** The agent can **execute the actions in the solution, one at a time.**

How the components of agent programs work



The representations along an axis of increasing complexity and expressive power

Atomic: (Block box) No internal structure

Factored: (Autonomous Vehicle) Each state have different attributes

Factored representations-including constraint satisfaction algorithms, propositional logic, planning Bayesian networks and the machine learning algorithms

Structured: (Connection between Variable to variables)

Relational databases and first-order logic , first-order probability models, knowledge-based learning and much of natural language understanding

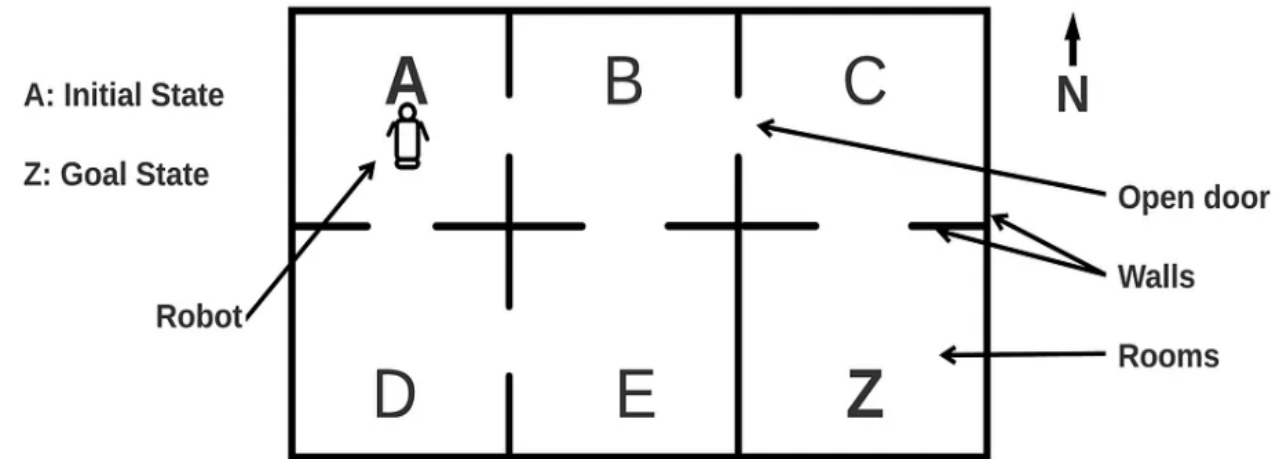
SOLVING PROBLEMS BY SEARCHING

Problem consists of four major components:

- **States**
- **Actions/Operations**
- **Goals**
- **Path.**
- Simply, problem-solving can be identified as a process involving transforming a set of states through actions or operations to a goal along a path/paths.
- Problem-solving agents use **atomic** representations.
- Agents that use **factored** or **structured** representations of states are called **planning agents**.
- **Problem solving agent** goes through these major components to reach goal state.

State space representation

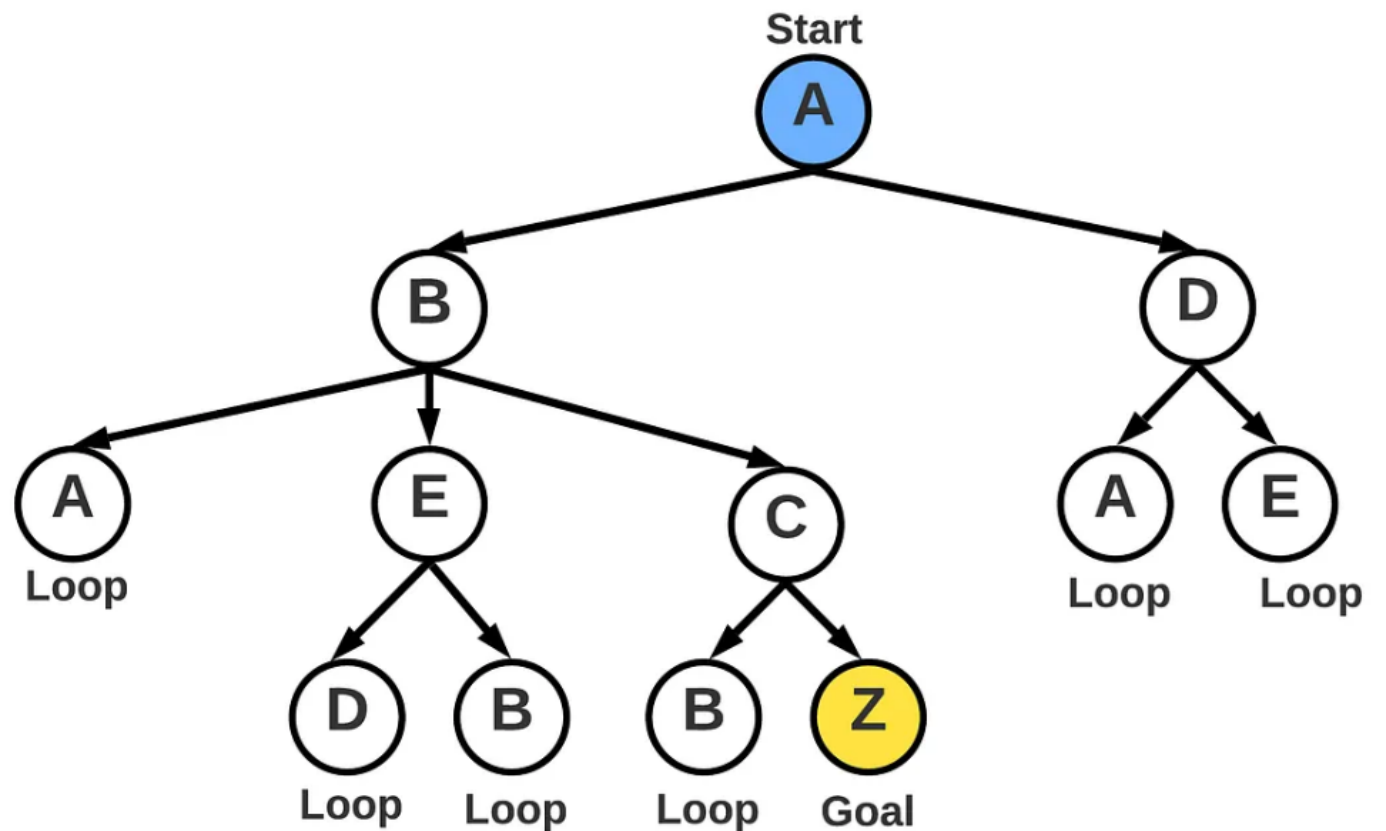
- **State-space** consists of all the possible states together with actions to solve a specific problem.
- In the **graphical representation** of state space (such as trees), the **states are represented by nodes** while the **actions are represented by arcs**.
- Any **state space** has an initial **‘Start’ node** and an ending **‘Goal’ node** or multiple Goal States.
- The **path from the initial start node to the final goal node** is known as the **‘Solution’** for the particular problem.
- *A problem may have multiple goals as well as paths.*



What is the state-space diagram for this?

There are many paths beside the direct path A, B, C, Z.

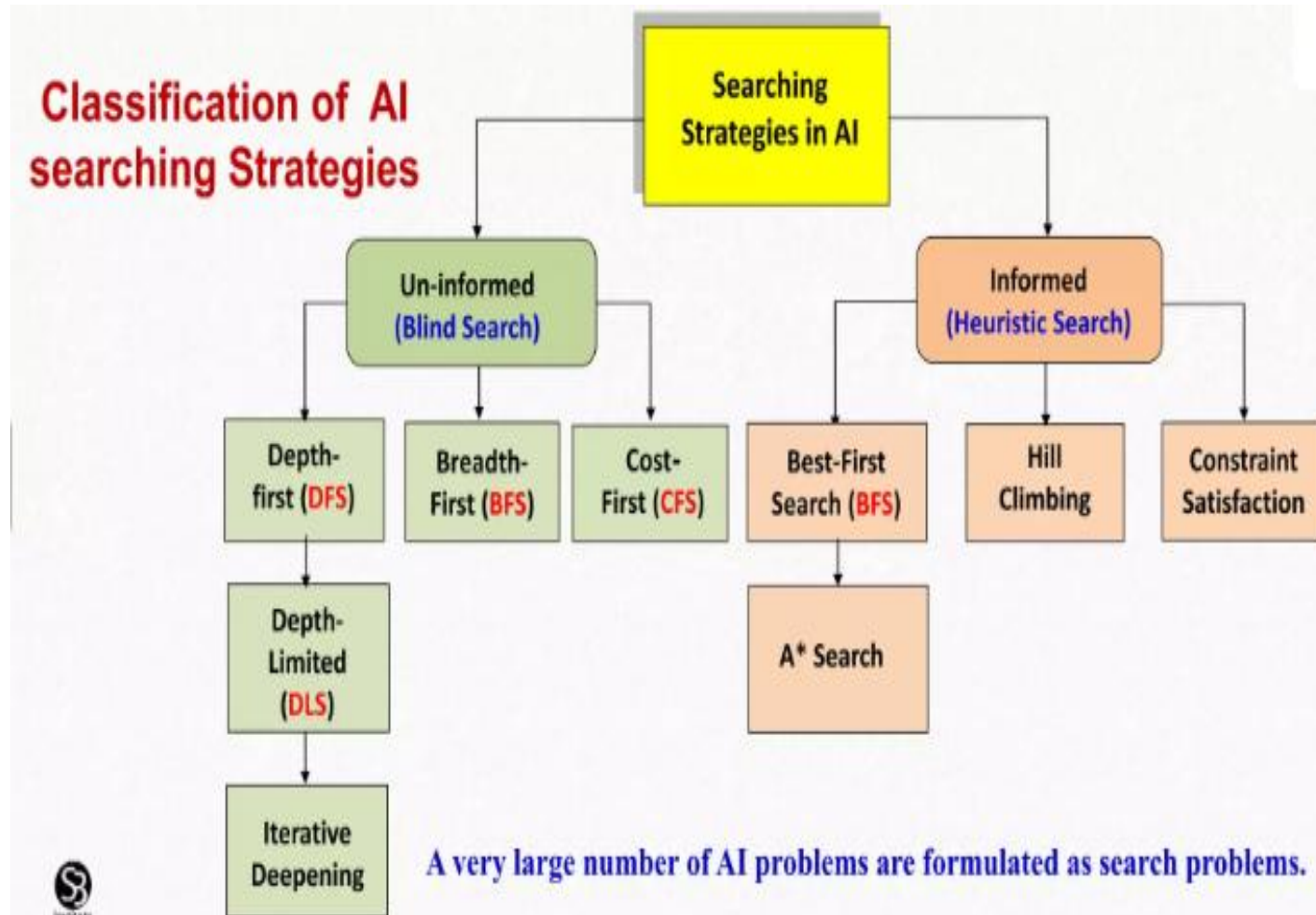
The main aim is to figure out the best/shortest path



Search algorithms

- A search algorithm defines how to find the path (solution) from the initial state to the goal state.
- **Informed search (heuristic):** Algorithms have **information on the goal state**, which helps in more efficient searching.
- **Un-Informed search:** have **no additional information on the goal node** other than the one provided in the problem definition.
- **Search algorithm are evaluated based on completeness, Optimality, time and space complexity**

18-08-2025



Complexity is measured with B:Branch (B:maximum number of successors of any node, D: Depth (i.e., the number of steps along the path from the root); M: Maximum length of any path

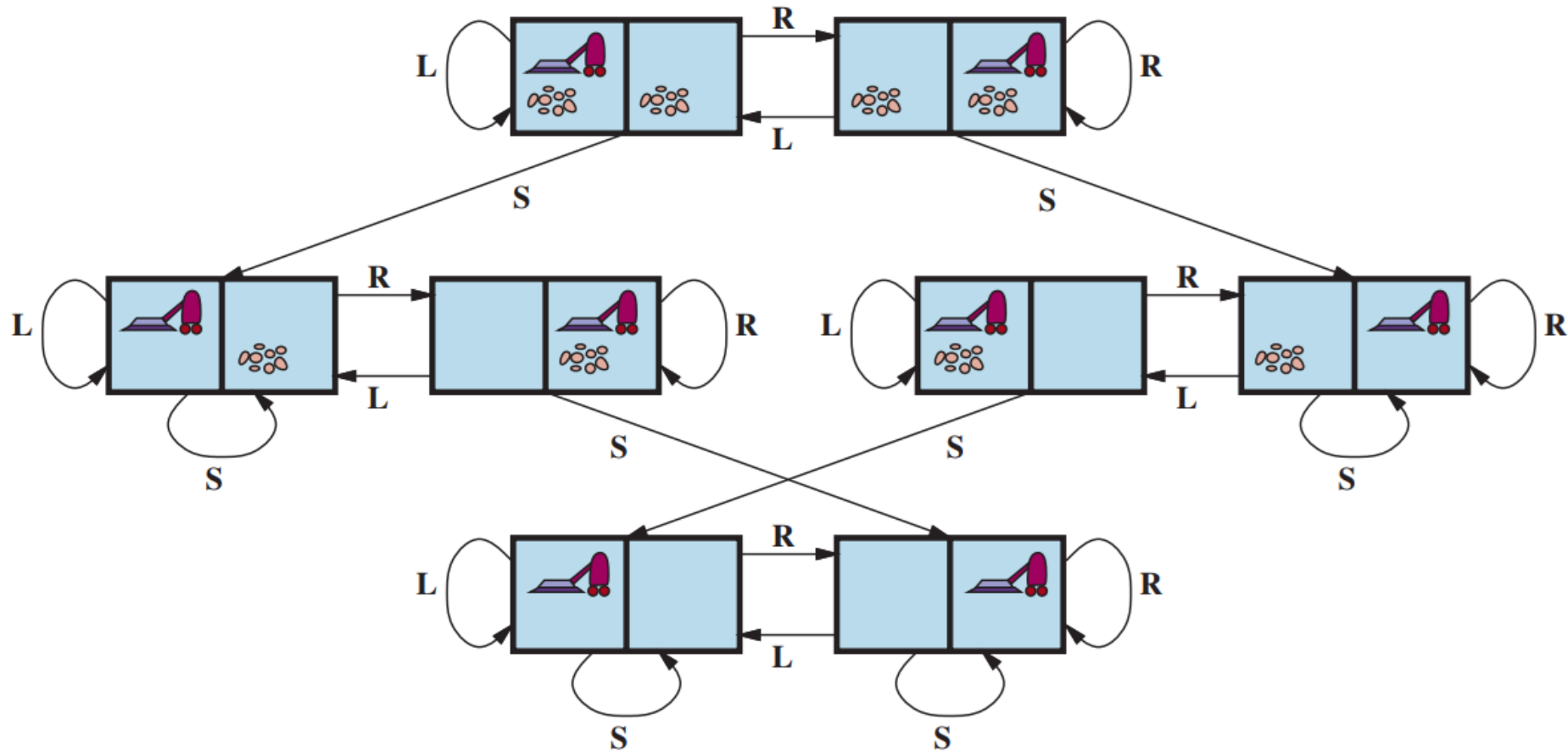
AI&DS-2025

Search Algorithms-Terminology

- A set of possible **states**(State Space) (s)
- The **initial state** that the agent starts in
- A set of one or more **goal states**
- The **actions** available to the agent. - $ACTIONS(s)$
- A **transition model**, which describes what each action does. $RESULT(s, a)$
- An **action cost function**, denoted by $ACTION-COST(S, a, S')$ from s to s' state.
- A sequence of actions forms a **path**
- **Solution** is a path from the initial state to a goal state
- The **total cost** of a path is the sum of the individual action costs.
- An **optimal solution** has the lowest path cost among all solution

Grid world problem-Vacuum world

- **Grid world problem** is a two-dimensional rectangular array of square cells in which agents can move from cell to cell.
- **States:** A state of the world says which objects are in which cells.
 - objects are the agent and any dirt
 - The agent can be in either of the two cells, and each cell can either contain dirt or not, so there are $2 \cdot 2 \cdot 2 = 8$ states
 - a vacuum environment with n cells has $n \cdot 2^n$ states
- **Initial state:** Any state can be designated as the initial state.
- **Actions:** Suck, move Left, and move Right
- **Transition model:** Suck removes any dirt from the agent's cell; Forward moves the agent ahead one cell in the direction it is facing
- **Goal states:** The states in which every cell is clean.
- **Action cost:** Each action costs 1.



States:
Initial state:
Actions:
Transition model:
Goal test:
Path cost:

The state-space graph for the two-cell vacuum world. There are 8 states and three actions for each state: L = Left, R = Right, S = Suck

Sokoban Puzzle



Sliding Tile Puzzle

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

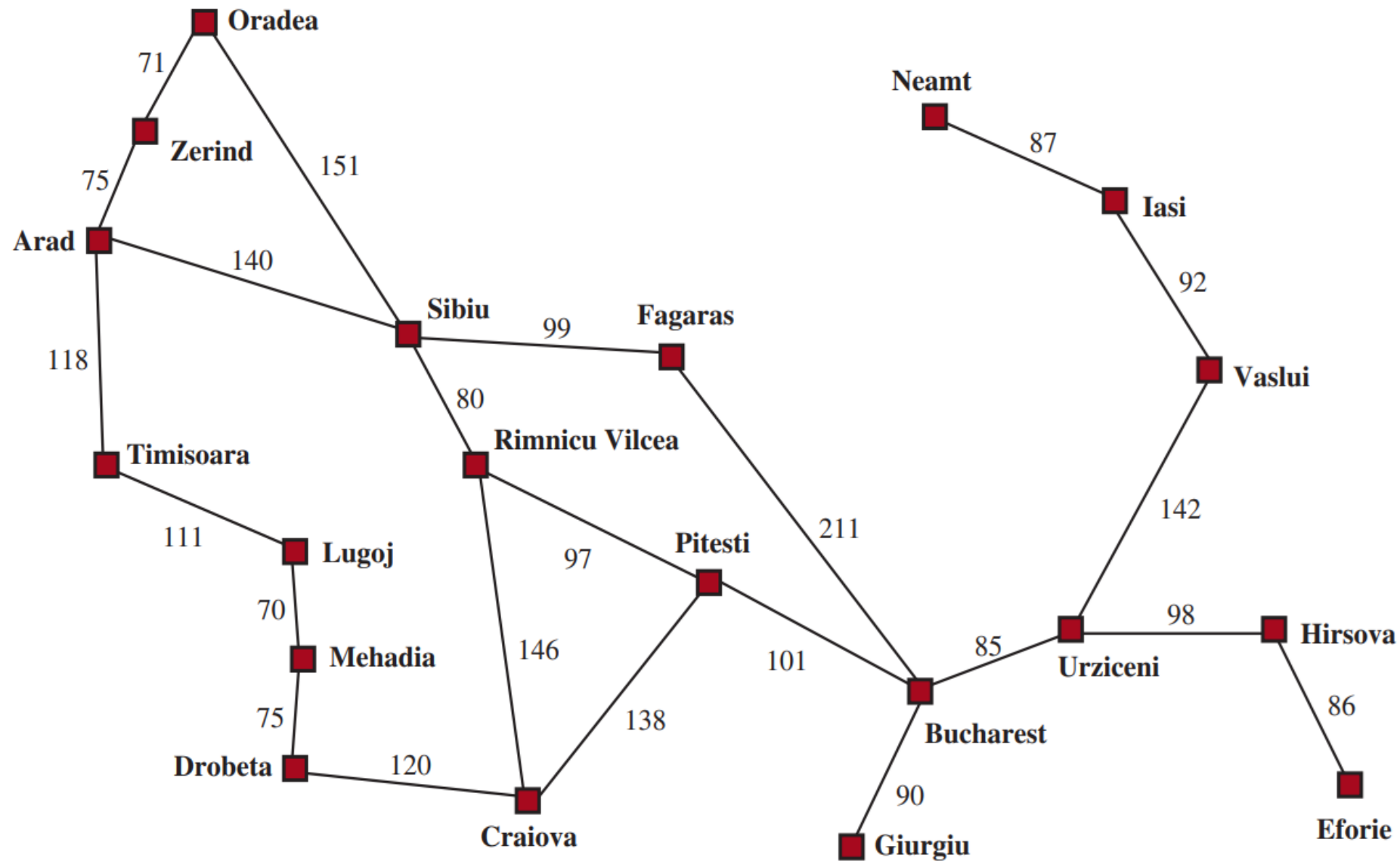
8-Queens Problem

- The goal of the 8-queens problem is to place eight queens on a chessboard such that no queen attacks any other.
- Rule: A queen attacks any piece in the same row, column or diagonal.
- **Incremental formulation:**
 - **States:** Any arrangement of 0 to 8 queens on the board is a state.
 - **Initial state:** No queens on the board.
 - **Actions:** Add a queen to any empty square.
 - **Transition model:** Returns the board with a queen added to the specified square.
 - **Goal test:** 8 queens are on the board, none attacked.
- In this formulation, we have $64 \cdot 63 \cdot \dots \cdot 57 \approx 1.8 \times 10^{14}$ possible sequences to investigate. A better formulation would prohibit placing a queen in any square that is already attacked

Real-world problems-Airline travel problem

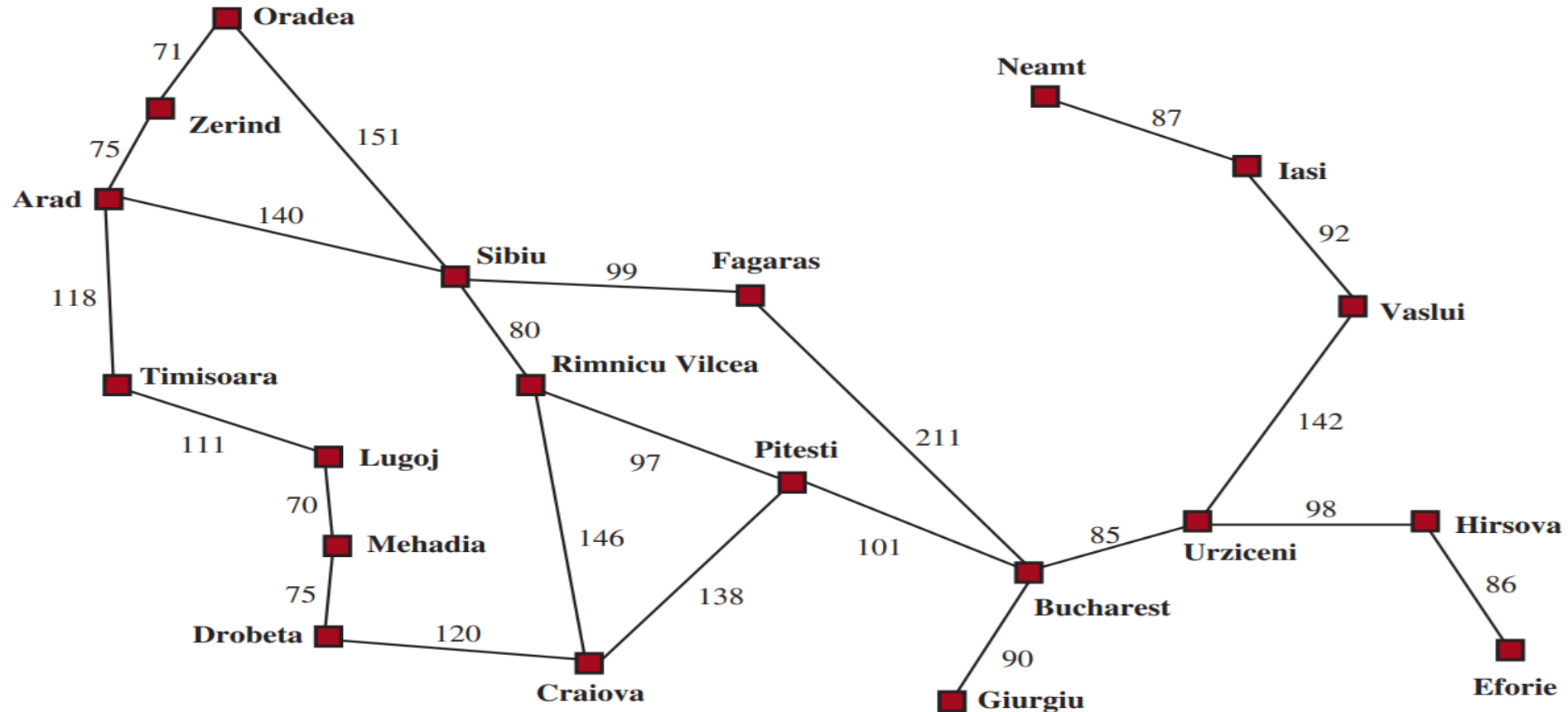
- **States:** Includes a location (e.g., an airport) and the current time
- **Initial state:** The user's home or airport
- **Actions:** Take any flight from the current location, in any seat class, leaving after the current time, leaving enough time for within-airport transfer if needed.
- **Transition model:** The state resulting from taking a flight will have the flight's destination as the new location and the flight's arrival time as the new
- **Goal state:** A **destination** city. Sometimes the goal can be more complex, such as "arrive at the destination on a **nonstop** flight."
- **Action cost:** A combination of monetary cost, waiting time, flight time, customs and immigration procedures, seat quality, time of day, type of airplane, frequent-flyer reward points, and so on.

- State Space?
- initial state?
- goal states?
- Actions?
- transition model?
- action cost function?
- Path?
- Solution?
- total cost?
- optimal solution?
- Repeated state
- Cycle
- Loopy path
- Redundant path



Search Algorithms

- A search algorithm takes a search problem as input and returns a solution or Failure



- A **solution** is an **action sequence**
- The possible action sequences **starting at the initial state form a search tree with the initial state at the root**
- **Expanding** the current state
- **Generating a new set of states** (generating a new set of states)-child nodes
- **Parent node (current node)**
- **Child node (new set of state)**
- **Leaf node**
- **Frontier(open list):** The **set of all leaf nodes available for expansion** at any given point is called the frontier.
- The **process of expanding nodes** on the frontier continues until either a solution is found or there are no more states to expand.
- **Search strategy:** How an algorithm chooses **which state to expand next** is called **Search Strategy**
- **Repeated state:** In(Arad) to Sibiu to Arad
- **Loopy path:** Loopy paths are a special case of the more general concept of **redundant paths**,
- **Redundant Path:** Which exists whenever there is **more than one way to get from one state to another**.
- **Example:** Consider the paths Arad–Sibiu (140 km long) and Arad–Zerind–Oradea–Sibiu (297 km long).
- **How to Avoid Redundant paths:** Use a separate data structures called **Explored Set(Closed List)**

(This remembers every expanded node. Newly generated nodes that match previously generated nodes—ones in the explored set or the frontier—can be discarded instead of being added to the frontier)

Search graph and Search tree

Search Graph: A search graph is a general representation of a problem where nodes represent states and edges represent transitions (actions) between states.

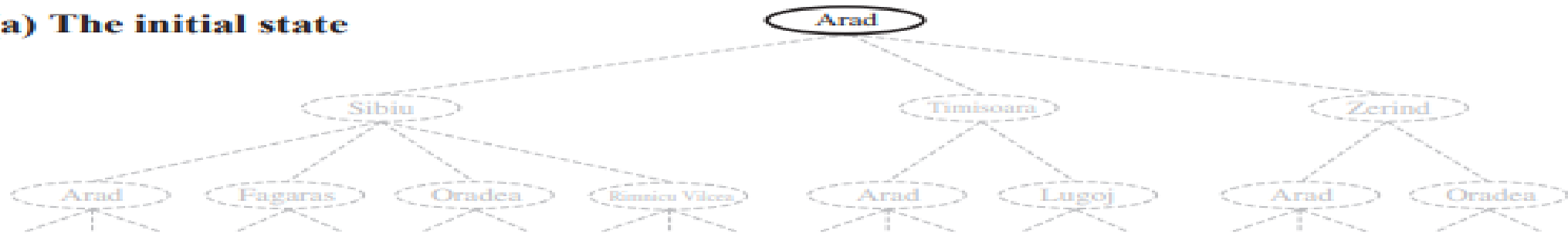
- **Cycles:** Graphs can have cycles, meaning a state can be revisited.
- **Multiple Paths:** There may be multiple paths to reach the same state.
- **Efficiency:** Algorithms like A* or Dijkstra's often use graphs to find optimal paths.
- **Use Case:** Suitable for problems where states can be revisited, such as navigation or route planning.

Search Tree: A search tree is a hierarchical structure where nodes represent states, and edges represent actions leading to child states. It is a subset of a graph with no cycles.

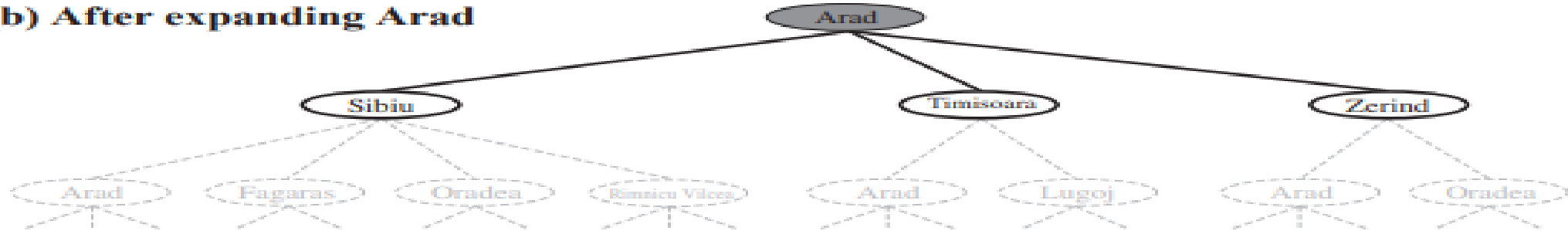
- **No Cycles:** Trees do not allow revisiting states.
- **Single Path:** Each state (node) is reached by only one unique path.
- **Exploration:** Algorithms like Depth-First Search (DFS) or Breadth-First Search (BFS) often use trees.
- **Use Case:** Ideal for problems where states are explored systematically, such as game trees or decision-making.

Conversion of Graph to Tree in Searching Problems

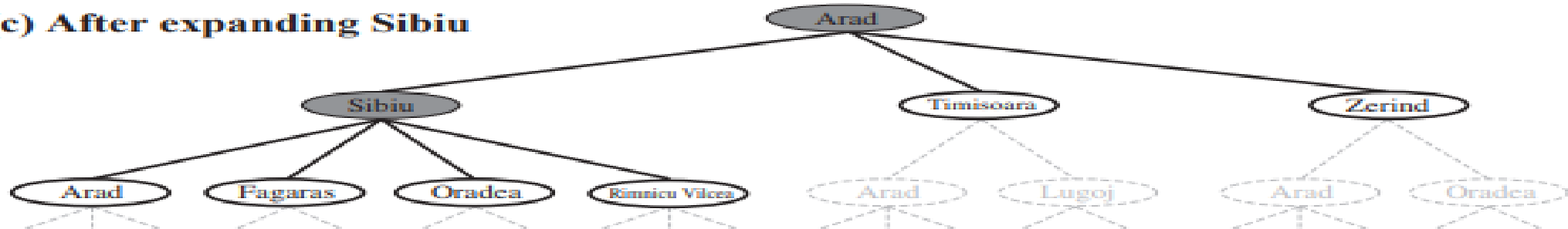
(a) The initial state



(b) After expanding Arad



(c) After expanding Sibiu



function TREE-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 expand the chosen node, adding the resulting nodes to the frontier

function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
 initialize the frontier using the initial state of *problem*
 initialize the explored set to be empty
 loop do
 if the frontier is empty **then return** failure
 choose a leaf node and remove it from the frontier
 if the node contains a goal state **then return** the corresponding solution
 add the node to the explored set
 expand the chosen node, adding the resulting nodes to the frontier
 only if not in the frontier or explored set

- GRAPH-SEARCH algorithm contains at most one copy of each state, as growing a tree directly on the state-space graph.
- This algorithm has another nice property: the frontier separates the state-space graph into the explored region and the unexplored region

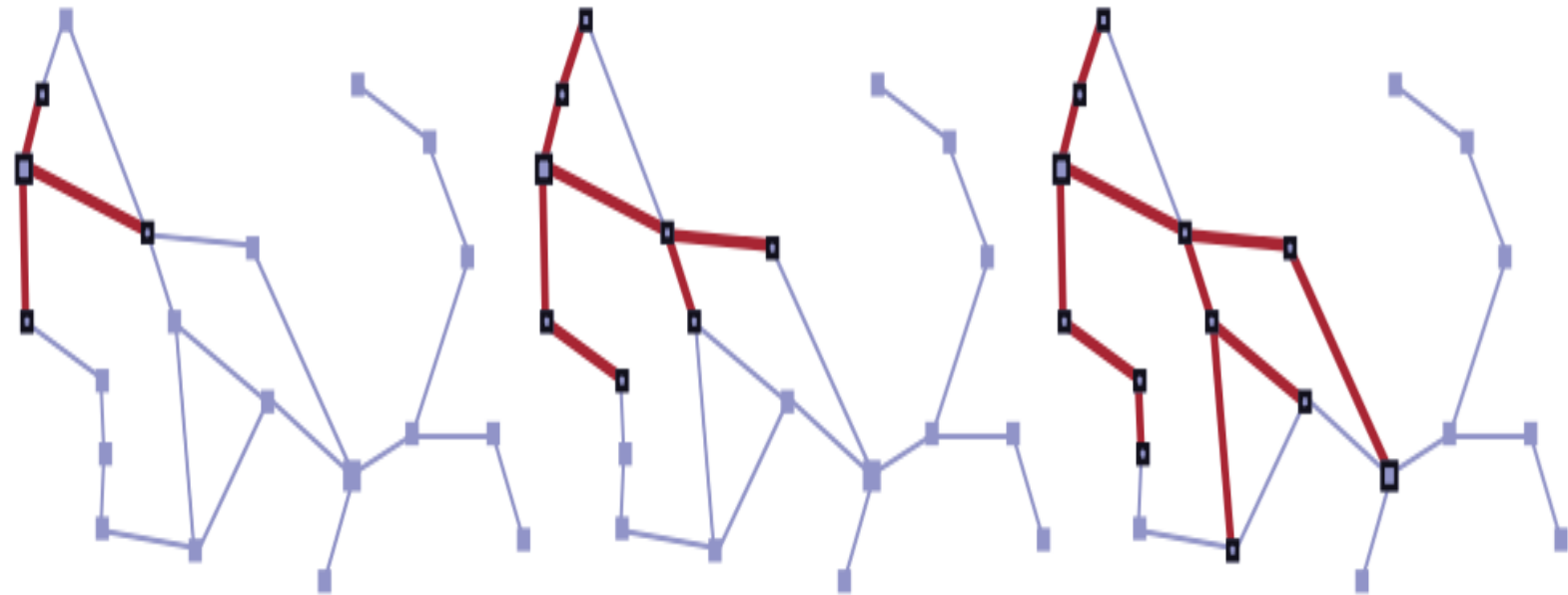
Ex: Searching between Arad to Bucharest

Generating a new node
(called a Child node or
successor node)

Parent node.

Frontier

Reached



Frontier separates two regions of the state-space graph: an interior region where every state has been expanded, and an exterior region of states that have not yet been reached

function CHILD-NODE(*problem, parent, action*) **returns** a node

return a node with

STATE = *problem.RESULT(parent.STATE, action)*,

PARENT = *parent*, ACTION = *action*,

PATH-COST = *parent.PATH-COST + problem.STEP-COST(parent.STATE, action)*

What is the difference between node and a state?

- A node is a bookkeeping data structure used to represent the search tree
- A state corresponds to a configuration of the world.
- Nodes are on particular paths, as defined by PARENT pointers, whereas states are not.
- Two different nodes can contain the same world state if that state is generated via two different search paths.
- The explored set(Closed Set) can be implemented with a hash table to allow efficient checking for repeated states
- Canonical form: logically equivalent states should map to the same data structure (used in sales mam problem)

Search data Structures

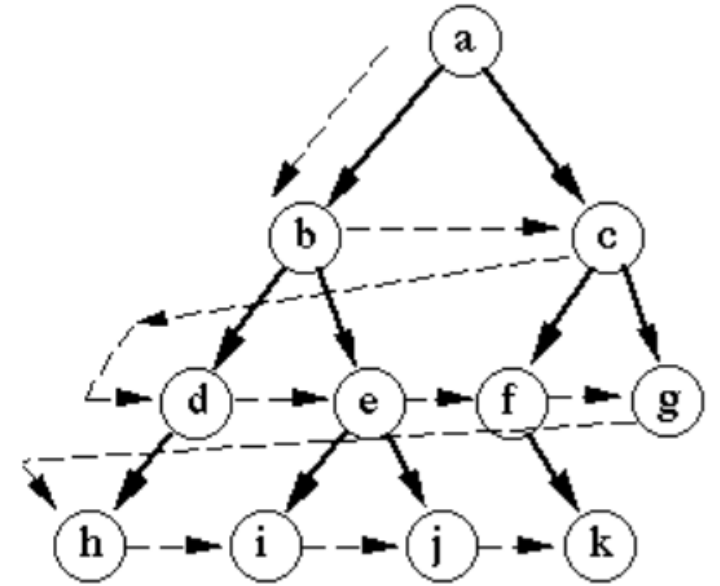
- Search algorithms require a data structure to keep track of the search tree
- For a node(Data structure with four components)
 - **node.STATE**: the state to **which the node corresponds**
 - **node.PARENT**: the node in the **tree that generated this node**
 - **node.ACTION**: the action that was **applied to the parent's state to generate this node**
 - **node.PATH-COST**: the **total cost of the path from the initial state to this node**. $g(n)$
- **Frontier(Data structure to store the frontier)(Queue-priority, FIFO, LIFO)**
- The **frontier** needs to be stored in such a way that the search algorithm **can easily choose the next node to expand** according to its preferred strategy. The appropriate data structure for this is a queue.
 - **IS-EMPTY**(frontier) returns true only if there are no nodes in the frontier.
 - **POP**(frontier) removes the top node from the frontier and returns it.
 - **TOP**(frontier) returns (but does not remove) the top node of the frontier.
 - **ADD**(node, frontier) inserts node into its proper place in the queue.

Uninformed Search (Blind Search)

- How do we decide which node from the frontier to expand next?
- Choose a node, n , with minimum value of some evaluation function, $f(n)$
- On each iteration **choose a node** on the frontier **with minimum $f(n)$ value**
- It returns it if its state is a goal state,
- otherwise apply EXPAND to generate child nodes.
- Each child node is added to the frontier if it has not been reached before, or is re-added if it is now being reached with a path that has a **lower path cost than any previous path**
- The algorithm returns either an indication of failure, or a node that represents a path to a goal.

Uninformed Search Strategies-Breadth-First Search

- An uninformed search algorithm is given **no clue** about how close a state is to the goal(s)
- When all actions have the **same cost**, an appropriate strategy is **breadth-first search**
- The root node is expanded first, Then all the successors of the root node are expanded next, and always added to the end of the OPEN list.
- When removing nodes from OPEN to be added to the CLOSED, check whether it is the goal node, if it is, then stops.
- A **first-in-first-out queue** will be faster than a priority queue



Breadth-First Search – Algorithm

1. Create a variable called NODE-LIST and set it to the initial state.
2. Until a goal state is found, or NODE-LIST is empty:
 - a) Remove the first element from NODE-LIST and call it E. If NODE-LIST was empty, then quit.
 - b) For element E do the following:
 - i. Apply the rule to generate a new state,
 - ii. If the new state is a goal state. quit and return this state.
 - iii. Otherwise, add the new state to the end of NODE-LIST



Uninformed Search Strategies-Breadth-First Search

Advantages of Breadth first search are:

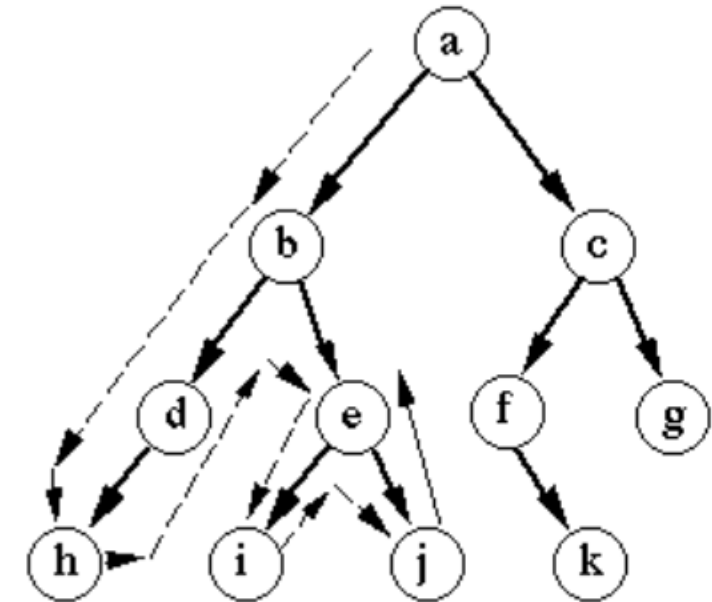
- One of the simplest search strategies
- BFS is Complete. If there is a solution, BFS is guaranteed to find it.
- If there are multiple solutions, then a minimal solution will be found

Disadvantages of Breadth first search are :

- The breadth first search algorithm cannot be effectively used unless the search space is quite small.

Uninformed Search Strategies-Depth-First Search

- In DFS, the state space is explored along branches depth-wise.
- It checks for the goal node along each branch.
- Since **DFS identifies the deeper level goals before the shallower level ones, it is not optimal.**
- **DFS is not complete also.** Because, if there is a loop in the one branch in the state space, even though we can find the goal node along another branch, still we can not go to that branch due to the loop.



Depth-First Search Algorithm



1. If the initial state is a goal state, quit and return success.
2. Otherwise, do the following until success or failure is signaled:
 - a) Generate a successor, E, of the initial state. If there are no more successors, signal failure.
 - b) Call Depth-First Search with E as the initial state.
 - c) If success is returned, signal success. Otherwise continue in this

Depth-First Search – Advantages and Disadvantages

Advantages of Depth-first search are:

- Depth-first search requires less memory since only the nodes on the current path are stored.
- The depth-first search may find a solution without examining much of the search space at all.

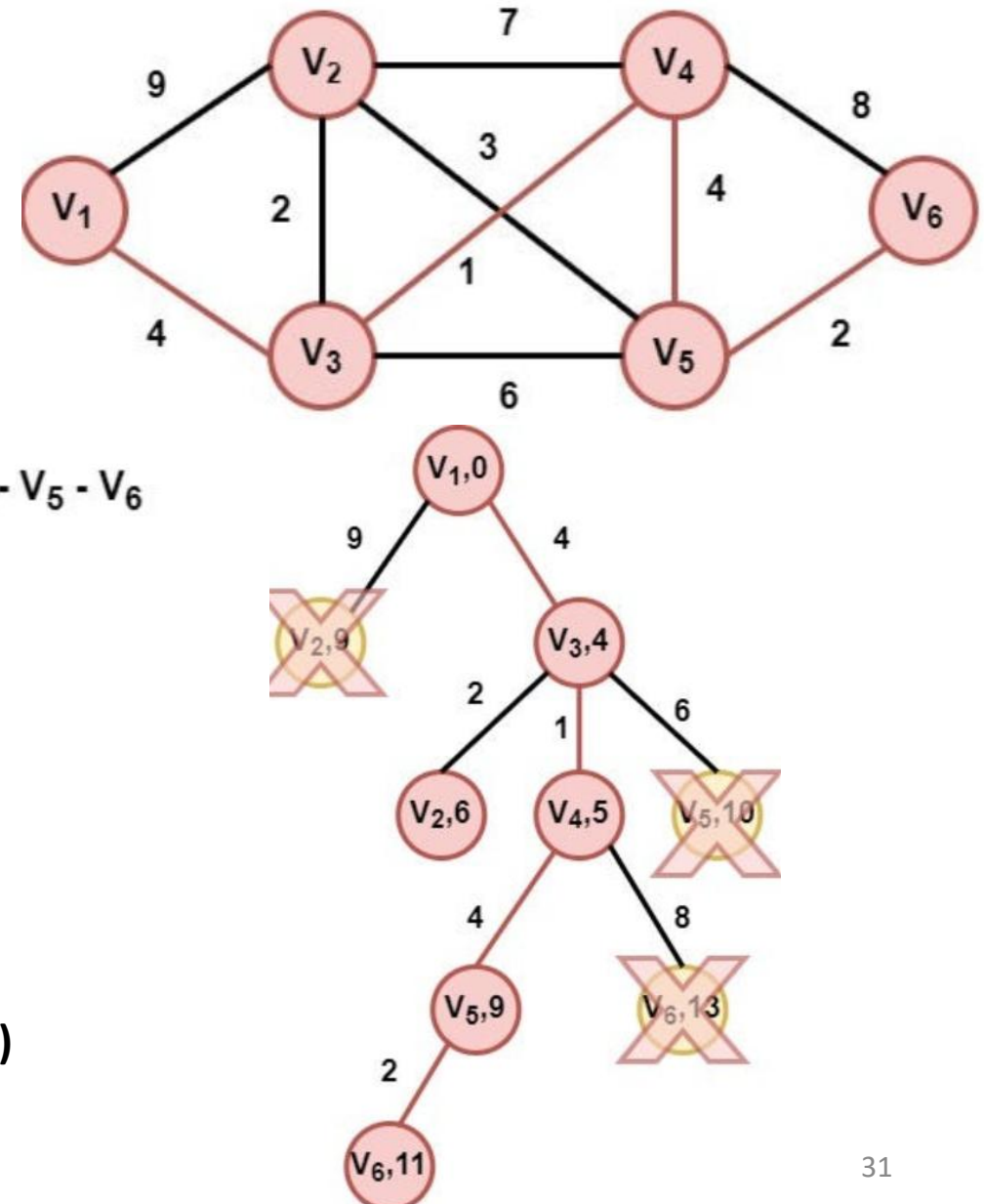
Disadvantages of Depth-first search are:

- May find a sub-optimal solution (one that is deeper or more costly than the best solution)

Uniform Cost Search

- In UCS, each action (edge) from one node to another node is given a value.
- Each node is associated with a **total path cost** which is calculated from the 'start node' to the particular node.
- In UCS, thus **lowest total path cost is considered** instead of depth.

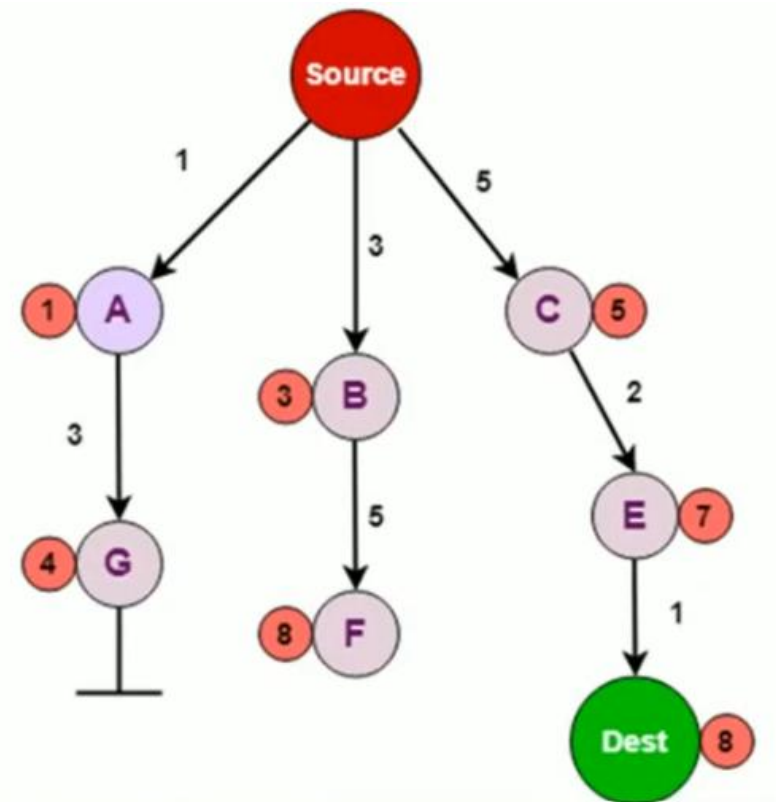
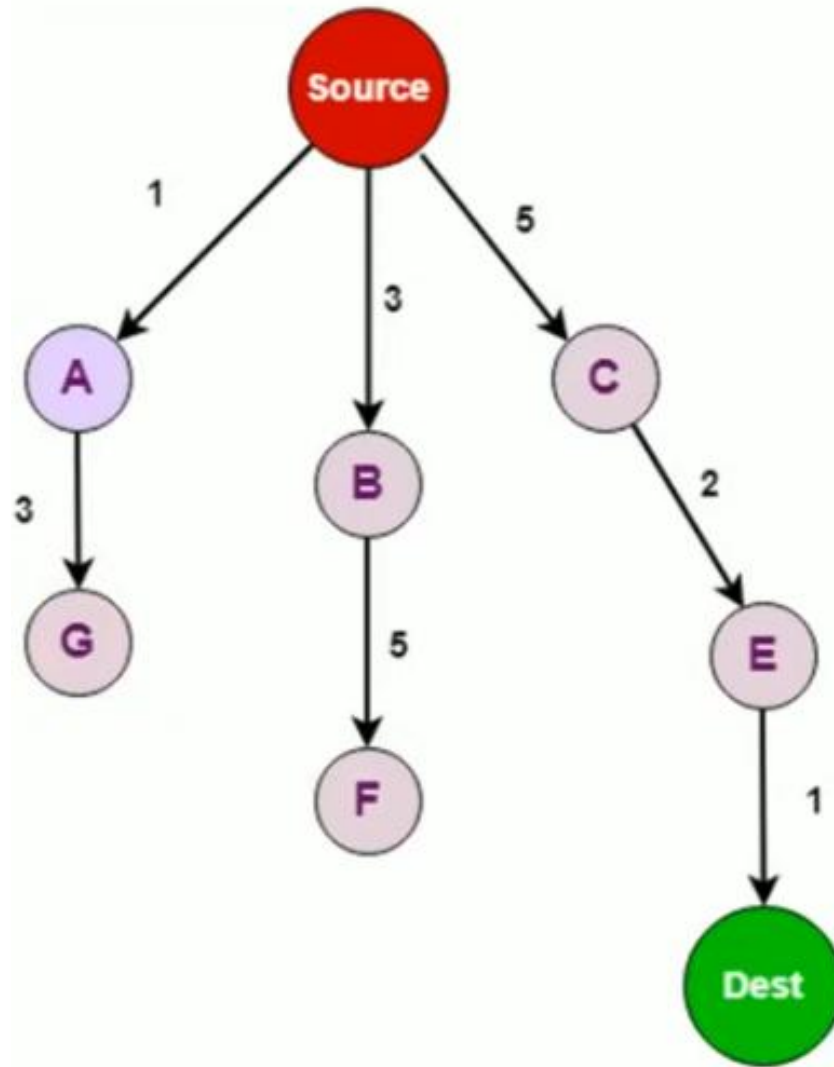
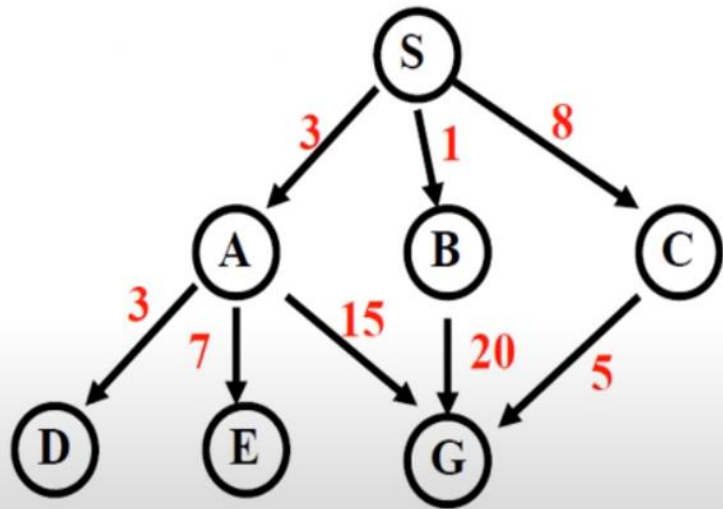
$$\begin{aligned} F(n) &= g(n) \\ &= \text{Summation}(\text{total path cost to each node from start node}) \\ &= 11 \end{aligned}$$



UCS Algorithm

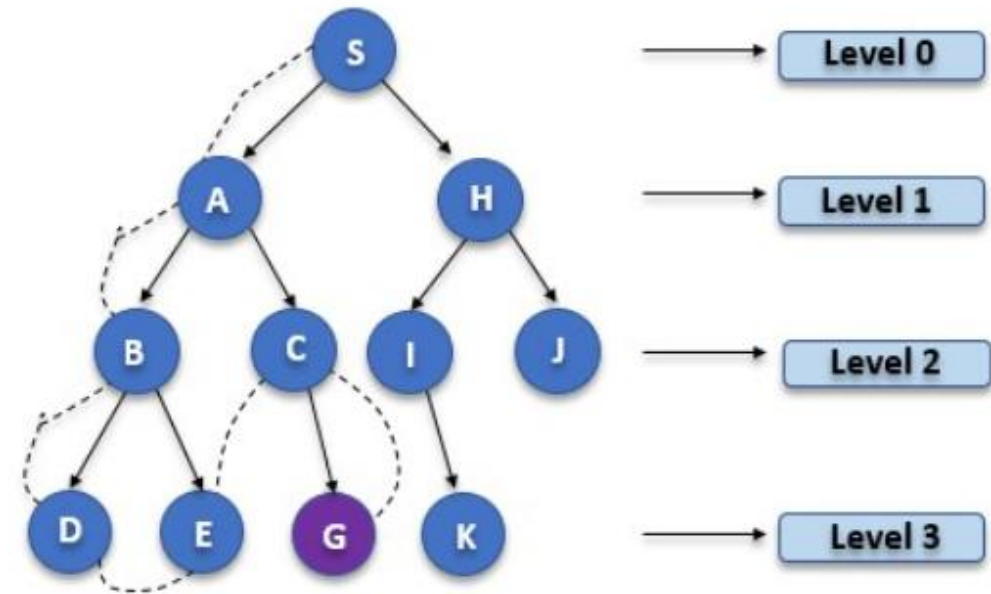
1. Insert the root node into the priority queue.
2. Remove the element with the highest priority.
3. If the removed node is the goal node:
 1. Print the total cost and stop the algorithm.
4. Else:
 1. Enqueue all the children of the current node to the priority queue, with their cumulative cost from the root as priority and the current node to the visited list.

If all the edges are of same cost. Then?

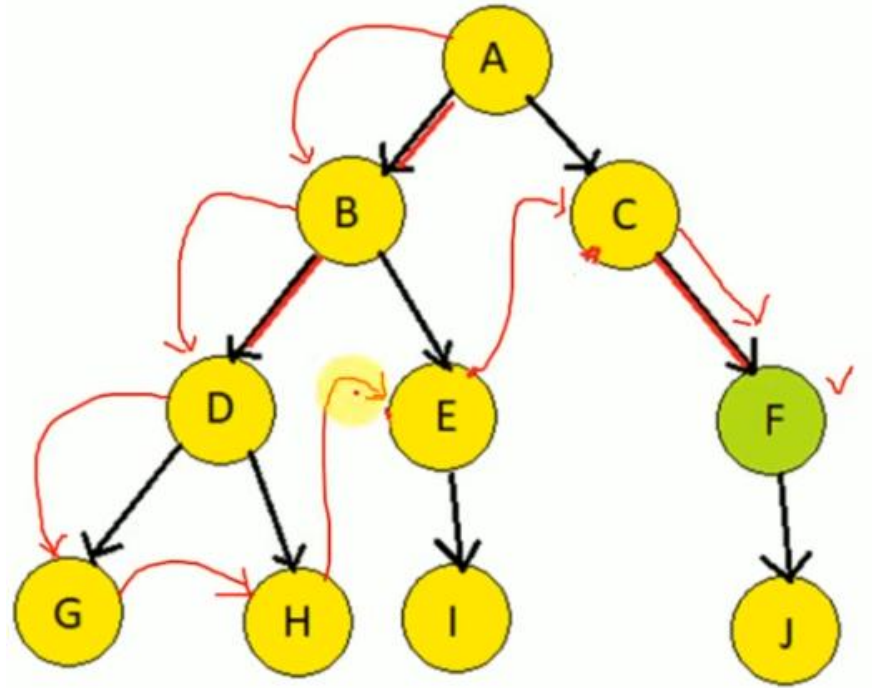


Depth Limited Search

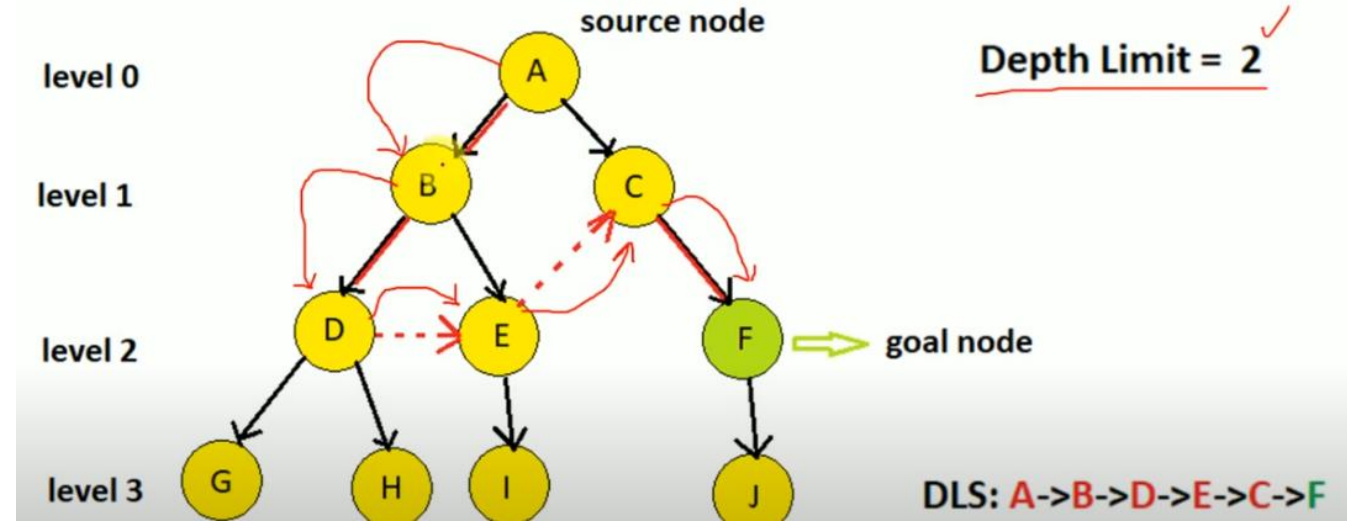
- DFS don't provide an optimal solution.
- **Set a depth limit** so that beyond that tree is not searched. So, if the goal is at a shallower level, then it is found quickly.
- When the **depth level is set, but the goal node is at other level**, so the search will **never be able to find the goal node**.
- In **DLS**, it is **neither complete nor optimal** because not all the nodes in the graph are checked and goal nodes cannot be found effectively.



In general DFS, if A is the initial node and F is the goal node then



DFS: A→B→D→G→H→E→I→C→F



DLS: A→B→D→E→C→F

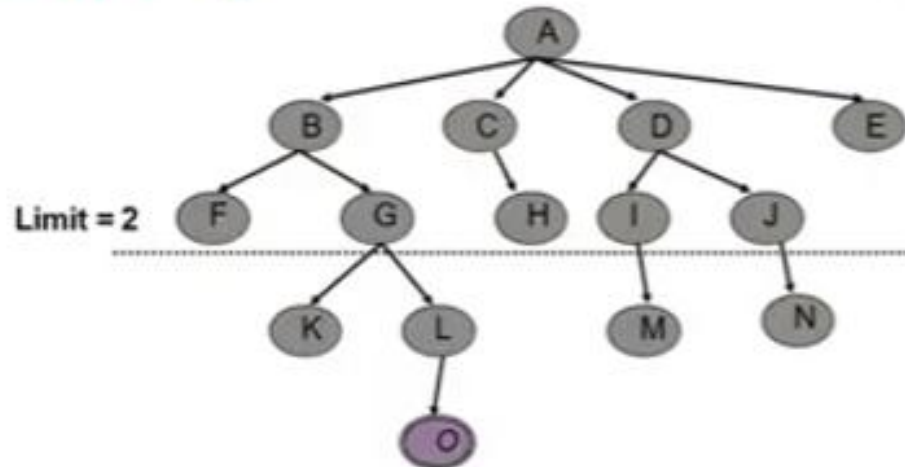
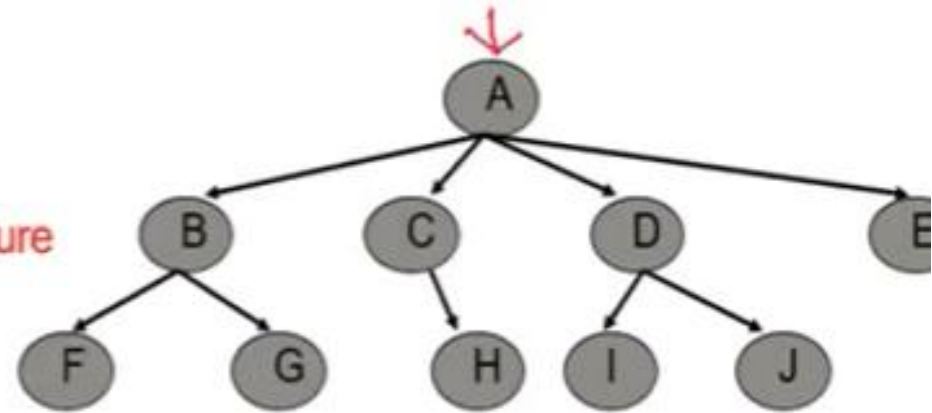
In Depth limited search, if A is the initial node and F is the goal node then

Depth Limited Search (DLS)...

- DLS algorithm returns **Failure (no solution)**
- The reason is that the goal is beyond the limit (Limit = 2): **the goal depth is (d=4)**

- A,B,F,
- G,
- C,H,
- D,I
- J,
- E, **Failure**

Limit = 2



It solves infinity problem,
but introduces many other
problems.

Iterative deepening depth-first search (IDS)

- In IDS, by increasing the depth limit iteratively and execute the DFS until the goal node is found.
- The use of DFS helps in the lesser usage of memory to store nodes.
- And since the state-space is explored iteratively (layer-by-layer) if the goal node is in a shallower level (optimal), it is found first before going deeper, and all the nodes in a layer are checked completely (completeness).
- **IDS possesses the advantage of both DFS (by consuming less memory space) and BFS (complete and optimal)** and it has eliminated the problems faced by DLS.
- Iterative deepening search may seem wasteful because states are generated multiple times. It turns out this is not too cost

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
  for depth = 0 to  $\infty$  do
    result  $\leftarrow$  DEPTH-LIMITED-SEARCH(problem, depth)
    if result  $\neq$  cutoff then return result
```

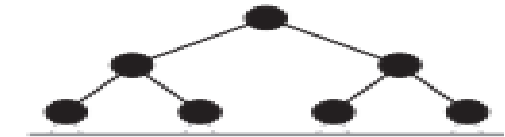
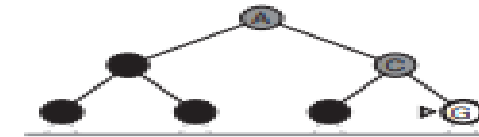
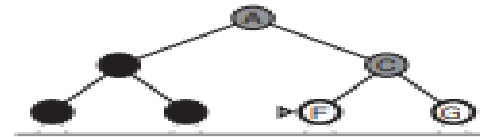
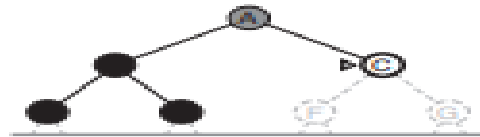
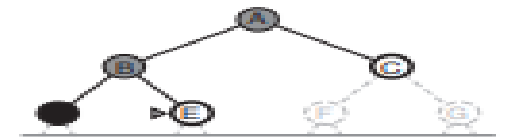
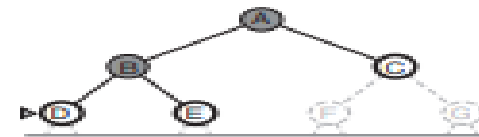
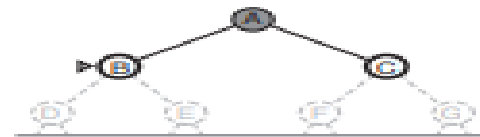
Limit = 0



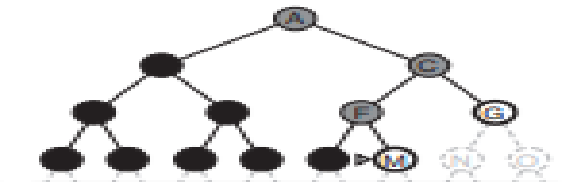
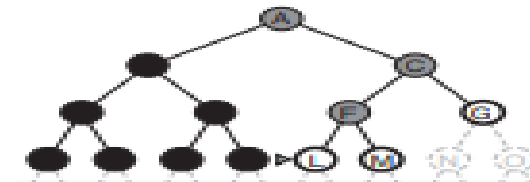
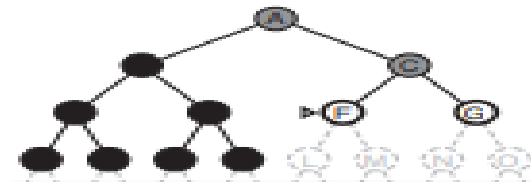
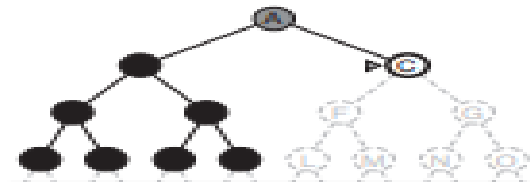
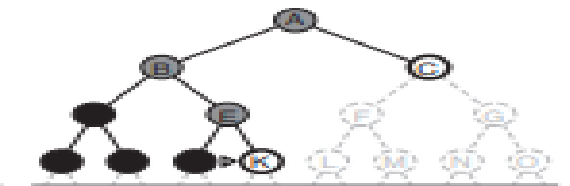
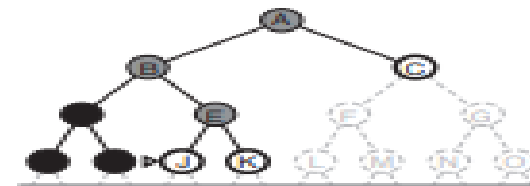
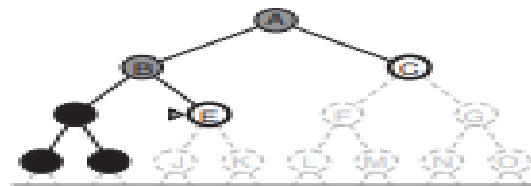
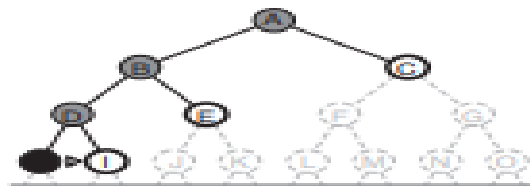
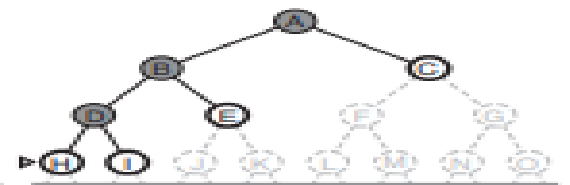
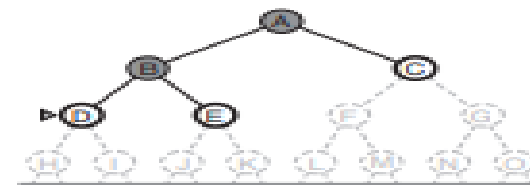
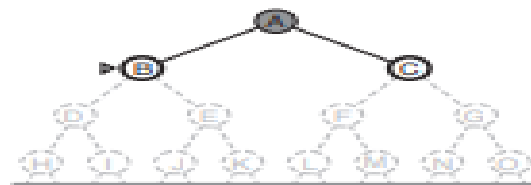
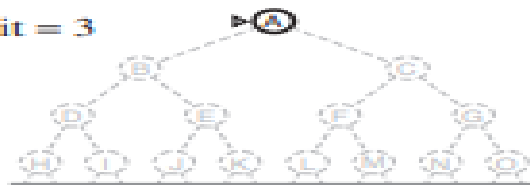
Limit = 1

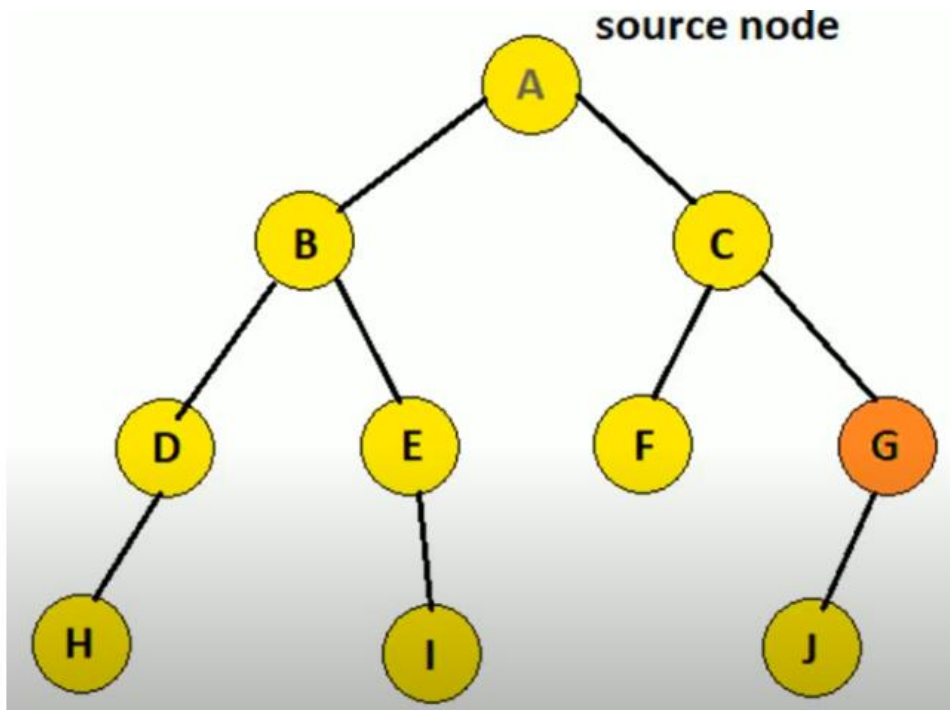


Limit = 2



Limit = 3

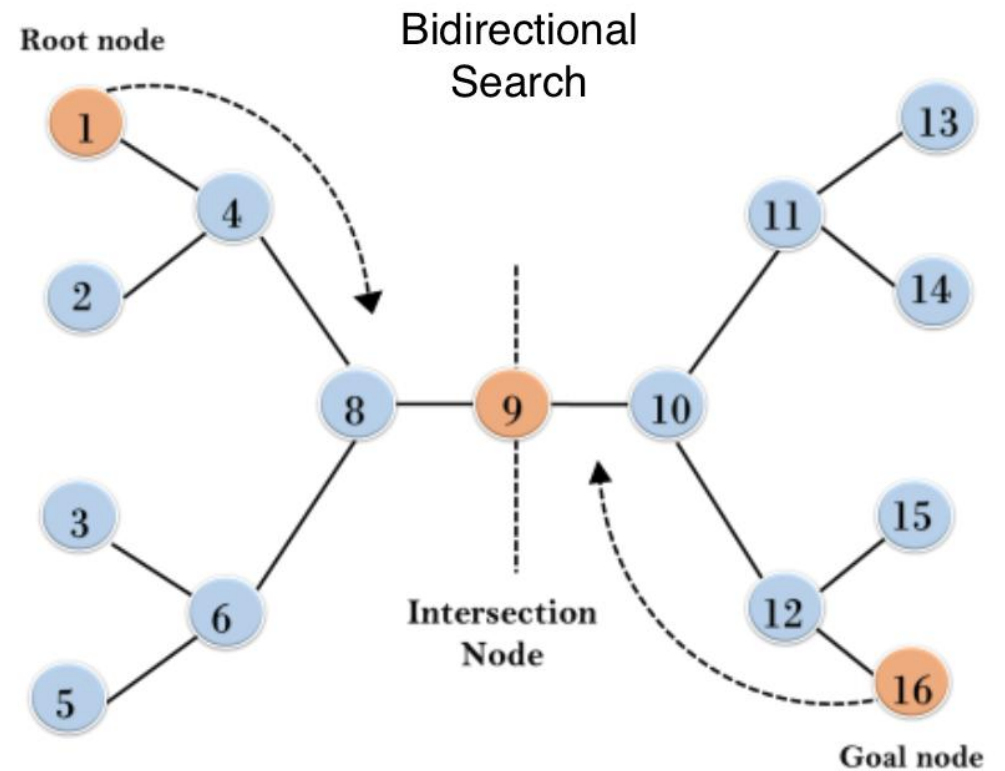
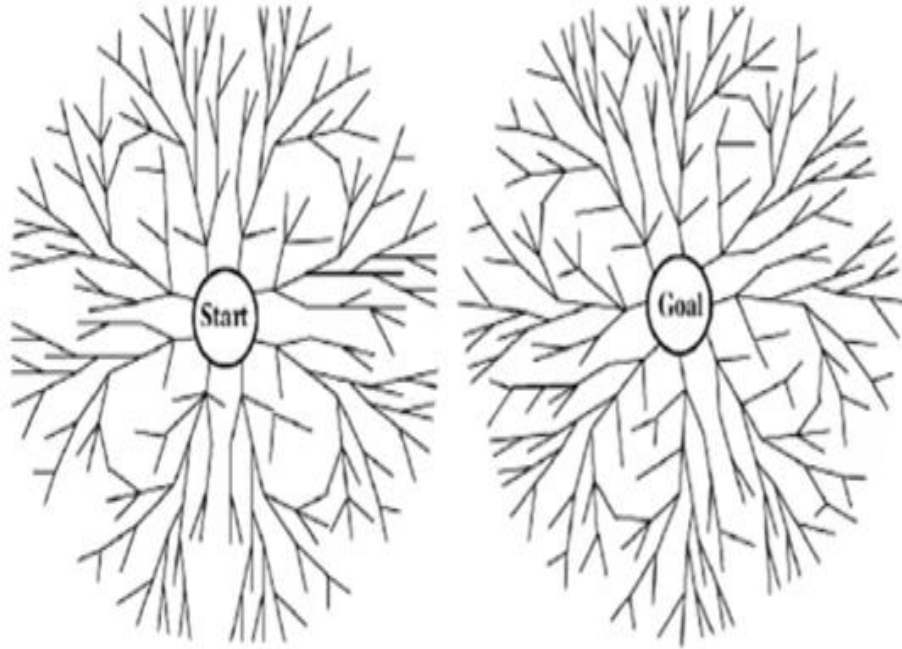




- By DFS it is **ABDHEICFG**
- By DLS, with limit-2 is **ABDECFG**
- If goal state is not there in limit-2, the limit will be extended iteratively.

Bi-directional Search

- Bi-directional searching, there are **two searching**, one from the start state toward the goal and another from the goal state toward the start.
- This algorithm **Stops** when both search meet in the middle.
- Can (sometimes) lead to finding a solution more quickly.



Comparison of Uninformed Search Algorithms

Criterion	Breadth-First	Uniform-Cost	Depth-First	Depth-Limited	Iterative Deepening	Bidirectional (if applicable)
Complete?	Yes ^a	Yes ^{a,b}	No	No	Yes ^a	Yes ^{a,d}
Time	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(b^m)$	$O(b^\ell)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lceil C^*/\epsilon \rceil})$	$O(bm)$	$O(b\ell)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes ^c	Yes	No	No	Yes ^c	Yes ^{c,d}

Figure 3.21 Evaluation of tree-search strategies. b is the branching factor; d is the depth of the shallowest solution; m is the maximum depth of the search tree; ℓ is the depth limit. Superscript caveats are as follows: ^a complete if b is finite; ^b complete if step costs $\geq \epsilon$ for positive ϵ ; ^c optimal if step costs are all identical; ^d if both directions use breadth-first search.

Informed (Heuristic) Search Strategies

Domain-specific hints about the location of goals



Find solutions more efficiently than an uninformed strategy

The hints come in the form of a heuristic function, denoted $h(n)$

Where $h(n) :=$ estimated cost of the cheapest path from the state at node n to a goal state.

Informed (Heuristic) Search Strategies

- Heuristic is a rule of thumb which **leads us to the probable solution.**
- Heuristic narrows down the search for solution and **eliminates the wrong options.**
- **Non polynomial problem will be converted to polynomial problem** in heuristic strategy.
- The **best (good) solution is guaranteed.**
- **Optimal solution is not guaranteed.**
- The general approach we consider is called **best-first search.**
- It uses **Euclidean (straight line) and Manhattan distance** to find the distances between to points.
- Heuristics are not theories but some common-sense experience like information.

Best First Search

- The primary objective of the best-first search is to expand the most promising nodes by relying on **heuristic function**.
- The **best first search can switch between BFS and DFS** by gaining the advantages of both algorithms.

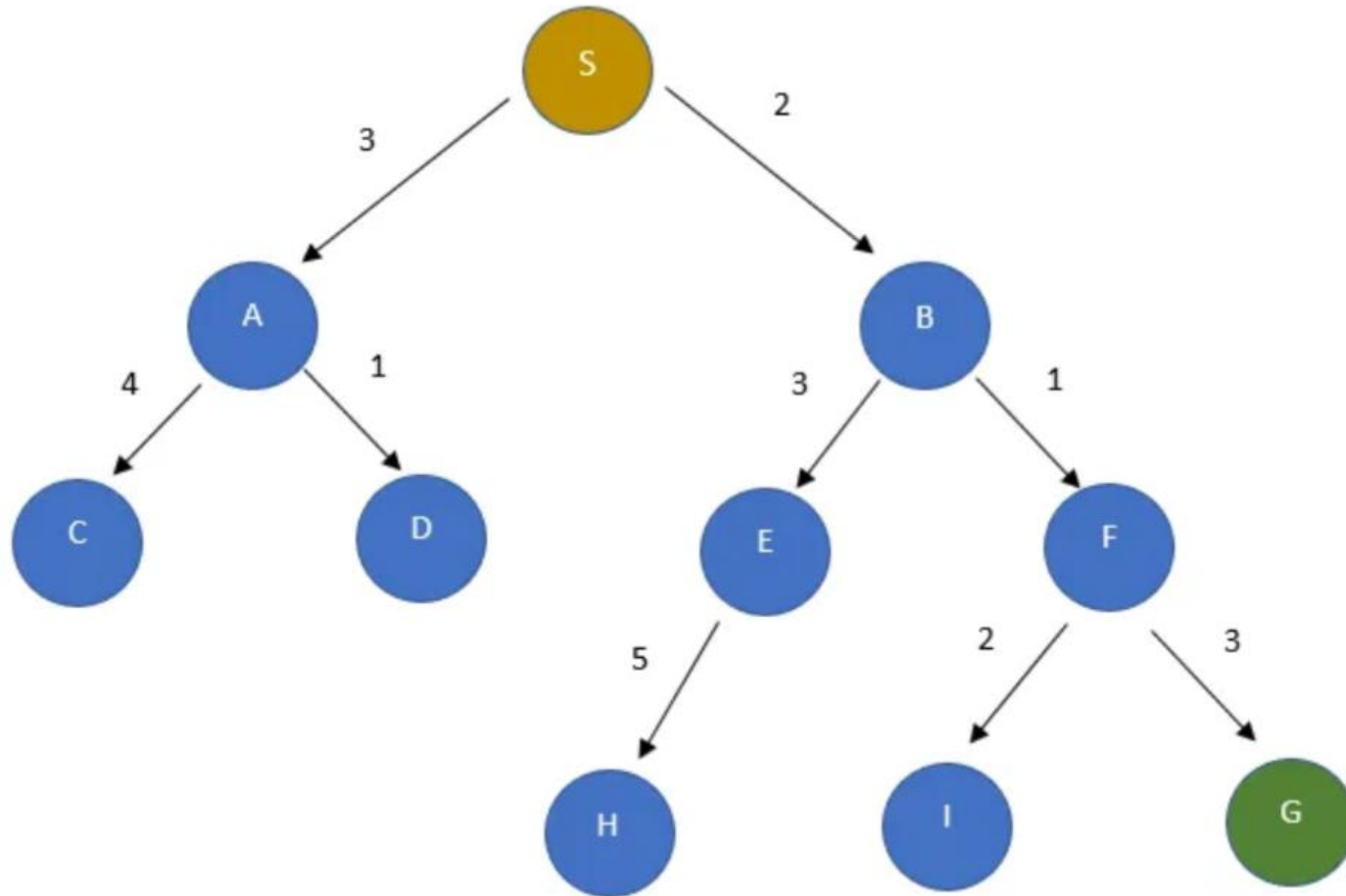
To implement a search procedure, we need to use two lists of nodes:

1. **OPEN**: Nodes that have been generated and had the heuristic function applied to them, but have not yet been explored (i.e., had their successors generated).
2. **CLOSED**: Nodes that have already been examined. These nodes need to be kept in memory if we want to search a graph rather than a tree, since whenever a new node is generated, we need to check whether it has been examined before.

Algorithm

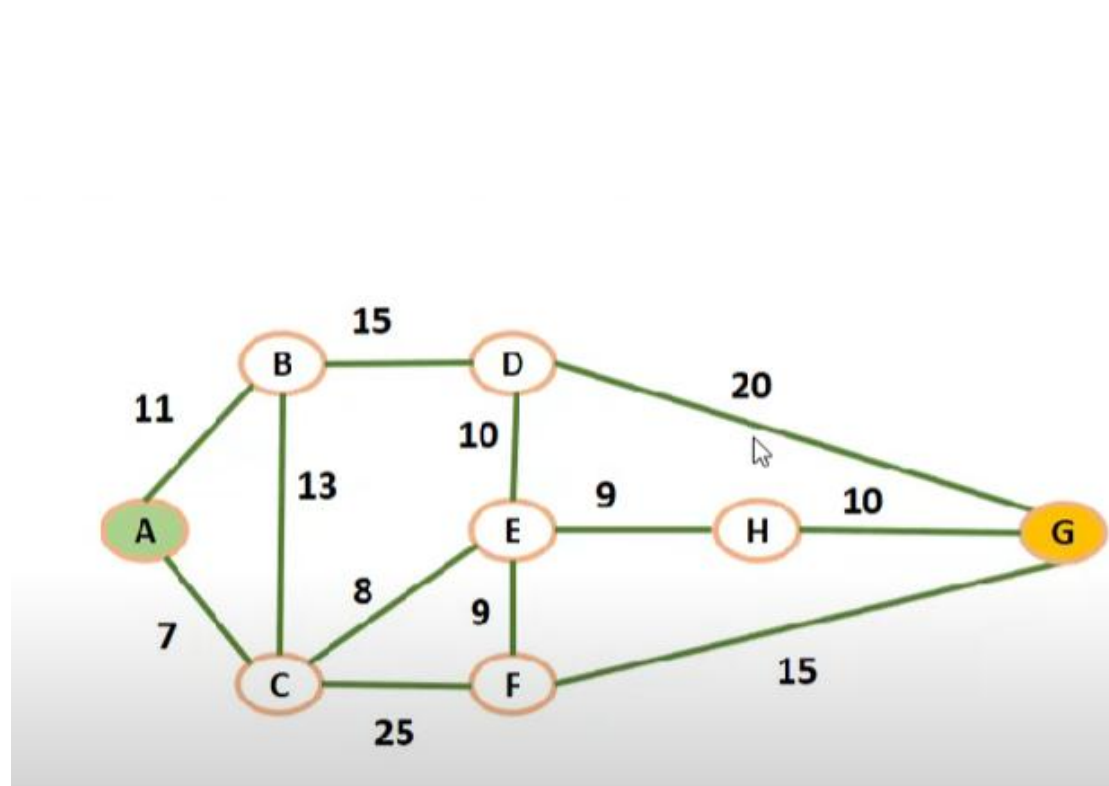
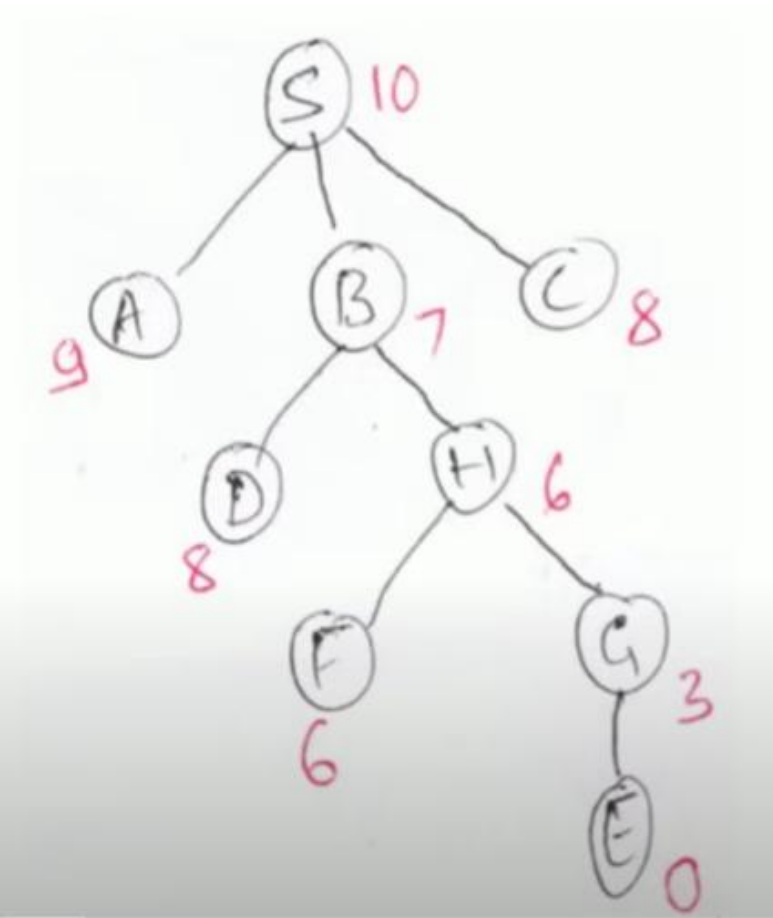
1. Create 2 empty lists: OPEN and CLOSED
2. Start from the initial node (say N) and put it in the '**ordered**' **OPEN list**
3. Repeat the next steps until the GOAL node is reached OR OPEN list is empty
 - Select the best node (say N) in the OPEN list and move it to the CLOSED list. Also, capture the information of the parent node.
 - If N is a GOAL node, then exit the loop returning 'True'. The solution can be found by backtracking the path.
 - If N is not the GOAL node, expand node N to generate the 'immediate' next nodes linked to node N and add all those to the OPEN list.
4. Reorder the nodes in the **OPEN list in ascending order**.

Example



node	H (n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0

Hence the final solution path will be: $S \rightarrow B \rightarrow F \rightarrow G$



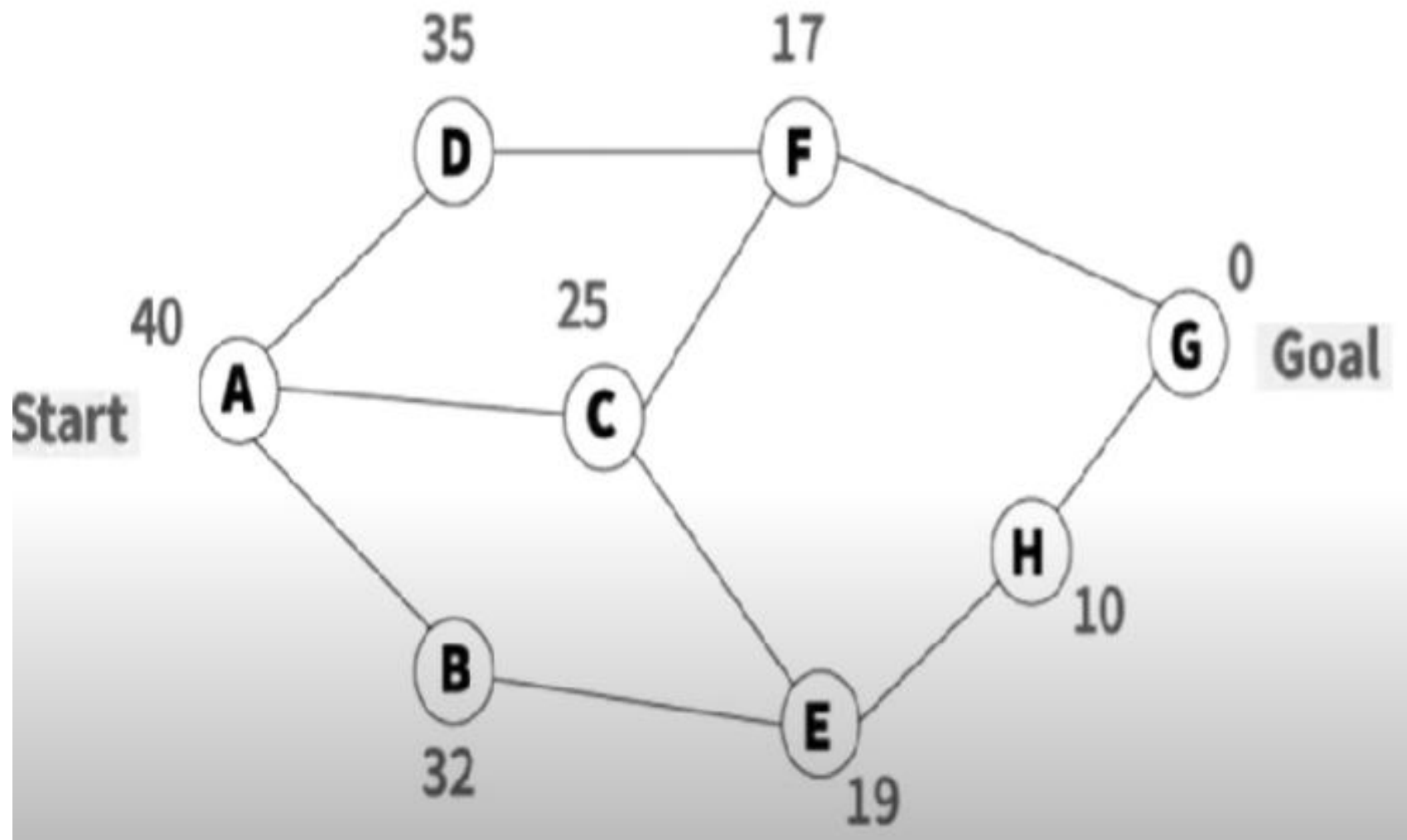
Straight line distance	
A – G	40
B – G	32
C – G	25
D – G	35
E – G	19
F – G	17
H – G	10
G – G	0

Greedy Best-First Search

- "greedy" algorithm considers *only* the best short-term choice at each juncture considering the **locally best choice** with **lowest $h(n)$** value
- The node that appears to be closest to the goal—on the grounds that this is likely to lead to a solution quickly
- Straight-line distance h_{SLD}
- Evaluation function, $f(n)$ —The evaluation function is construed as a cost estimate, so the node with the lowest evaluation is expanded first.
- heuristic function $h(n)$ = estimated cost of the cheapest path from the state at node n to a goal state.
- Thus, it evaluates nodes by using the heuristic function, which is: $f(n) = h(n)$

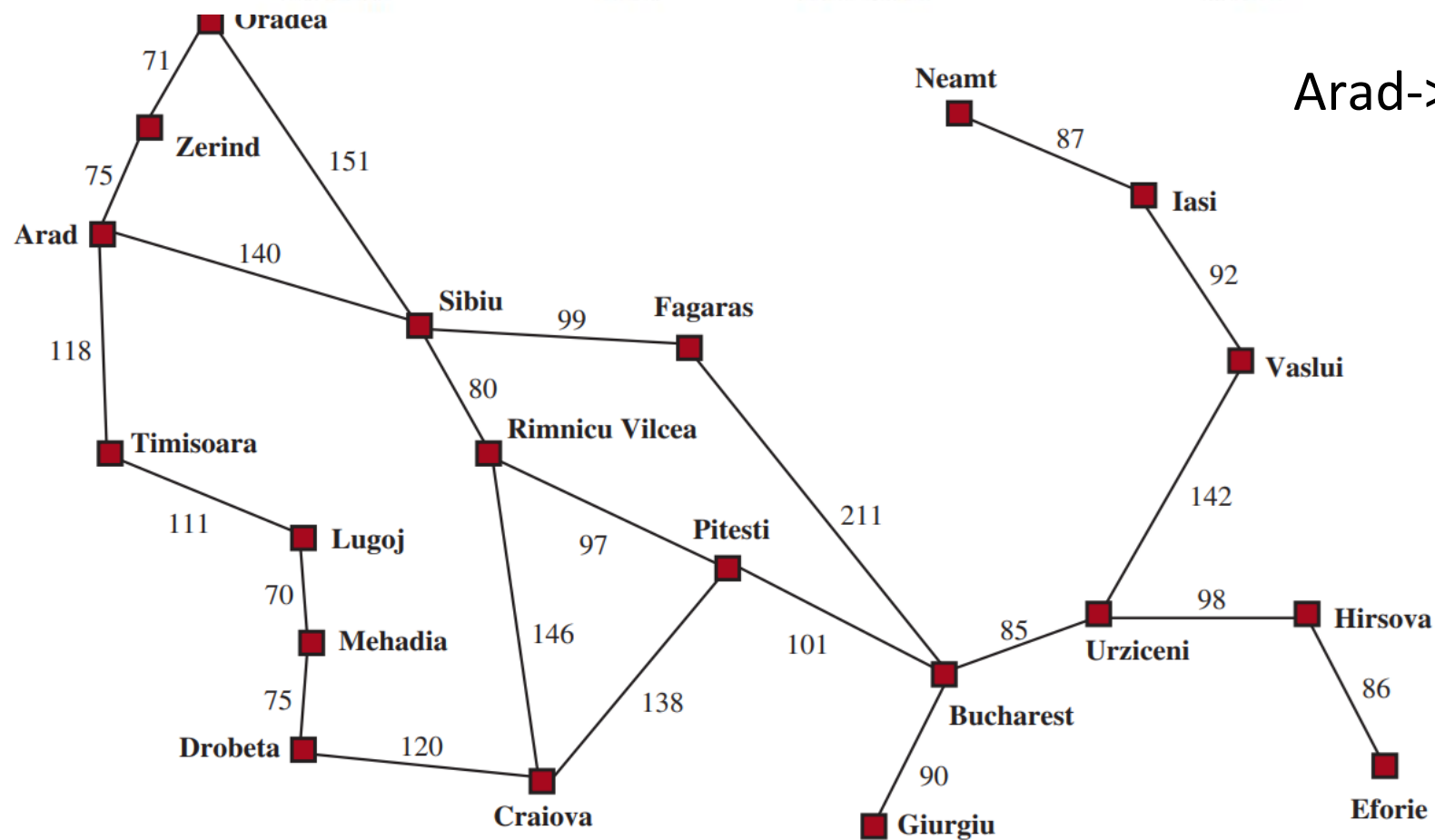
Algorithm: Greedy Best-First Search

1. Start with OPEN containing just the initial state.
2. Until a goal is found or there are no nodes left on OPEN do:
 - a) Pick the best node on OPEN.
 - b) Generate its successors.
 - c) For each successor do:
 - i. If it has not been generated before, evaluate it, add it to OPEN, and **record its parent**.
 - ii. If it has been generated before, change the parent if this new path is better than the previous one. In that case, update the cost of getting to this node and to any successors that this node may already have.



Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244

Mehadia	241
Neamt	234
Oradea	380
Pitesti	100
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

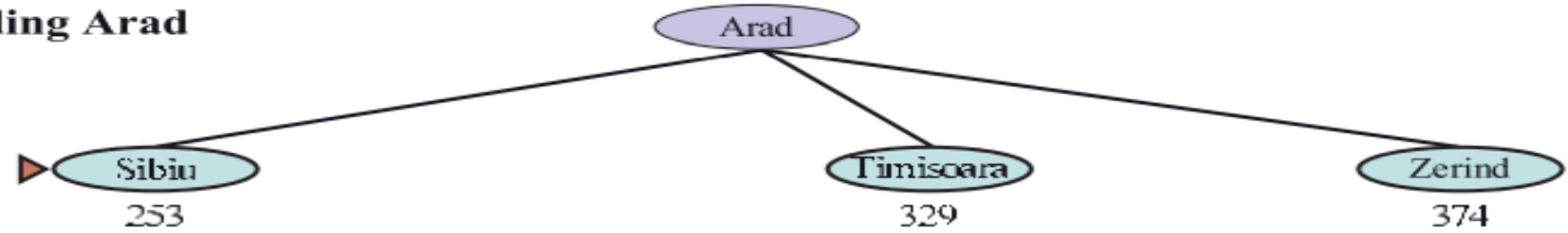


Arad->Bucharest

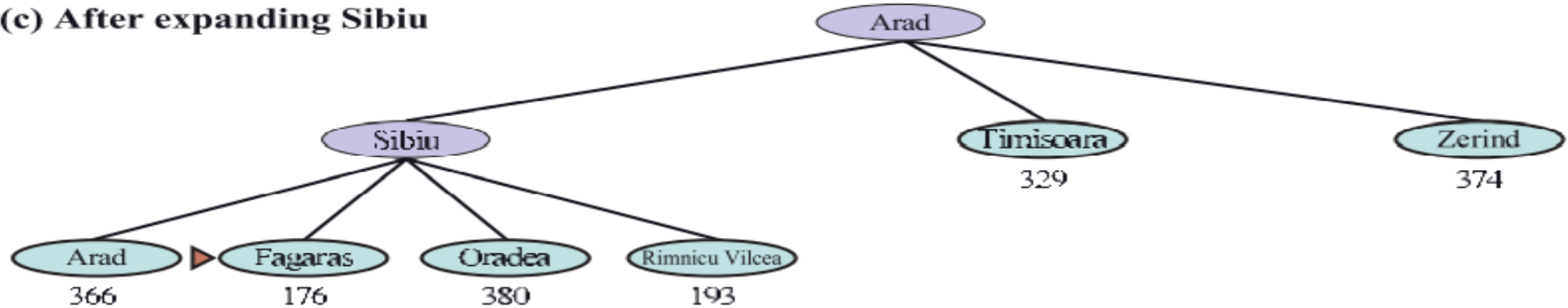
(a) The initial state



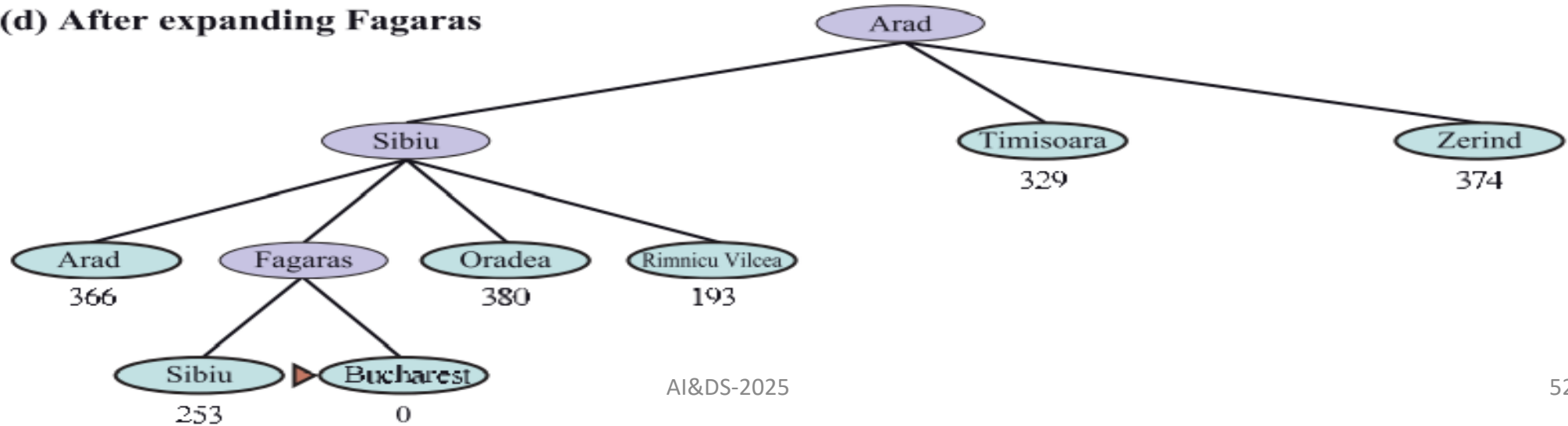
(b) After expanding Arad



(c) After expanding Sibiu



(d) After expanding Fagaras



Greedy Best First Search

1. **Complete?** No - it can get stuck in loops.
2. **Time?** $O(bm)$, but a good heuristic can give dramatic improvement.
3. **Space?** $O(bm)$ - it keeps all nodes in memory.
4. **Optimal?** No. For example, the path Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Virea $\rightarrow$ Pitesti $\rightarrow$ Bucharest is shorter than the path chosen by greedy best-first search.

where m is the maximum depth of the search space

A* Search Algorithm

- In the A* algorithm, we consider **both path cost and heuristics**.
- In A* the $f(n)$ function comprises two components: **path cost $[g(n)]$ and heuristic value $[h(n)]$** .

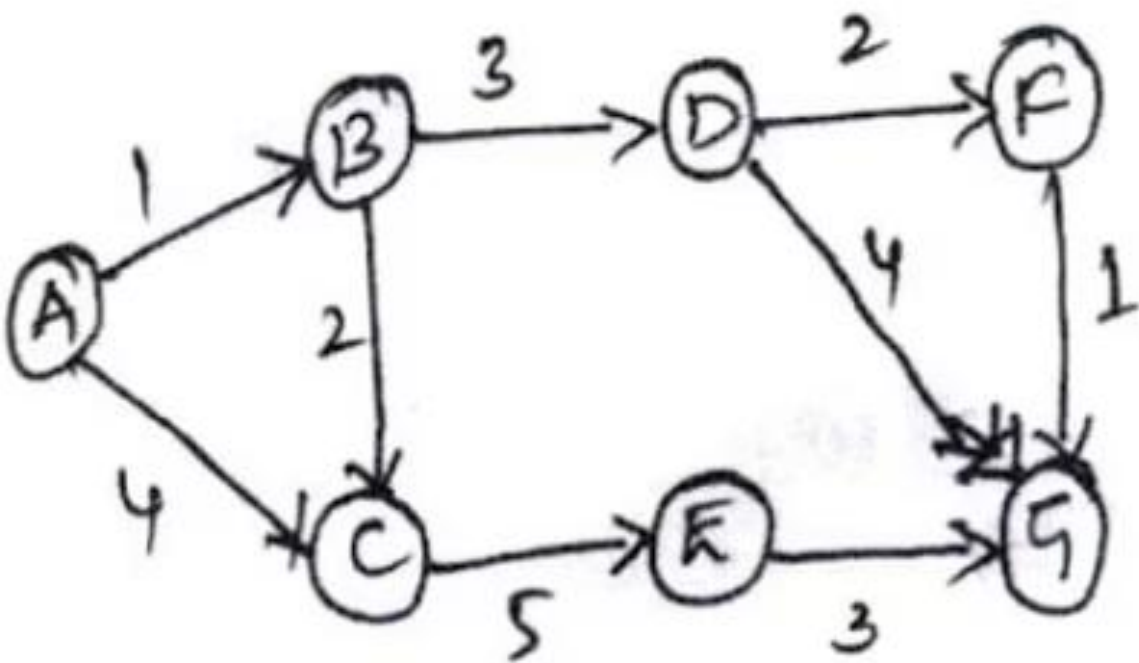
$$f(n)_a = g(n)_a + h(n)_a$$

$f(n)_a$ = Evaluation value at the particular node (node 'a')

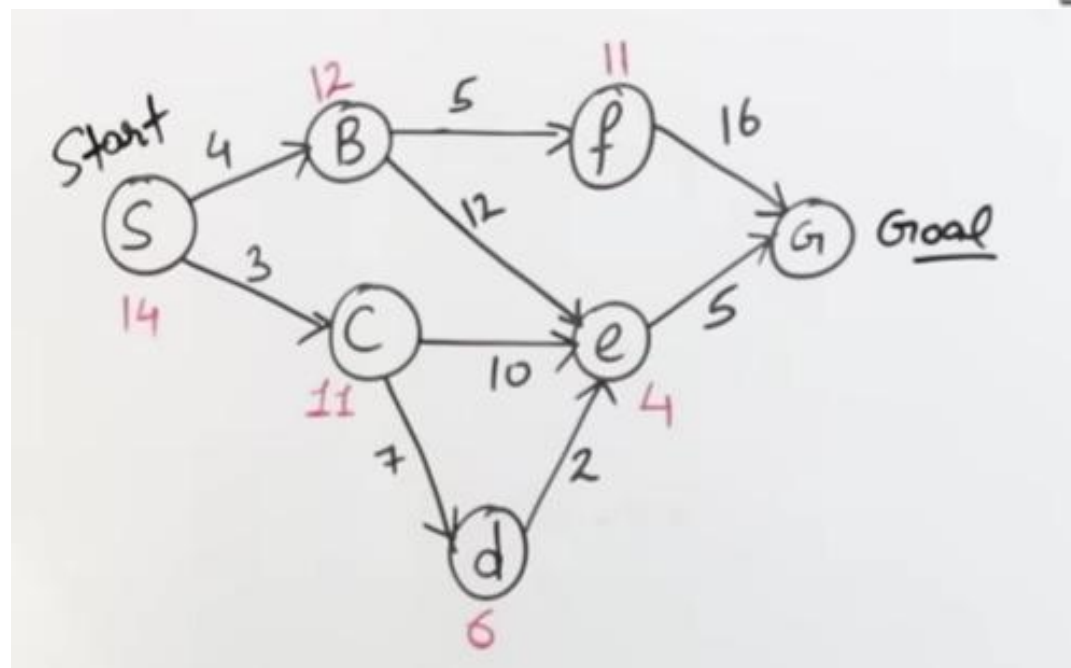
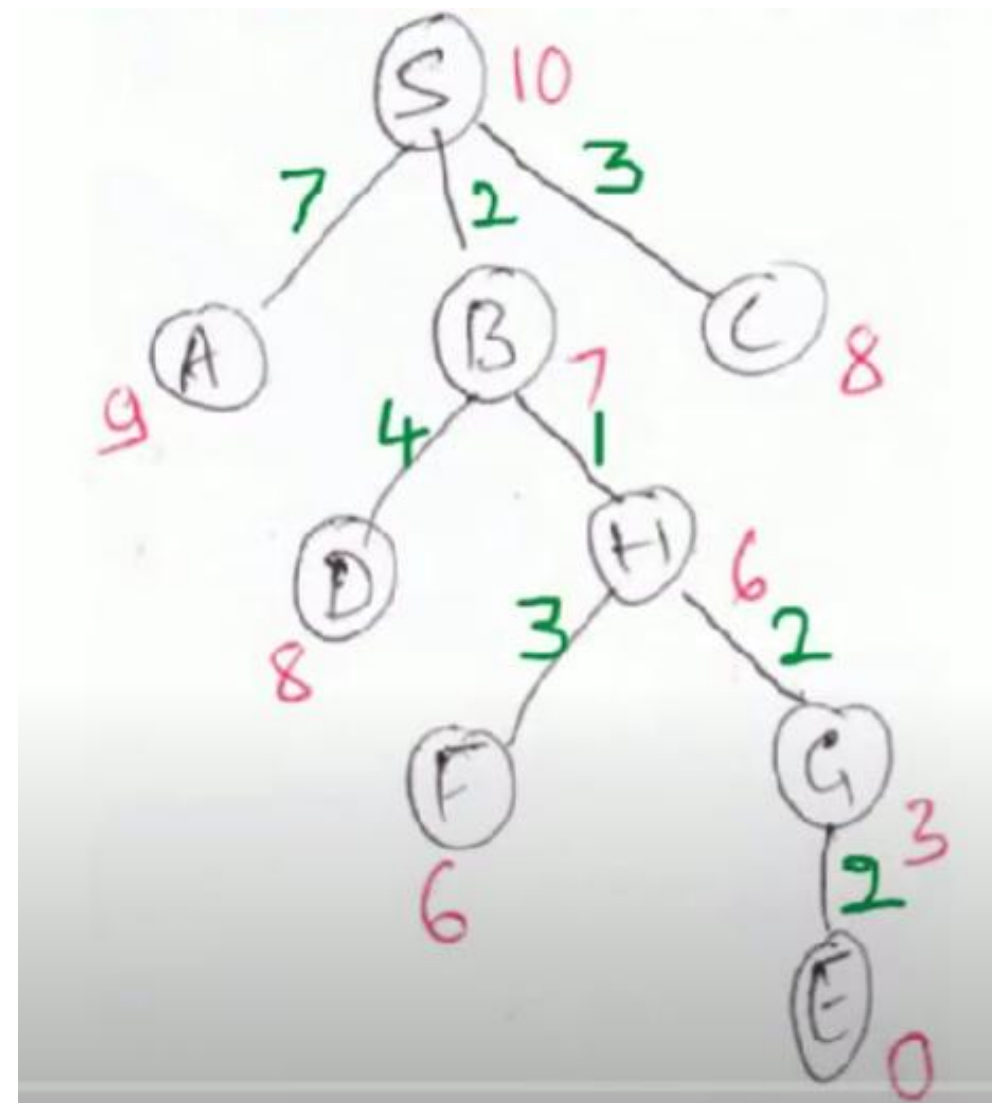
$g(n)_a$ = Total path cost from start node to particular node (node 'a')

$h(n)_a$ = The heuristic value of the particular node (node 'a')

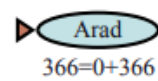
- Greedy Best First Search is efficient but not optimal or complete, as it focuses solely on minimizing the heuristic $h(n)$.
- Uniform Cost Search is optimal and complete, but not efficient, as it only considers the actual cost $g(n)$ from the initial state.
- **A* Search combines the strengths of both Greedy Best First Search and Uniform Cost Search**, resulting in an efficient, complete, and optimal algorithm.



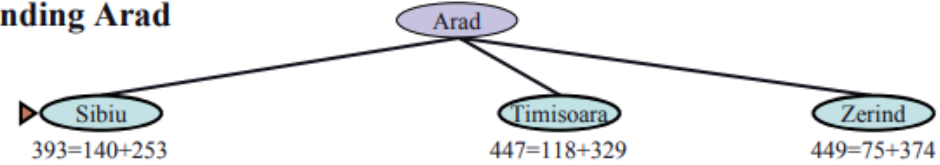
A	5
B	6
C	4
D	3
E	3
F	1
G	0



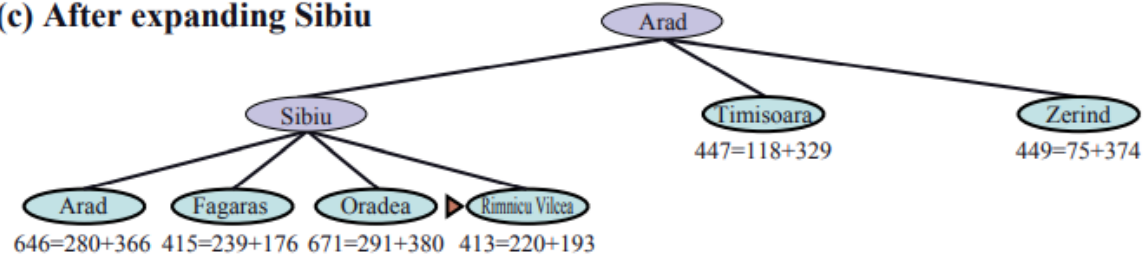
(a) The initial state



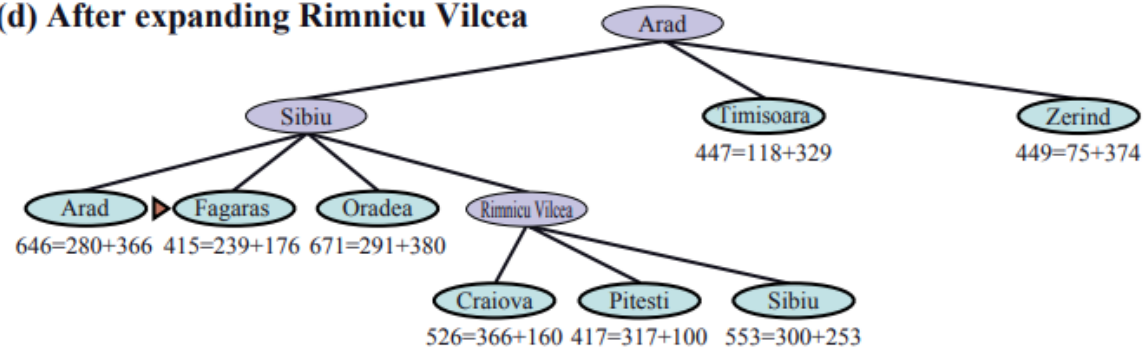
(b) After expanding Arad



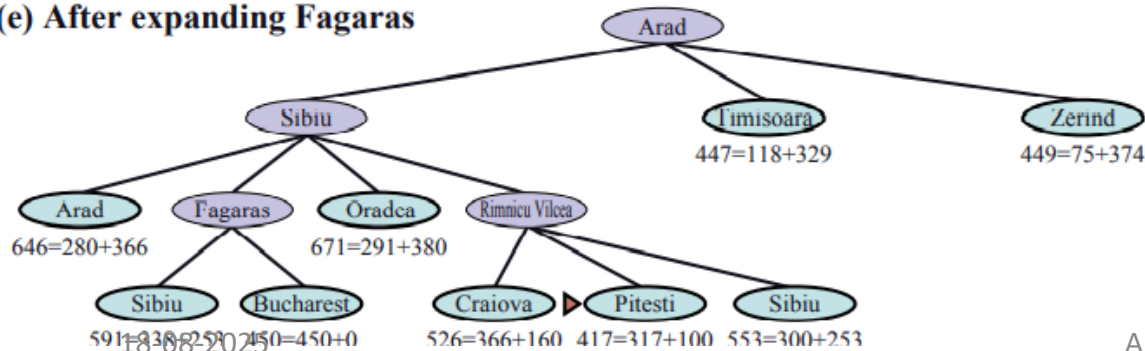
(c) After expanding Sibiu



(d) After expanding Rimnicu Vilcea



(e) After expanding Fagaras



Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

Conditions for optimality: Admissibility and consistency

- The first condition requires for optimality is that $h(n)$ be an **admissible heuristic**.
- **Admissible heuristic** is one that **never overestimates the cost to reach the goal** ($H(n) \leq H$)
Here H is total path cost)
- $g(n)$ is the actual cost to reach n along the current path
- $f(n) = g(n) + h(n)$
- Admissible heuristics are by nature optimistic because they think the cost of solving the problem is less than it actually is.
- For **consistency** it is $h(x) \leq d(x, y) + h(y)$. [triangle inequality property]

Algorithm Type	Uninformed						Informed	
Criterion	BFS	DFS	DLS	IDS	BDS	UCS	Greedy	A*
Complete	Yes	No	No	Yes	Yes	Yes	No	Yes
Optimal	Yes	No	No	Yes	Yes	Yes	No	Yes
Time Complexity	$O(b^d)$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{\frac{d}{2}})$	$O(b^d)$	$O(b^m)$	$O(b^d)$
Space Complexity	$O(b^{d+1})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{\frac{d}{2}})$	$O(b^{d+1})$	$O(b^m)$	$O(b^d)$
b – branching factor d – depth of the shallowest goal m – max. depth of search tree l – depth limit of the search								

Problem Reduction (Decomposition)

- Problem reduction is a key method in artificial intelligence (AI) that helps solve complex issues.
- It works by breaking down big problems into smaller, more manageable parts.
- This approach makes it easier for AI systems to find solutions.

Formulations:

AND/OR Graphs

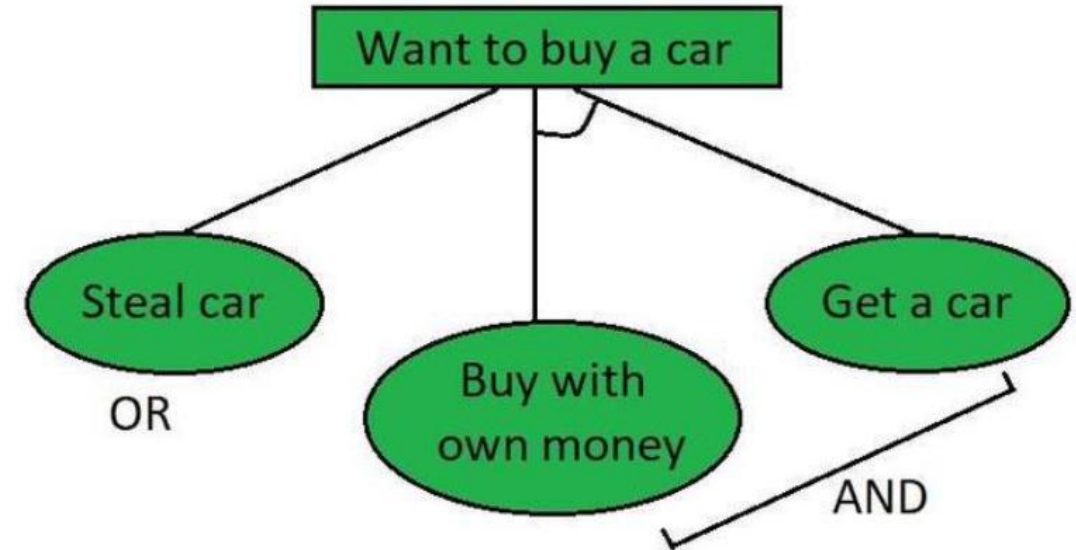
- An OR node represents a choice between possible decompositions
- An AND node represents a given decomposition

Game Trees

- Max nodes represent the choice of my opponent
- Min nodes represent my choice

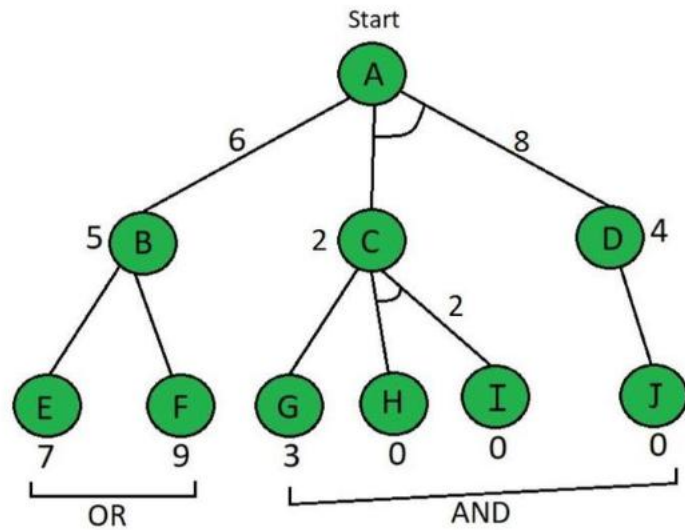
AO* algorithm

- The AO* algorithm is an advanced search algorithm utilized in artificial intelligence, particularly in problem-solving and decision-making contexts.
- It is an **extension of the A* algorithm**.
- **AO algorithm*** (short for "And-Or Star") is a powerful **best-first search** method used to solve problems that can be represented as a **directed acyclic graph (DAG)**.
- The AND nodes represent tasks where all child nodes must be satisfied, while OR nodes offer multiple alternative paths where only one child node needs to be satisfied to achieve the goal.



1. The AO* algorithm works on the formula: **$f(n) = g(n) + h(n)$** where,

1. $g(n)$: The actual cost of traversal from initial state to the current state.
2. $h(n)$: The estimated cost of traversal from the current state to the goal state.
3. $f(n)$: The actual cost of traversal from the initial state to the goal state.



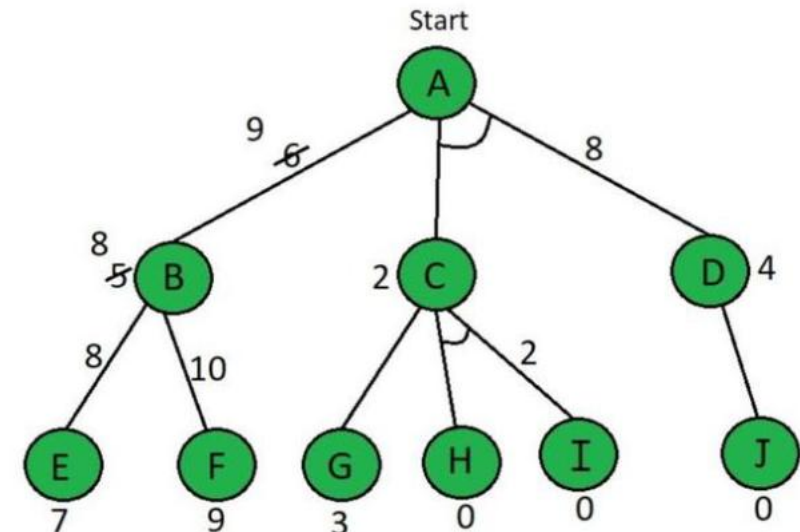
$$\begin{aligned} f(B \rightarrow E) &= g(E) + h(E) \\ &= 1 + 7 \\ &= 8 \end{aligned}$$

$$\begin{aligned} f(B \rightarrow F) &= g(F) + h(F) \\ &= 1 + 9 \\ &= 10 \end{aligned}$$

$$\begin{aligned} f(A \rightarrow B) &= g(B) + \text{updated } h(B) \\ &= 1 + 8 \\ &= 9 \end{aligned}$$

$$\begin{aligned} f(A \rightarrow B) &= g(B) + h(B) \\ &= 1 + 5 \quad (g(n) = 1 \text{ is the default path cost}) \\ &= 6 \end{aligned}$$

$$\begin{aligned} f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\ &= 1 + 2 + 1 + 4 \quad (C \text{ \& } D \text{ are AND nodes}) \\ &= 8 \end{aligned}$$



Now, we compare $f(A \rightarrow B) = 9$ with $f(A \rightarrow C + D) = 8$. Since $f(A \rightarrow C + D)$ is smaller, we explore this path and move to node **C**.

For node **C**:

$$\begin{aligned} f(C \rightarrow G) &= g(G) + h(G) \\ &= 1 + 3 \\ &= 4 \end{aligned}$$

$$\begin{aligned} f(C \rightarrow H + I) &= g(H) + h(H) + g(I) + h(I) \\ &= 1 + 0 + 1 + 0 \quad (H \text{ \& } I \text{ are AND nodes}) \\ &= 2 \end{aligned}$$

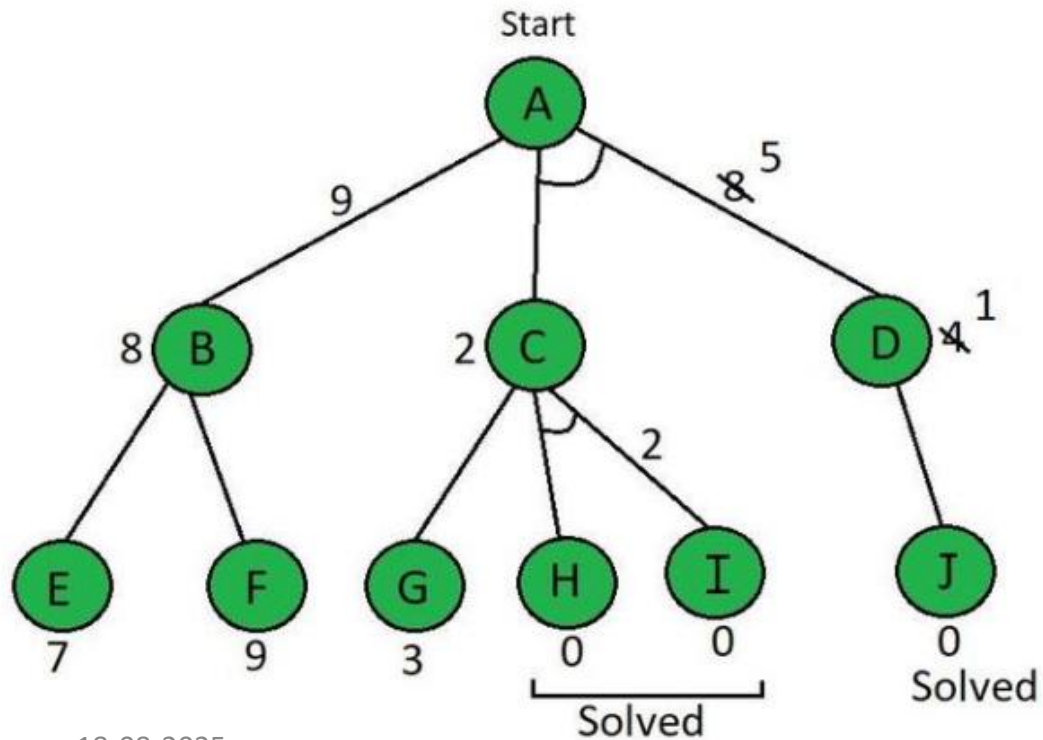
$f(C \rightarrow H+I)$ is selected as the path with the lowest cost and the heuristic is also left unchanged because it matches the actual cost. Paths H & I are solved because the heuristic for those paths is **0**, but Path **A→D** needs to be calculated because it has an **AND**.

For node **D**:

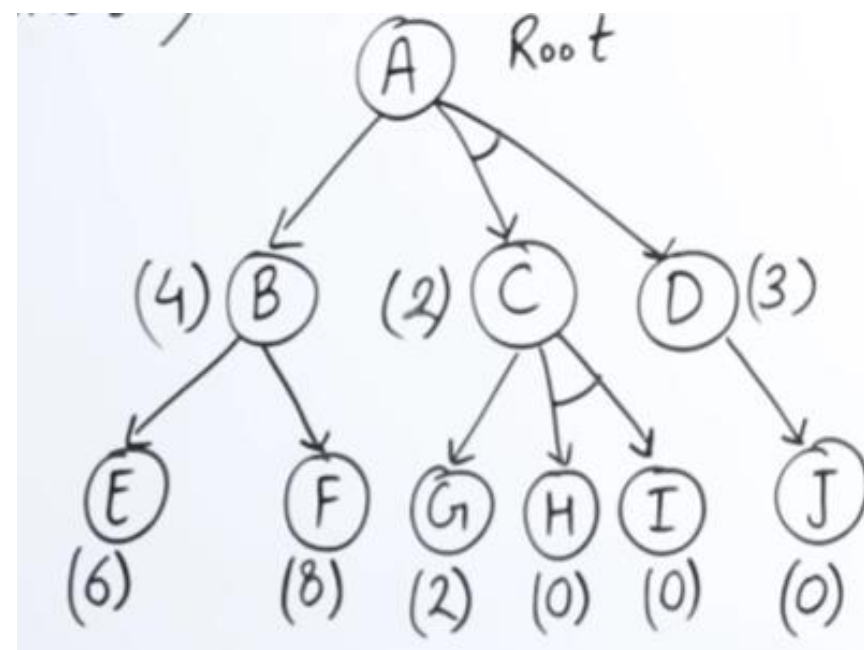
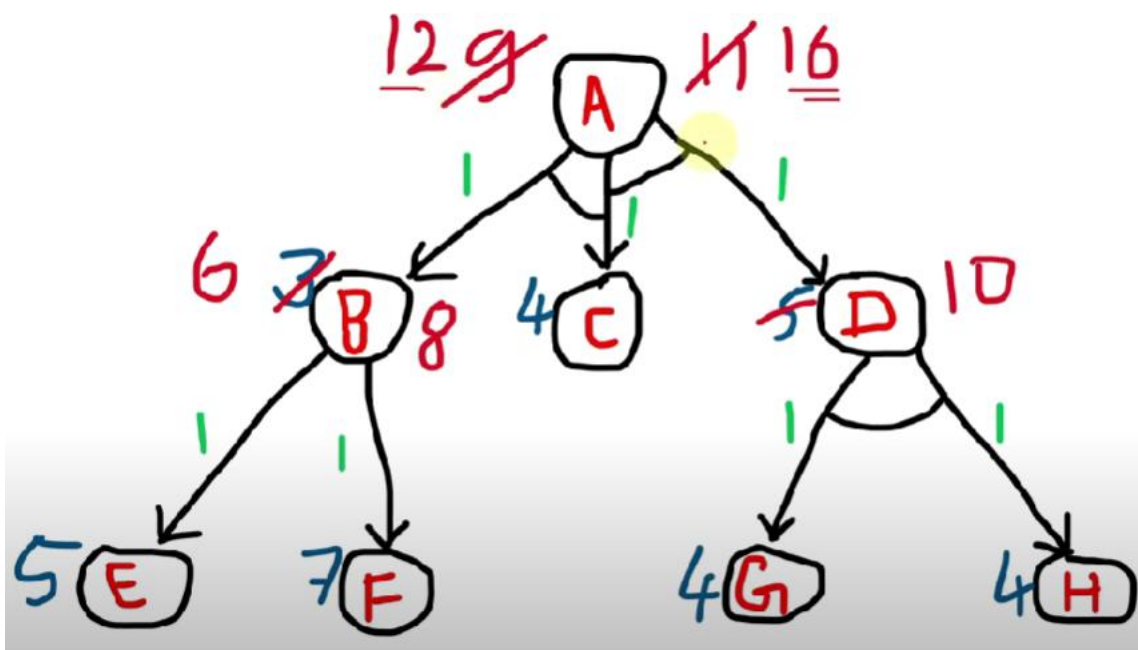
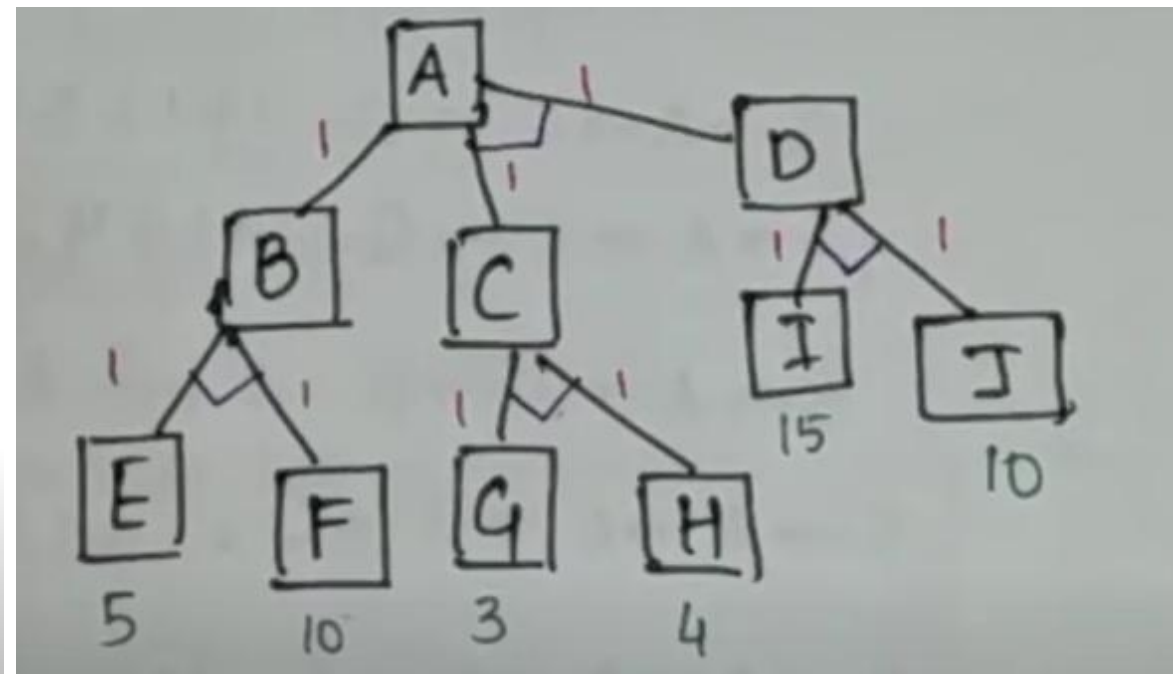
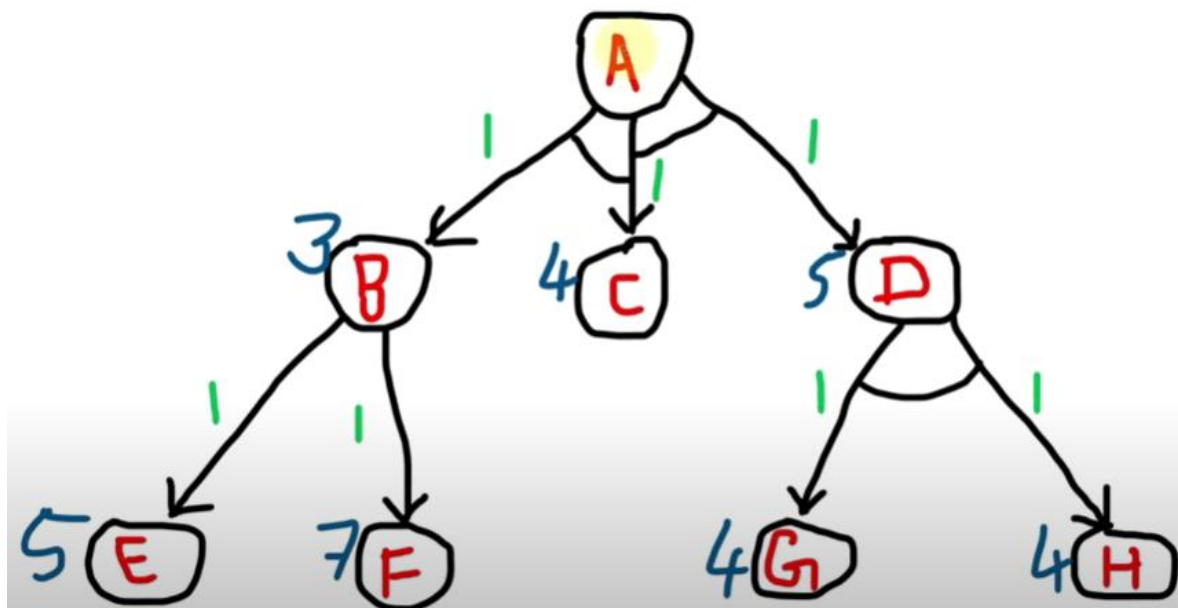
$$\begin{aligned}f(D \rightarrow J) &= g(J) + h(J) \\&= 1 + 0 \\&= 1\end{aligned}$$

After updating the heuristic for **D**, we recalculate:

$$\begin{aligned}f(A \rightarrow C + D) &= g(C) + h(C) + g(D) + h(D) \\&= 1 + 2 + 1 + 1 \\&= 5\end{aligned}$$



Now fully solved graph is



Aspect	<i>A Algorithm*</i>	<i>AO Algorithm*</i>
Search Type	Best-first search	Best-first search
Type of Search	Informed search using heuristics	Informed search using heuristics
Solution Optimality	Always gives the optimal solution	Does not guarantee an optimal solution
Path Exploration	Explores all possible paths	Stops exploring once a solution is found
Memory Usage	Uses more memory	Uses less memory
Endless Loop	May go into an endless loop without proper checks	Cannot go into an endless loop

ADVERSARIAL SEARCH AND GAMES

- Multi agent environments: **more than one agent** is searching for the solution in the same search space
- **Searches in which two or more players with conflicting goals are trying to explore the same search space for the solution, are called adversarial searches, often known as Games.**

Types of Games in AI:

- **Perfect information:** agents can look into the complete board. Ex: Chess, Checkers, Go, etc.
- **Imperfect information:** agents can't see each other moves. Ex: tic-tac-toe, Battleship, blind, Bridge, etc.
- **Deterministic games:** follow a strict pattern and set of rules Ex: chess, Checkers, Go, tic-tac-toe, etc.
- **Non-deterministic games (stochastic games):** The factor of chance or luck is introduced by either dice or cards.
Example: Backgammon, Monopoly, Poker, etc.

Zero-sum games of perfect information: Deterministic, fully observable environments

In which two agents act alternately and in which the utility values at the end of the game are always equal and opposite.

One player of the game try to maximize one single value, while another player tries to minimize it. It involves

Embedded thinking...

Embedded thinking or backward reasoning!



- One agent is trying to figure out what to do.
- How to decide? He thinks about the consequences of the possible actions.
- He needs to think about his opponent as well...
- The opponent is also thinking about what to do etc.
- Each will imagine what would be the response from the opponent to their actions.
- This entails an embedded thinking.

Formalization of the problem:

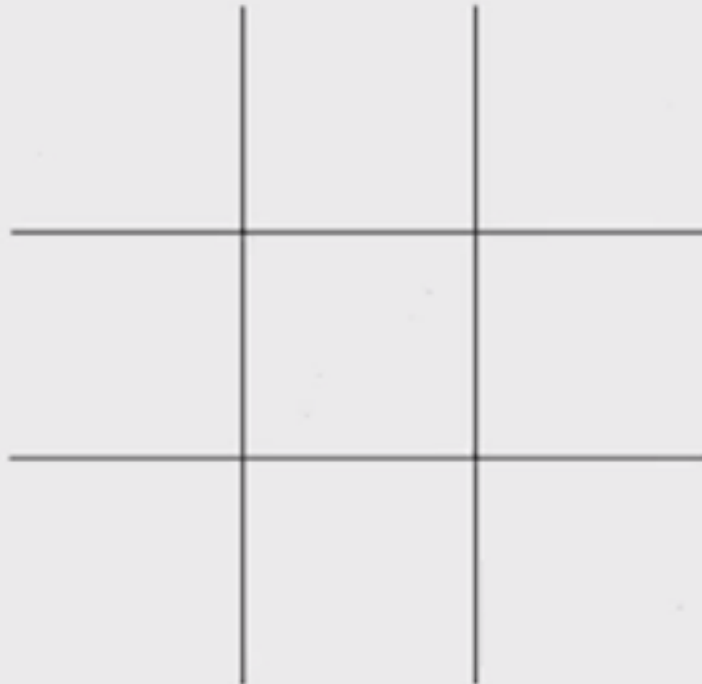
A game can be defined as a type of search in AI which can be formalized of the following elements:

- **Initial state:** It specifies how the game is set up at the start.
- **Player(s):** It specifies which player has moved in the state space.
- **Action(s):** It returns the set of legal moves in state space.
- **Result(s, a):** It is the transition model, which specifies the result of moves in the state space.
- **Terminal-Test(s):** Terminal test is true if the game is over, else it is false at any case. The state where the game ends is called terminal states.
- **Utility(s, p):** A utility function gives the final numeric value for a game that ends in terminal states s for player p . It is also called payoff function. For Chess, the outcomes are a win, loss, or draw and its payoff values are +1, 0, $\frac{1}{2}$. And for tic-tac-toe, utility values are +1, -1, and 0.

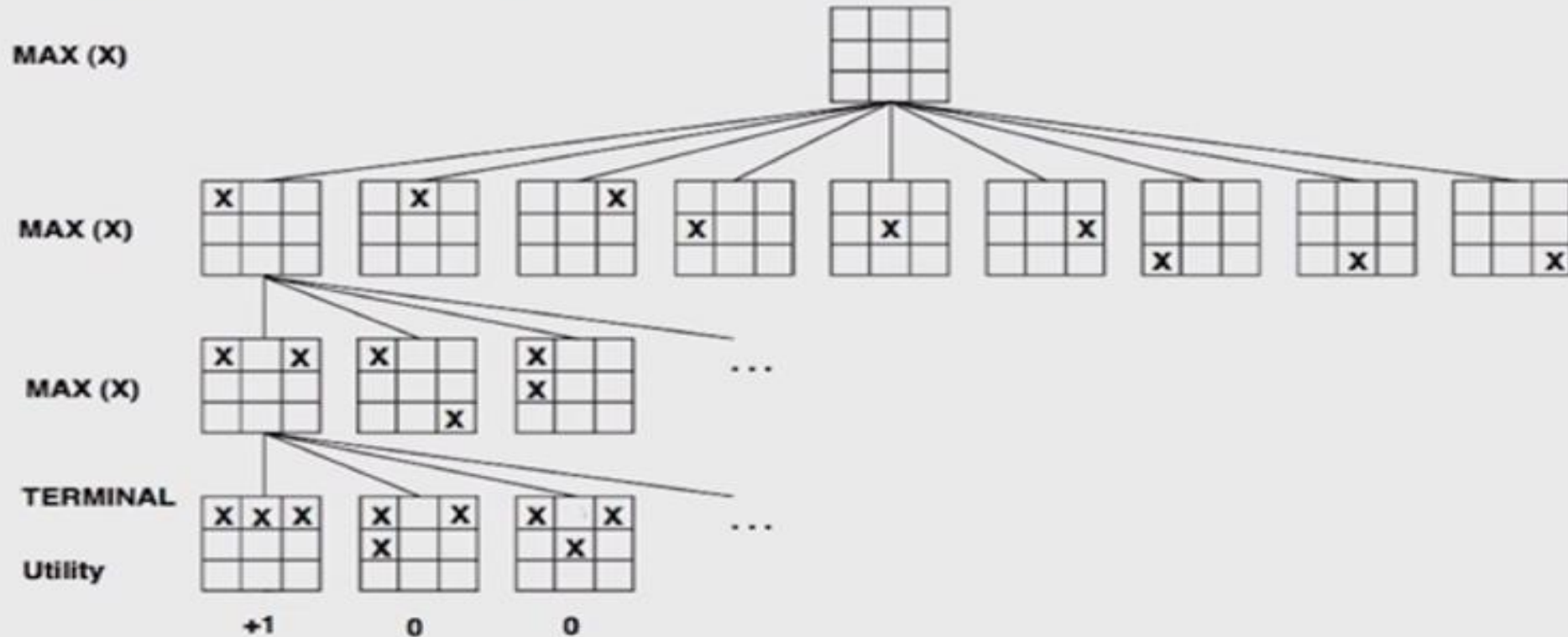
Single player...

Assume we have a tic-tac-toe with one player.

Let's call him Max and have him play three moves only for the sake of the example.



Single player...

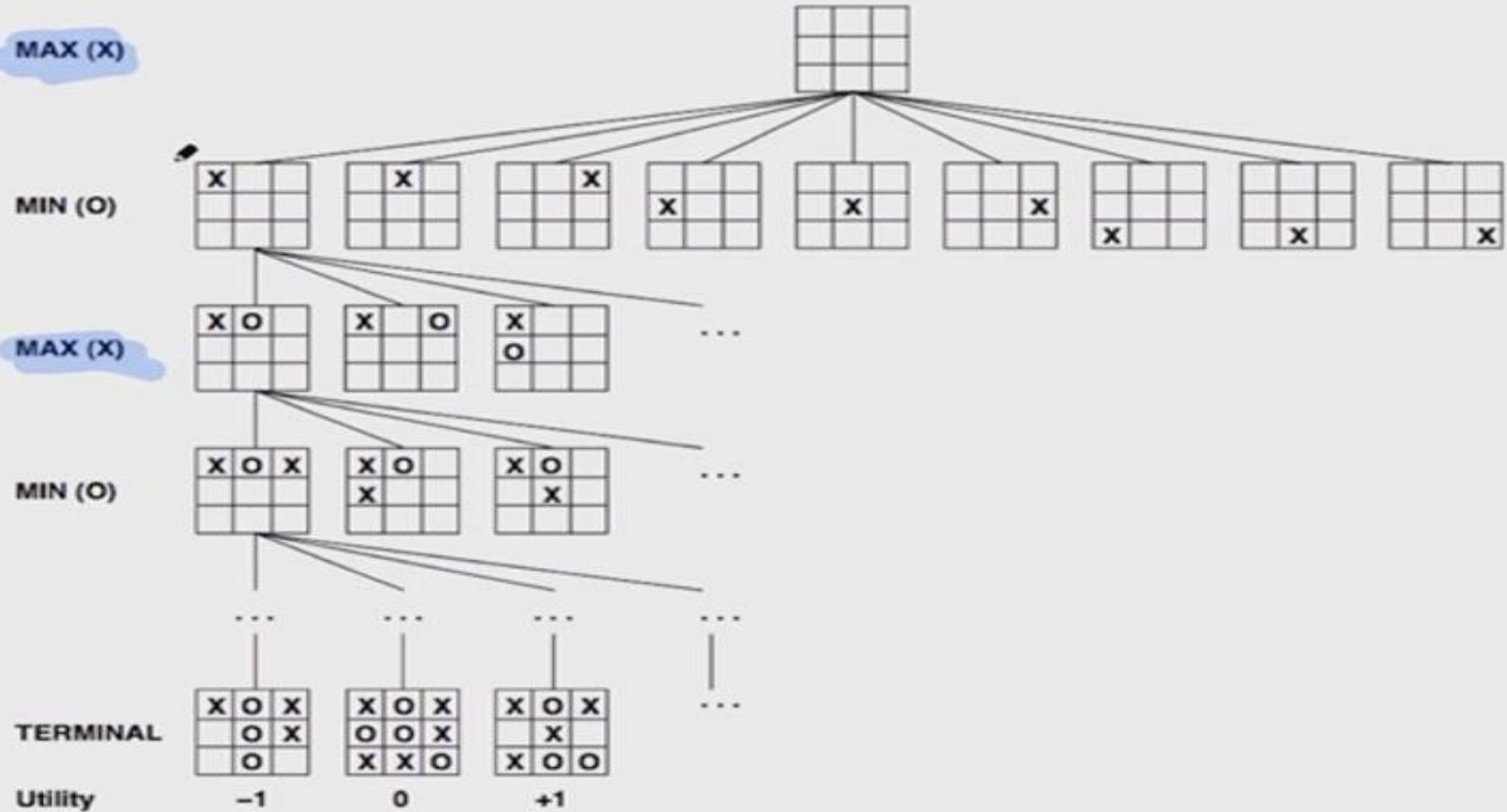


In the case of one player, nothing will prevent Max from winning (choose the path that leads to the desired utility here 1), unless there is another player who will do everything to make Max lose, let's call him Min (the Mean :))

Adversarial search: minimax

- Two players: Max and Min
- Players alternate turns
- Max moves first
- Max maximizes results
- Min minimizes the result
- Compute each node's minimax value's the best achievable utility against an optimal adversary
- Minimax value $\equiv$ best achievable payoff against best play

Minimax example



Game tree: (Search tree, Space graph)

A game tree is a tree where nodes of the tree are the game states and Edges of the tree are the moves by players. Game tree involves initial state, actions function, and result Function.

Example: Tic-Tac-Toe game tree

Adversarial Search for the minimax procedure works as follows:

- It aims to find the optimal strategy for MAX to win the game.
- It follows the approach of **Depth-first search**.
- In the game tree, optimal leaf node could appear at any depth of the tree.
- Propagate the minimax values up to the tree until the terminal node discovered.

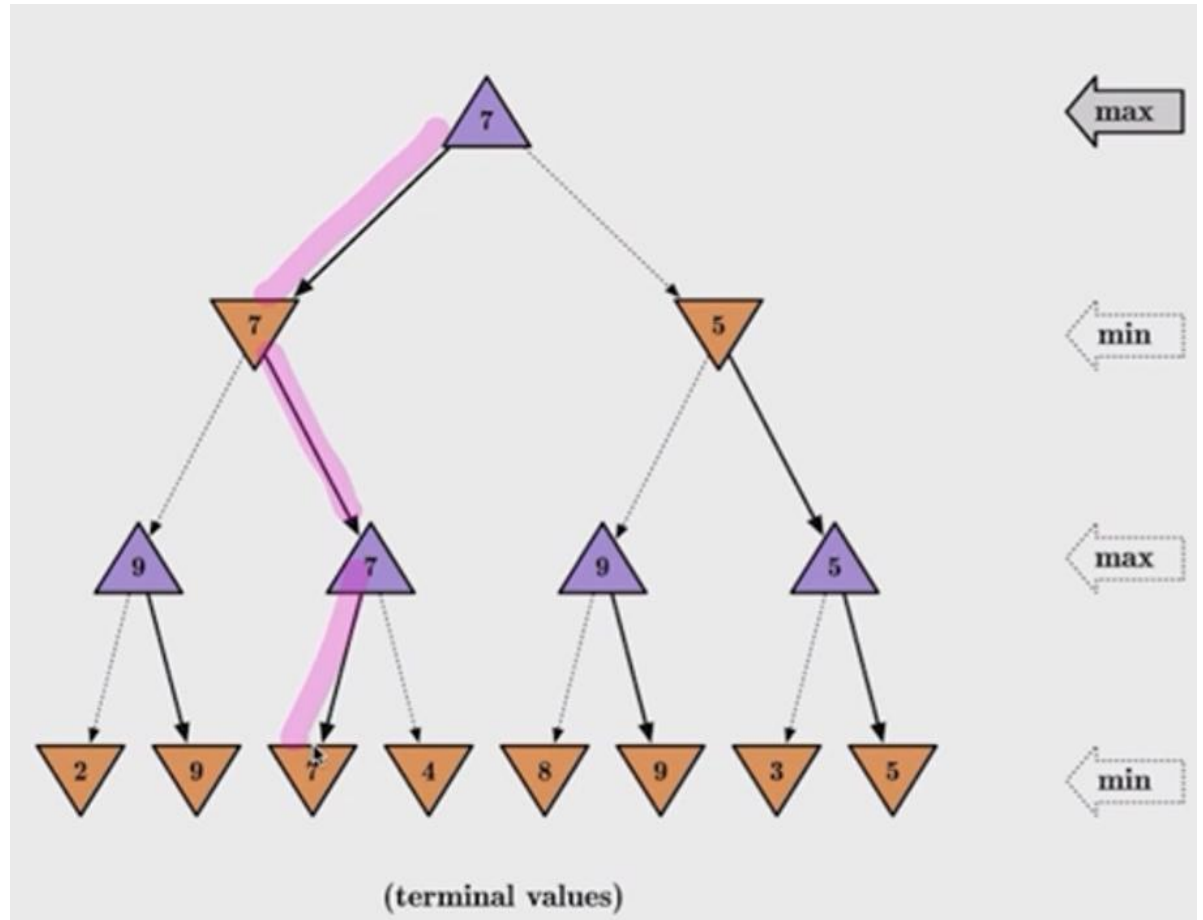
Minimax algorithm:

- Back tracking algorithm
- Best move strategy used
- Max always try to maximize the utility (Best move)
- Min will try to minimize (Worst move)
- Use in two-player games. (Chess, tic-tac-toe)

Properties:

- **Optimality:** Guarantees the best possible move if both players play perfectly.
- **Deterministic:** Provides consistent results for the same inputs.
- **Time Complexity:** $O(b^d)$, where:
bb = branching factor (number of possible moves at each level),
dd = depth of the game tree.

1. Generate the whole game tree to leaves
2. Apply utility (payoff) function to leaves
3. Uses DFS for expanding the tree
4. Back-up values from leaves toward the root:
 1. a Max node computes the maximum value from its child values
 2. a Min node computes the minimum value from its child values
5. When value reaches the root: optimal move is determined.



Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory.

- In a given game tree, the optimal strategy can be determined from the minimax value of each node, which can be written as MINIMAX(n).
- MAX prefer to move to a state of maximum value and MIN prefer to move to a state of minimum value then:

$$\text{For a state } S \text{ MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{If } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{If } \text{PLAYER}(s) = \text{MIN.} \end{cases}$$

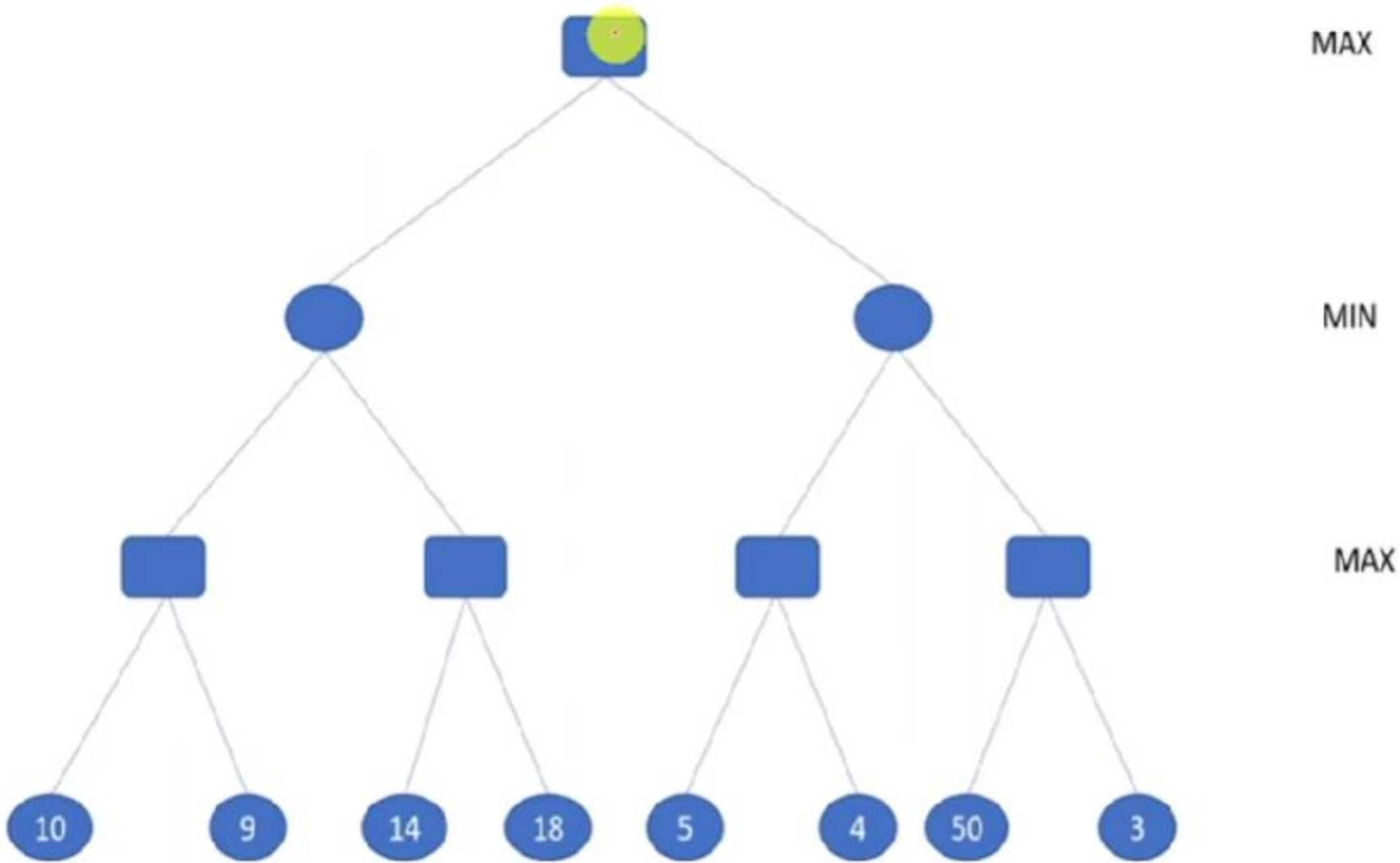
Adversarial search: minimax

- If state is terminal node: Value is $utility(state)$
- If state is MAX node: Value is highest value of all successor node values (children)
- If state is MIN node: Value is lowest value of all successor node values (children)

function MINIMAX-DECISION($state$) **returns** *an action*
 return $\arg \max_{a \in \text{ACTIONS}(s)} \text{MIN-VALUE}(\text{RESULT}(state, a))$

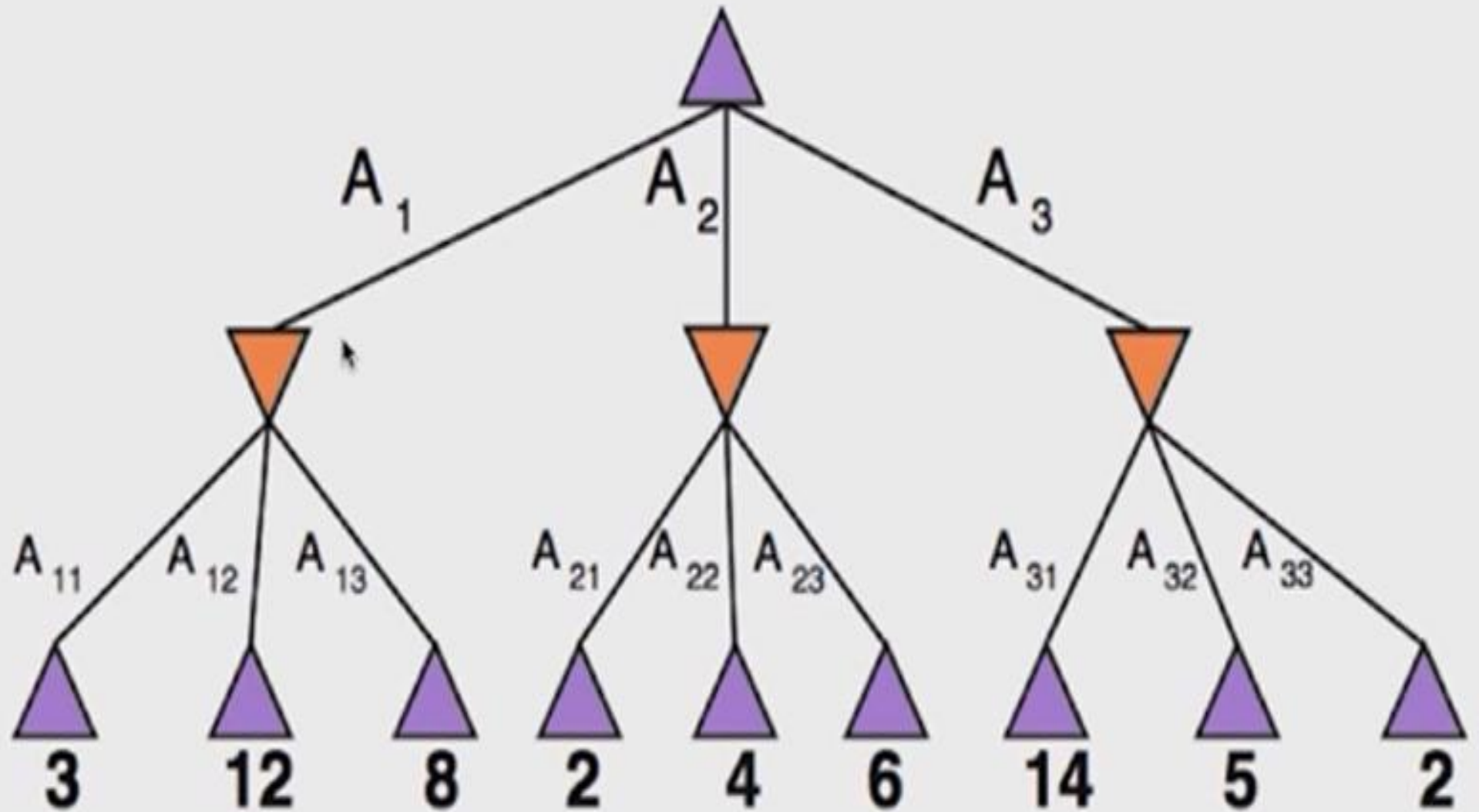
function MAX-VALUE($state$) **returns** *a utility value*
 if TERMINAL-TEST($state$) **then return** UTILITY($state$)
 $v \leftarrow -\infty$
 for each a **in** ACTIONS($state$) **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a)))$
 return v

function MIN-VALUE($state$) **returns** *a utility value*
 if TERMINAL-TEST($state$) **then return** UTILITY($state$)
 $v \leftarrow \infty$
 for each a **in** ACTIONS($state$) **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a)))$
 return v



MAX

MIN



A two-ply game tree.

Properties of Mini-Max algorithm:

- **Complete-** Min-Max algorithm is Complete.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS wh

Tic-Tac-Toe

- * ≈ 5 legal moves on average, total of 9 moves (9 plies).
- * $5^9 = 1,953,125$
- * $9! = 362,880$ terminal nodes

Limitation of the minimax Algorithm:

- The main drawback of the minimax algorithm is that it gets really slow for complex games.
- This type of games has a huge branching factor, and the player has lots of choices to decide.

Alpha-Beta Pruning

- In the minimax search algorithm that the number of game states it has to examine are **exponential in depth** of the tree.
- There is a technique by which without checking each node of the game tree we can compute the correct minimax decision, and this technique is called **pruning**.
- This involves **two threshold parameter Alpha and beta** for future expansion, so it is called **alpha-beta pruning**. It is also called as **Alpha-Beta Algorithm**.
- Alpha-beta pruning can be applied at any depth of a tree, and sometimes it not only prune the tree leaves but also entire sub-tree.
- The two-parameter can be defined as:
 - Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximizer. The initial value of alpha is $-\infty$.
 - Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimizer. The initial value of beta is $+\infty$.
- By pruning these nodes, it makes the algorithm fast.

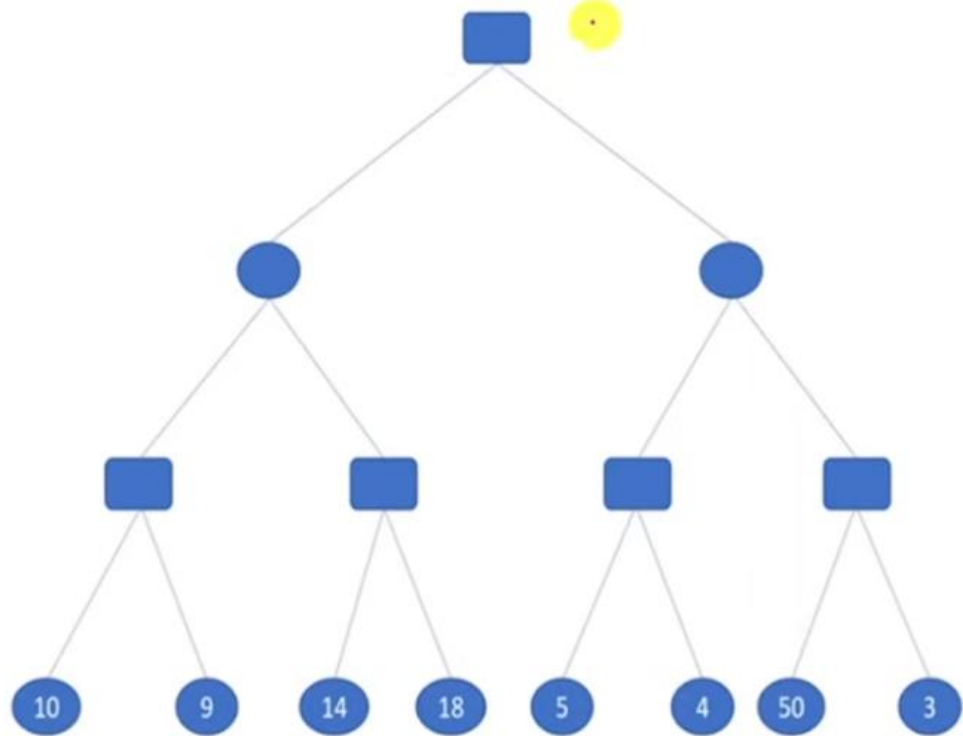
Condition for Alpha-beta pruning:

The main condition which required for alpha-beta pruning is:

$$\alpha \geq \beta$$

Key points about alpha-beta pruning:

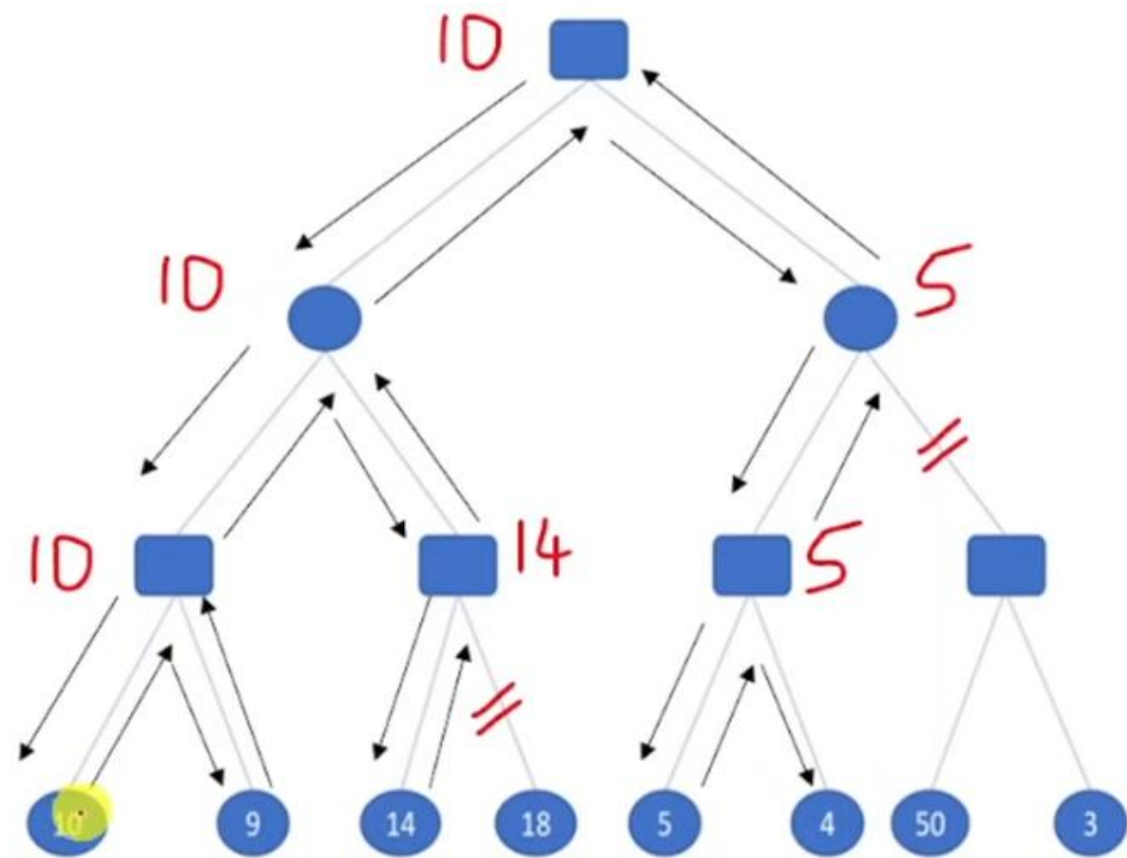
- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to upper nodes instead of values of alpha and beta.
- We will only pass the alpha, beta values to the child nodes.



MAX [α]

MIN [β]

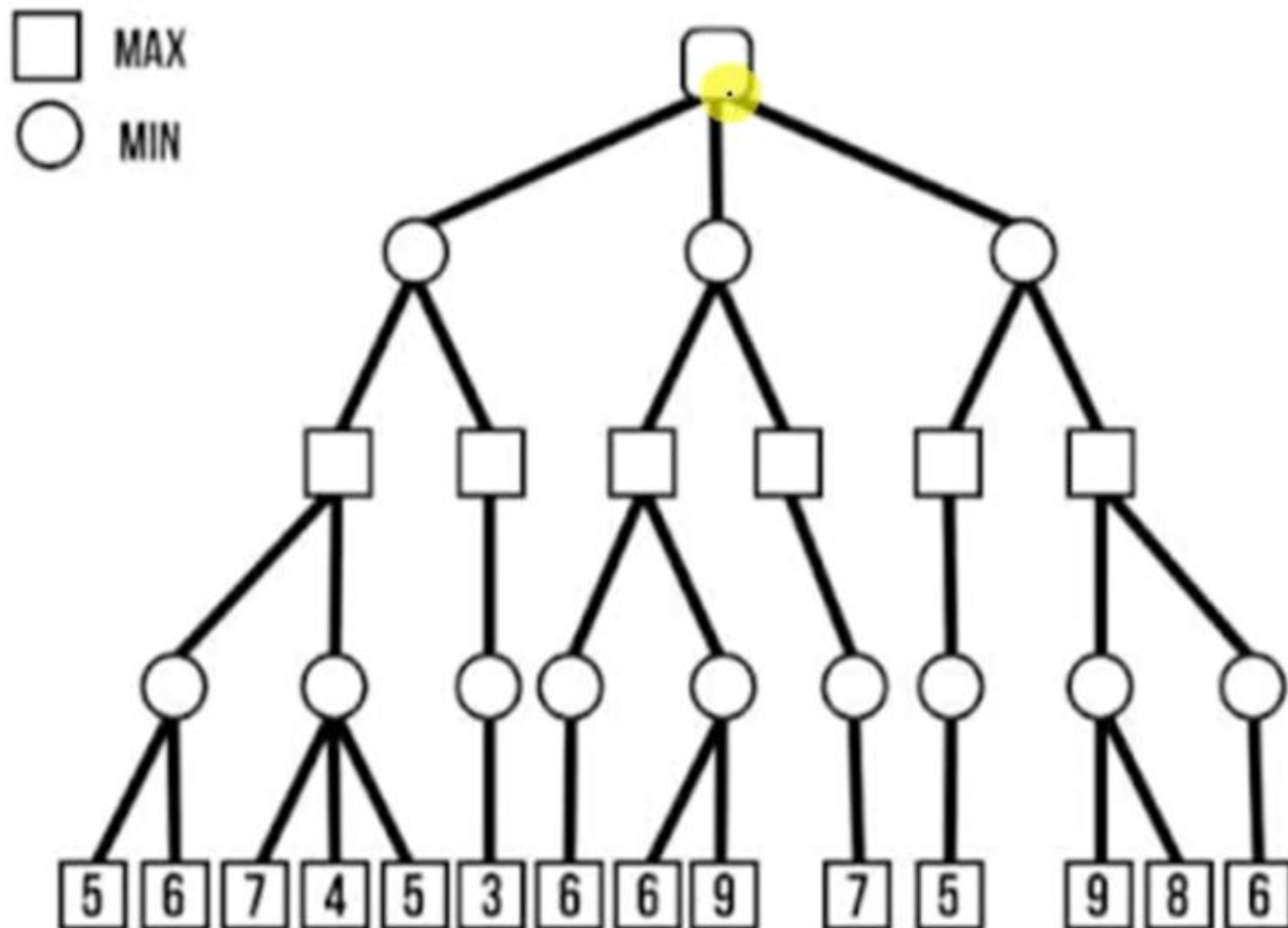
MAX [α]



MAX [α]

MIN [β]

MAX [α]

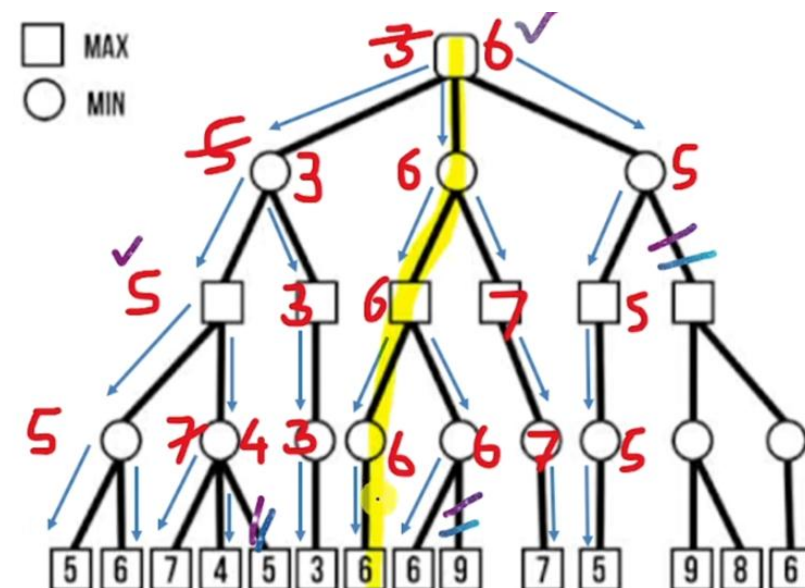


$Max \alpha$

$Min \beta$

$Max \alpha$

$Min \beta$



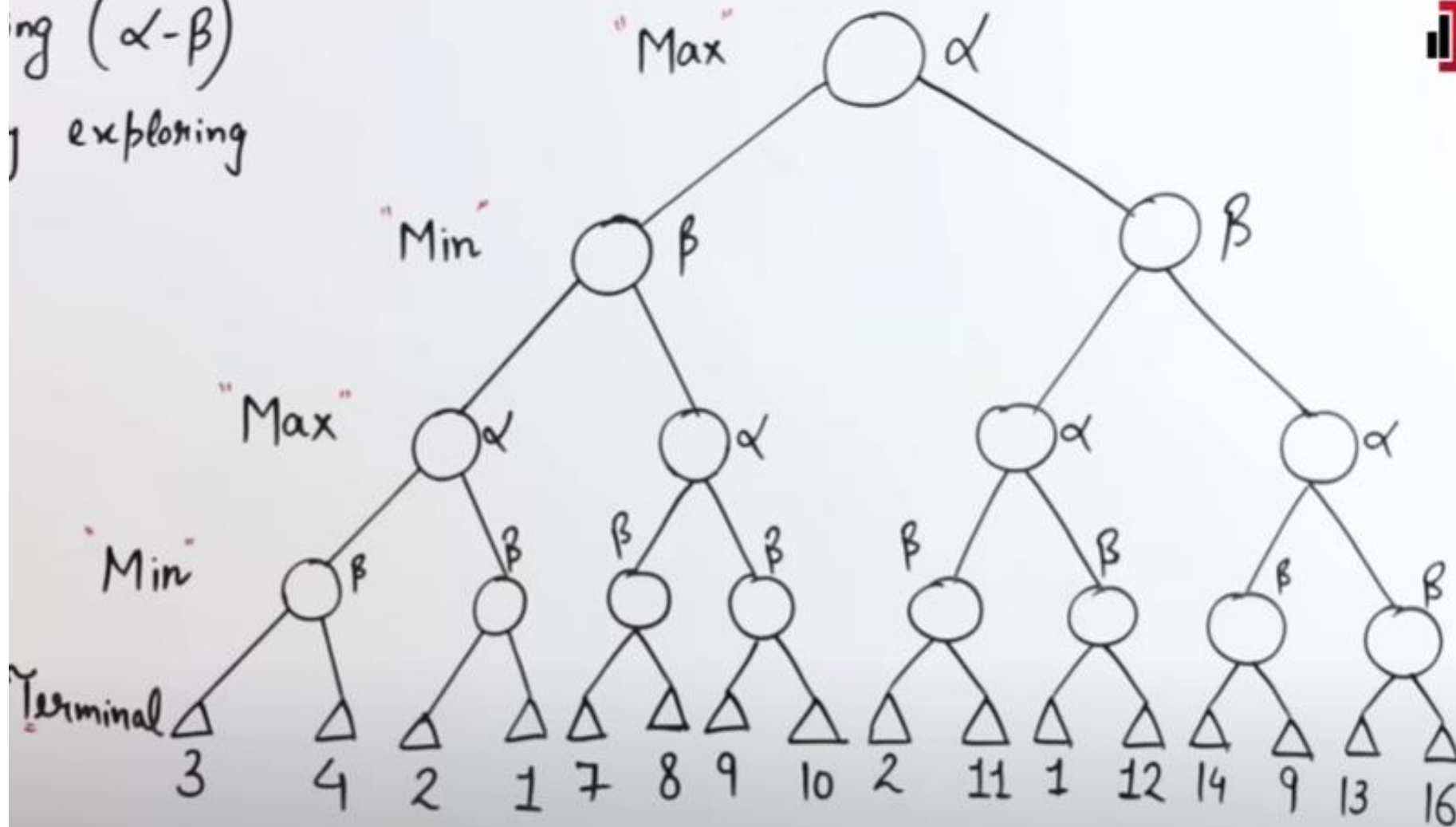
$Max \alpha$

$Min \beta$

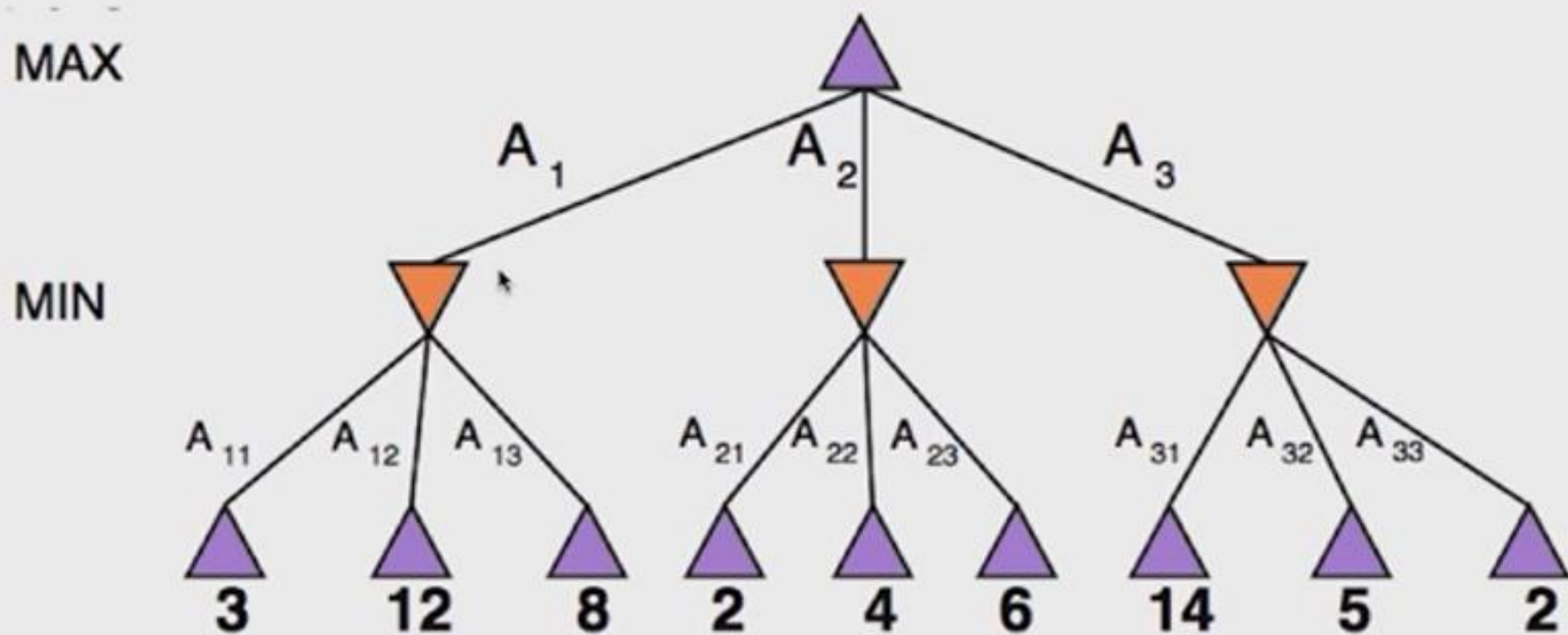
$Max \alpha$

$Min \beta$

ing ($\alpha - \beta$)
 exploring



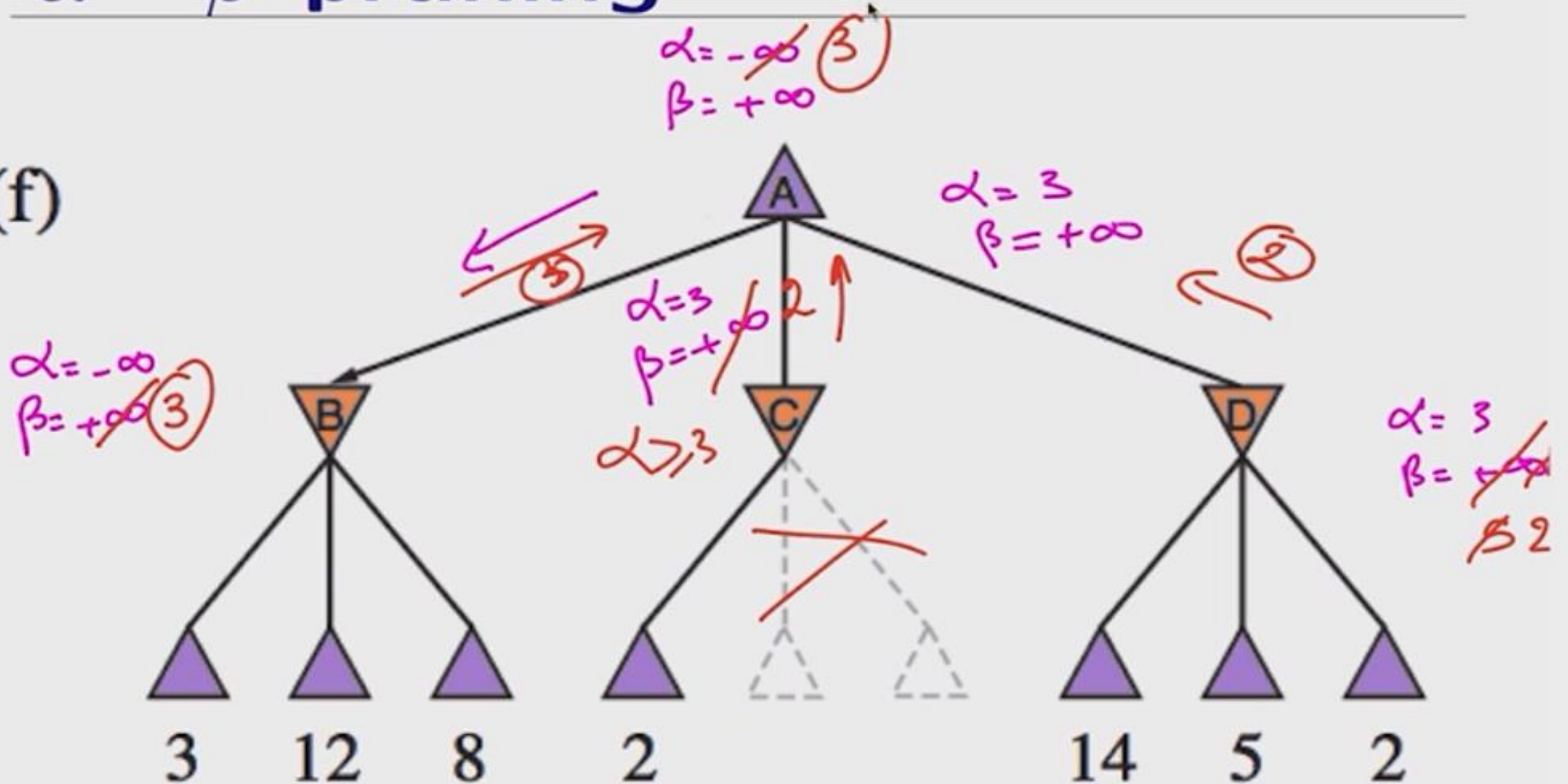
$\alpha - \beta$ pruning



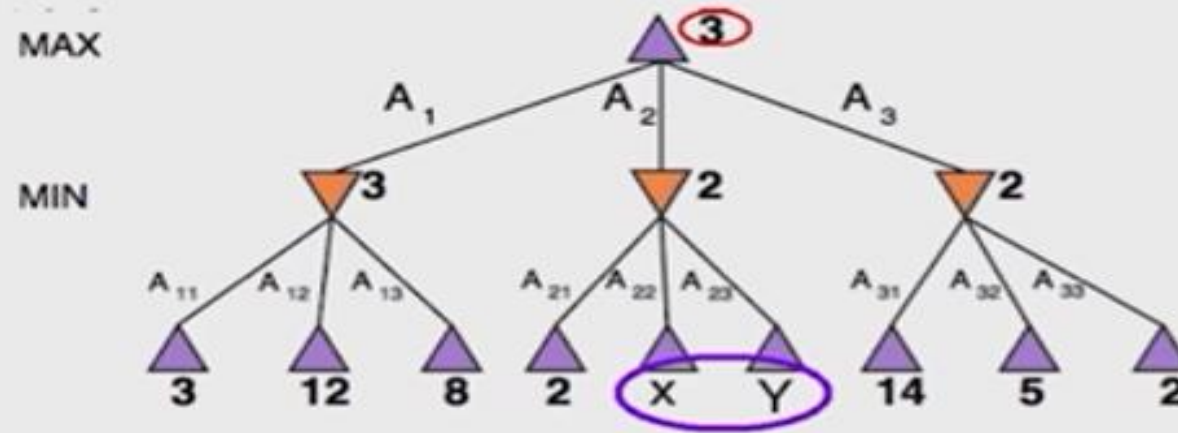
A two-ply game tree.

$\alpha - \beta$ pruning

(f)



$\alpha - \beta$ pruning



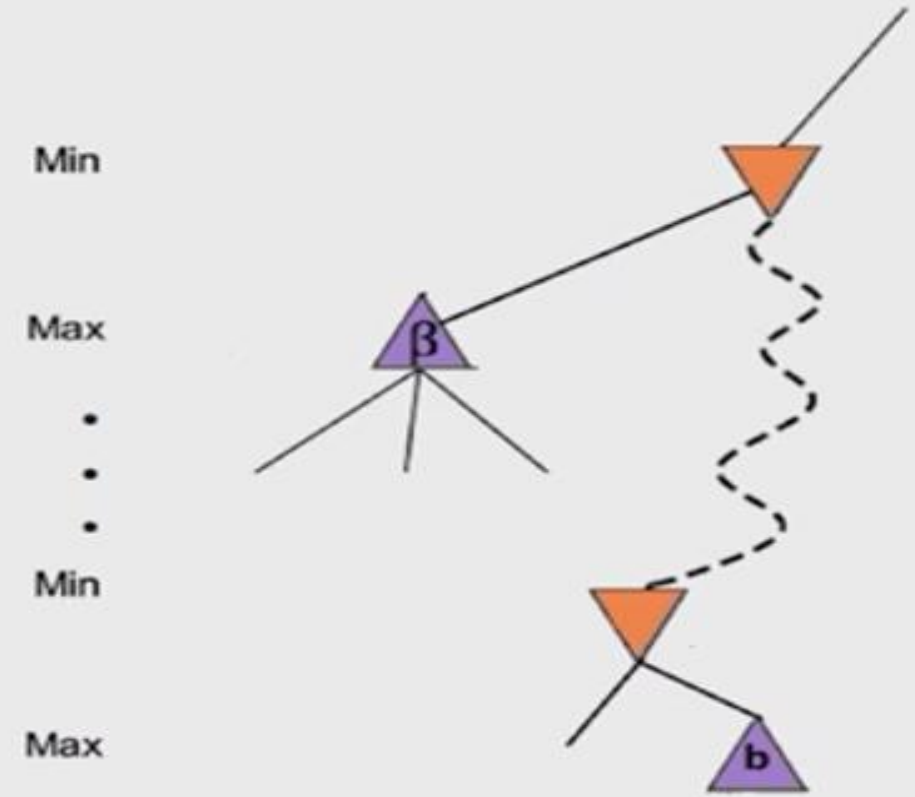
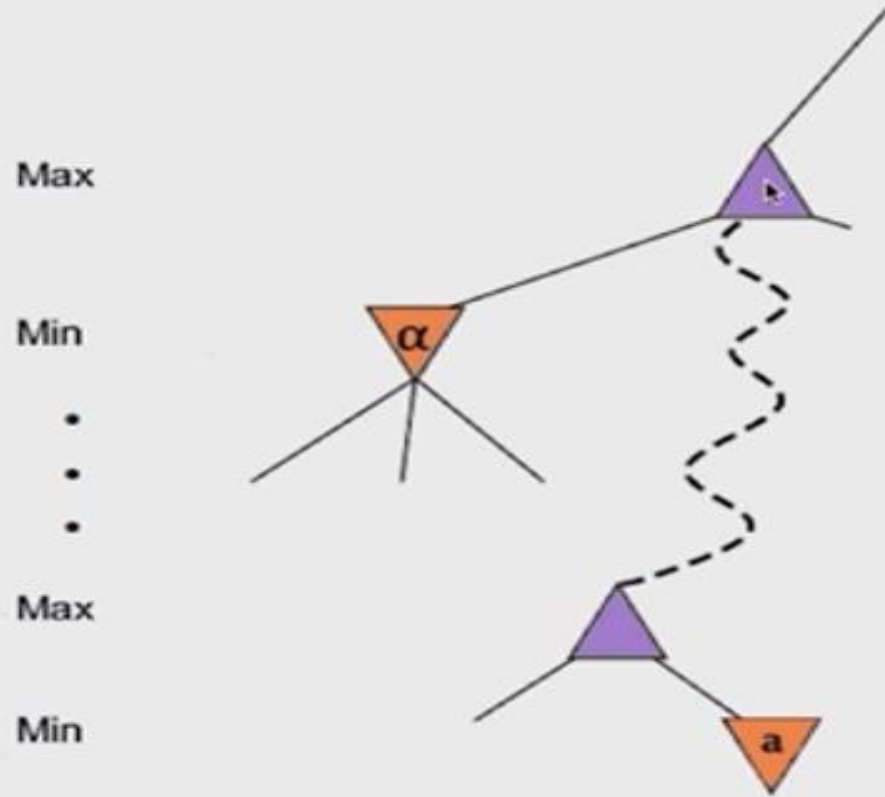
$$\begin{aligned} \text{Minimax}(\text{root}) &= \max(\min(3, 12, 8), \min(2, X, Y), \min(14, 5, 2)) \\ &= \max(3, \min(2, X, Y), 2) \\ &= \max(3, z, 2) \quad \text{where } z = \min(2, X, Y) \leq 2 \\ &= 3 \end{aligned}$$

Minimax decisions are independent of the values of X and Y .

$\alpha - \beta$ pruning

- **Strategy:** Just like minimax, it performs a DFS.
- **Parameters:** Keep track of two bounds
 - α : largest value for Max across seen children (current lower bound on MAX's outcome).
 - β : lowest value for MIN across seen children (current upper bound on MIN's outcome).
- **Initialization:** $\alpha = -\infty$, $\beta = \infty$
- **Propagation:** Send α , β values *down* during the search to be used for pruning.
 - Update α , β values by *propagating upwards* values of terminal nodes.
 - Update α only at Max nodes and update β only at Min nodes.

$\alpha - \beta$ pruning



- If α is better than a for Max, then Max will avoid it, that is prune that branch.
- If β is better than b for Min, then Min will avoid it, that is prune that branch.

function ALPHA-BETA-SEARCH($state$) **returns** an action
 $v \leftarrow \text{MAX-VALUE}(state, -\infty, +\infty)$
 return the *action* in $\text{ACTIONS}(state)$ with value v

function MAX-VALUE($state, \alpha, \beta$) **returns** a utility value
 if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow -\infty$
 for each a **in** $\text{ACTIONS}(state)$ **do**
 $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 if $v \geq \beta$ **then return** v
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 return v

function MIN-VALUE($state, \alpha, \beta$) **returns** a utility value
 if $\text{TERMINAL-TEST}(state)$ **then return** $\text{UTILITY}(state)$
 $v \leftarrow +\infty$
 for each a **in** $\text{ACTIONS}(state)$ **do**
 $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(\text{RESULT}(s, a), \alpha, \beta))$
 if $v \leq \alpha$ **then return** v
 $\beta \leftarrow \text{MIN}(\beta, v)$

Games: conclusion

- Games are modeled in AI as a search problem and use heuristic to evaluate the game.
- Minimax algorithm chooses the best move given an optimal play from the opponent.
- Minimax goes all the way down the tree which is not practical given game time constraints.
- Alpha-Beta pruning can reduce the game tree search which allows to go deeper in the tree within the time constraints.
- Pruning, bookkeeping, evaluation heuristics, node re-ordering and IDS are effective in practice.

Constraint Satisfaction Problem

- A problem is solved when each **variable has a value** that satisfies all the constraints on the variable. A problem described this way is called a **constraint satisfaction problem**
- It's used to find values for a group of variables which is used for fulfilling a set of rules of CSP.
- Associated tasks with CSP are resource allocation, planning, scheduling, Map coloring, floor planning for VLSI and decision making.

Assignment and solution of Constraint Satisfaction Problems (CSPs)

- An assignment is complete when every variable is assigned a value.
- A solution to a CSP is a complete assignment that satisfies all constraints

Components of CSP:

- **Variables** are represented by $V: \{X_1, X_2, \dots, X_n\}$. For example, in a Sudoku puzzle, each cell is a variable.
- **Domains** are represented by $D: \{D_1, D_2, \dots, D_n\}$. The set of possible values that each variable can take. In Sudoku, the domain for each cell is the numbers 1 to 9.
- **Constraints $\{C_1, C_2, \dots, C_m\}$** Rules that define the relationships between variables. (Scope and relation)
 - Scope is set of variables that participate in constraint
 - Relation defines the values that variable can take.

Example: In Sudoku game, its 9x9.i.e.,81 boxes present
Each 3x3 is considered as grid

- Here variables are spaces in the sudoku... that is 81
- Domain is the range of values.....that is 1-9
- Here constraint is the same no. should not be repeated in a row and in that column

2		5	3	8	4		9
	7					5	
9		4			6		7
5				4			2
			5		7		
6				3			8
4		6			8		1
	2					6	
8		1	2	9	7		4

Types of Variables

- Discrete variables
- Continuous variables

Varieties of Domains

- **Finite domains:** Have a finite number of possible values, such as colors or integers. Examples include map coloring and the 8-queens problem.
- **Infinite domains (integers, strings, etc.):** Have an infinite number of possible values, such as real numbers.
- **Continuous domains:** Have an infinite number of possible values, but they can be represented by a finite set of parameters, such as the coefficients of a polynomial function.
- **Discrete Domains:** A domain that has distinct, separated values, and countable with no immediate values between them.

Varieties of Constraints

- **Unary constraints:** Involve a single variable, e.g., $SA \neq \text{green}$
- **Binary constraints:** Involve pairs of variables, e.g., $SA \neq WA$
- **Global constraints:** Involve 3 or more variables

Example: All diff constraint, which specifies that all variables must have different values (e.g., cryptarithmic puzzles, Sudoku)

- **Preferences (soft constraints):**
 - Example: "red is better than green"
 - Often represented by a cost for each variable assignment
 - Used in constrained optimization problems

Example of CSP - the Map Coloring problem.

Problem Description:

- The problem is to color the regions of a given map such that no two adjacent regions have the same colors.
- The regions in the map are the variables, and the set of possible colors for each region is the domain.

Representation of the Problem:

- Variables: $V_i = \{WA, NT, Q, NSW, V, SA, T\}$
- Domains: $D_i = \{\text{red, green, blue}\}$
- Constraints: Adjacent regions must have different colors, e.g. $WA \neq NT$.
- So (WA,NT) must be in $\{(\text{red,green}), (\text{red,blue}), (\text{green,red}), \dots\}$



Example: Map coloring



Variables: $X = \{WA, NT, Q, NSW, V, SA, T\}$

Domains: $D_i = \{\text{red, green, blue}\}$

Constraints: adjacent regions must have different colors;

e.g., $WA \neq NT$ or $(WA, NT) \in \{(\text{red, green}), (\text{red, blue}), \text{etc.}\}$

Example: Map coloring



Example:

{WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green}

Important Terminologies used in CSP

Successor Functions:

- A value can be assigned to any unassigned variable, provided that it does not conflict with previously assigned values

Goal test:

- The current assignment is complete, and the goal is tested to see if it has been reached.

Path cost:

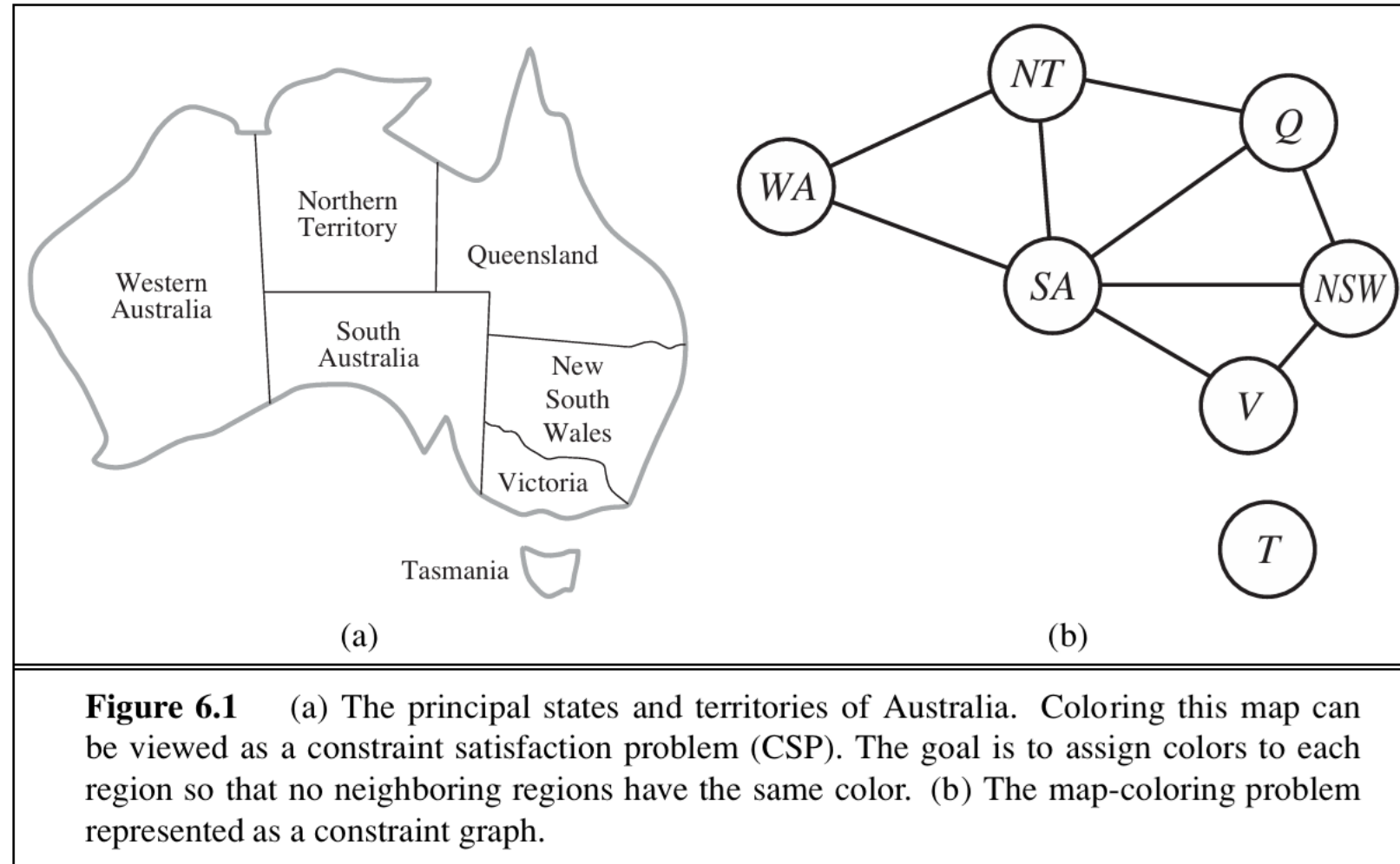
- The cost is computed for every step.

Most of the time, these problems have many solutions.

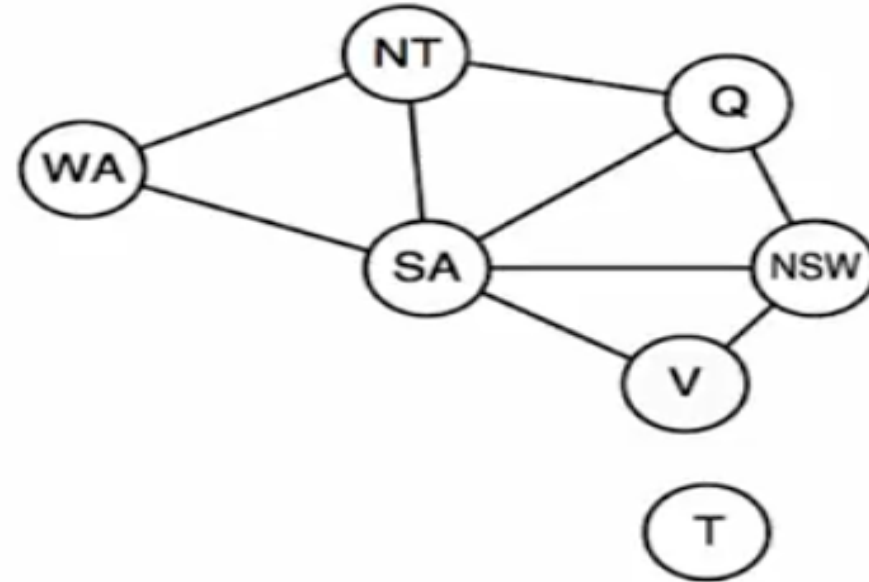
Identifying Solutions:

The topology of the constraint graph can be used to identify solutions easily.

Map Coloring: Map coloring can sometimes be represented as a graph coloring problem.



Constraint graph



Binary CSP: each constraint relates at most two variables
Constraint graph: nodes are variables, arcs show constraints

CSP algorithms: use the graph structure to speed up search.
E.g., Tasmania is an independent subproblem!

Real-world CSPs

- Assignment problems, e.g., who teaches what class?
- Timetabling problems, e.g., which class is offered when and where?
- Hardware configuration
- Spreadsheets
- Transportation scheduling
- Factory scheduling
- Floor planning
- Notice that many real-world problems involve real-valued variables

Searching for the goal state in Constraint Satisfaction Problems (CSPs) are:

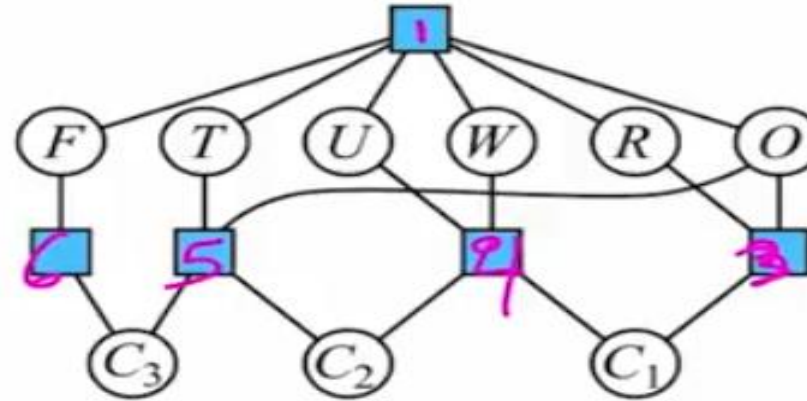
1. **Complete Assignment:** Every solution must be a complete assignment, and therefore appears at depth ' n ', where ' n ' is the number of variables.
2. **Search Tree Depth:** The search tree extends only to depth ' n ', as the goal is to find a complete assignment of all variables.
3. **Search Algorithms: Depth-first search algorithms** are popular for solving CSPs.
4. **Irrelevant Path:** The path by which a solution is reached is irrelevant, as the goal is to find a valid complete assignment.
5. **Local Search Methods:** Local search methods work well for this formulation of CSPs, where the focus is on finding a valid complete assignment.

Cryptarithmic Problem

- Each letter stands for a unique digit
- The aim is to find a substitution of digits for letters such that the resulting sum is arithmetically correct, with the added restriction that **no leading zeroes** are allowed.
- Where the goal is to assign each letter a digit from 0 to 9 so that the arithmetic works out correctly.
- The rules are that all occurrences of a letter must be assigned the same digit, and no digit can be assigned to more than one letter.

Example Cryptarithmic

$$\begin{array}{r}
 C_3 \ C_2 \ C_1 \\
 T \ W \ O \\
 + \ T \ W \ O \\
 \hline
 F \ O \ U \ R
 \end{array}$$



Variables: $X = \{F, T, U, W, R, O, C_1, C_2, C_3\}$

Domain: $D = \{0, 1, 2, \dots, 9\}$

Constraints:

- ① • Alldiff(F, T, U, W, R, O)
- ② • $T \neq 0, F \neq 0$
- ③ • $O + O = R + 10 * C_1$
- ④ • $C_1 + W + W = U + 10 * C_2$
- ⑤ • $C_2 + T + T = O + 10 * C_3$
- ⑥ • $C_3 = F$

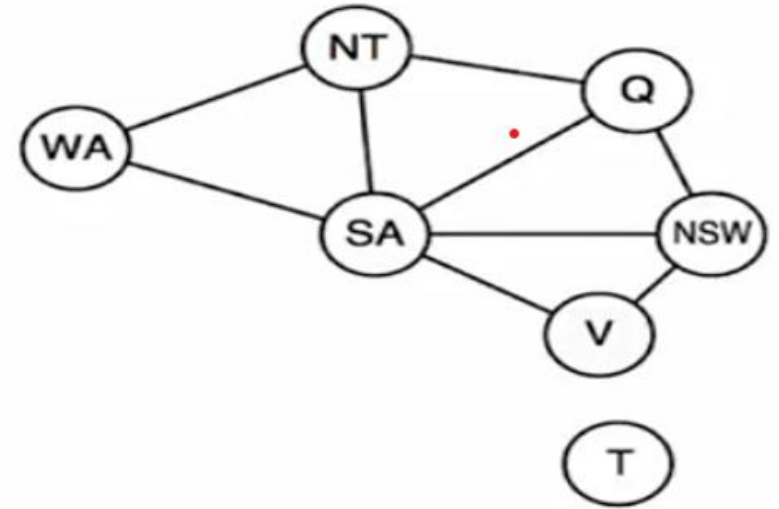
n-ary constraint

6-ary

- $TO + GO = OUT$ $O=1, G=8, U=0, T=2$
- $SEND + MORE = MONEY$ $S=9, E=5, N=6, D=7, M=1, O=0, R=8, Y=2$
- $LOGIC + LOGIC = PROLOG$ $P=9, R=8, O=0, G=4, L=9, C=2, I=5$

Constraint Propagation : Inference in CSPs

- The process of using the constraints to reduce the number of legal values for a variable, which in turn can reduce the legal values for another variable, and so on.
- The key idea is local consistency. Its types are:
 1. Node consistency
 2. Arc consistency
 3. Path consistency
 4. k-consistency
- They use the constraints to infer which variable/value pairs are consistent and which are not.



Node Consistency: (Unary Constraint)

- A single variable (corresponding to a node in the CSP network) is node-consistent if all the values in the variable's domain satisfy the variable's unary constraints.
- A network is node-consistent if every variable in the network is node-consistent.
- For example, in the variant of the Australia map-coloring problem where South Australians dislike red, the variable SA starts with domain {red, green, blue}, and we can make it node consistent by eliminating red, leaving SA with the reduced domain {green, blue}.

Arc Consistency: (Binary Constraint)

- A variable in a CSP is arc-consistent if every value in its domain satisfies the variable's binary constraints.
- A network is arc-consistent if every variable is arc consistent with every other variable.
- The most popular algorithm for arc consistency is called **AC-3**.

function AC-3(csp) **returns** false if an inconsistency is found and true otherwise

inputs: csp , a binary CSP with components (X, D, C)

local variables: $queue$, a queue of arcs, initially all the arcs in csp

while $queue$ is not empty **do**

$(X_i, X_j) \leftarrow \text{REMOVE-FIRST}(queue)$

if REVISE(csp, X_i, X_j) **then**

if size of $D_i = 0$ **then return** false

for each X_k **in** $X_i.\text{NEIGHBORS} - \{X_j\}$ **do**

 add (X_k, X_i) to $queue$

return true

function REVISE(csp, X_i, X_j) **returns** true iff we revise the domain of X_i

$revised \leftarrow false$

for each x **in** D_i **do**

if no value y in D_j allows (x, y) to satisfy the constraint between X_i and X_j **then**

 delete x from D_i

$revised \leftarrow true$

return revised

Figure 6.3 The arc-consistency algorithm AC-3. After applying AC-3, either every arc is arc-consistent, or some variable has an empty domain, indicating that the CSP cannot be solved. The name “AC-3” was used by the algorithm’s inventor (Mackworth, 1977) because it’s the third version developed in the paper.

Path Consistency:

- A two-variable set $\{X_i, X_j\}$ is path-consistent with respect to a third variable X_m if, for every assignment $\{X_i = a, X_j = b\}$ consistent with the constraints on $\{X_i, X_j\}$, there is an assignment to X_m that satisfies the constraints on $\{X_i, X_m\}$ and $\{X_m, X_j\}$.
- This is called path consistency because one can think of it as looking at a path from X_i to X_j with X_m in the middle.

K- Consistency:

- Stronger forms of propagation can be defined with the notion of k-consistency. A CSP is k-consistent if, for any set of $k - 1$ variables and for any consistent assignment to those variables, a consistent value can always be assigned to any kth variable.
- 1-consistency says that, given the empty set, we can make any set of one variable consistent: this is what we called node consistency.
- 2-consistency is the same as arc consistency. For binary constraint networks.
- 3-consistency is the same as path consistency.

Backtracking Search for CSPs

Variables: in a map coloring problem, the variables are the regions or countries that need to be colored. $X: \{WA, NT, SA, Q, NSW, V, T\}$, where each variable represents a state or territory of Australia.

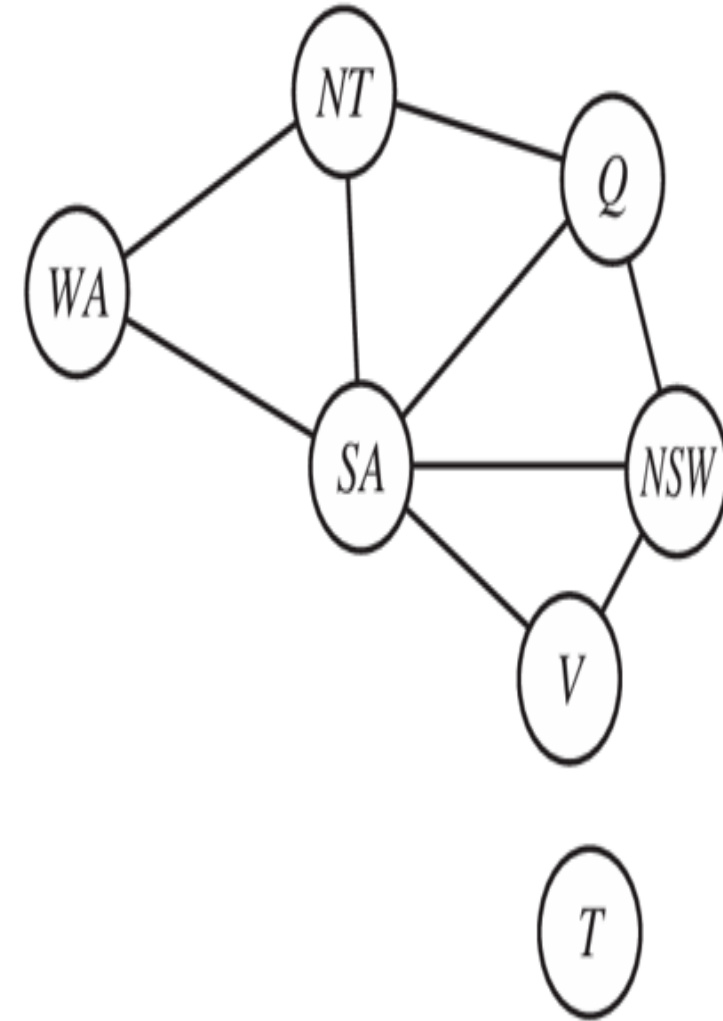
Domains: the domain of each variable is the set of colors that can be used to color the map. $D: \{\text{red, green, blue}\}$, where each variable has the same domain of three colors.

Constraints: Constraints can be unary, binary, or higher-order, depending on how many variables they involve. For example, in a map coloring problem, a binary constraint is that two adjacent regions cannot have the same color.

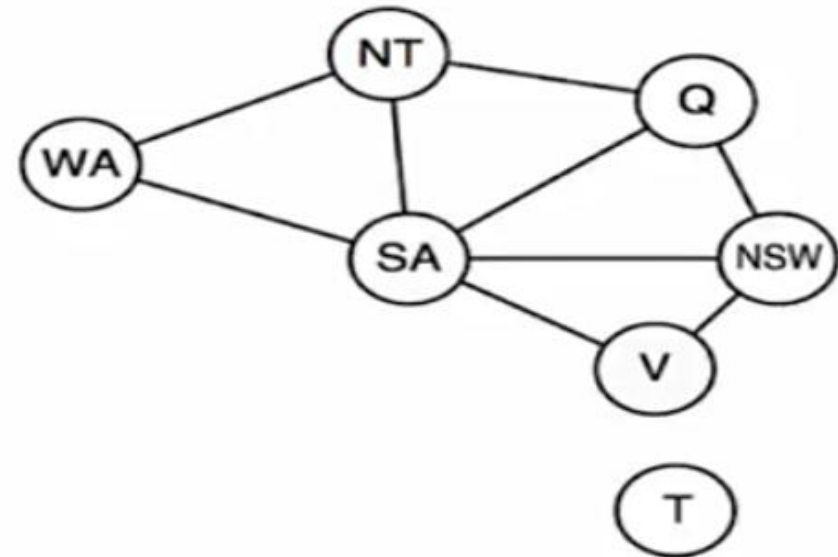
$C: \{ \langle (WA, NT), WA \neq NT \rangle, \langle (WA, SA), WA \neq SA \rangle, \langle (NT, SA), NT \neq SA \rangle, \langle (NT, Q), NT \neq Q \rangle, \langle (SA, Q), SA \neq Q \rangle, \langle (SA, NSW), SA \neq NSW \rangle, \langle (SA, V), SA \neq V \rangle, \langle (Q, NSW), Q \neq NSW \rangle, \langle (NSW, V), NSW \neq V \rangle \}$, where each constraint is a **binary constraint** that states that two adjacent regions must have different colors.

Solving a CSP Problem

- One of the simplest algorithms to solve a CSP problem is the **backtracking algorithm**, which is a **recursive depth-first search** that tries to assign values to variables one by one and backtracks if a conflict is found.
- The algorithm can be improved by using various heuristics, such as **variable ordering**, **value ordering**, and **constraint propagation**.



- **Variable ordering:** Choose the variable with the most constraints on the remaining variables, i.e., the **most connected variable**. This is also known as the **minimum remaining values (MRV)** heuristic.
- **Value ordering:** Choose the value that rules out the fewest choices for the neighboring variables, i.e., the **least constraining value (LCV)** heuristic.
- **Constraint propagation:** Use **forward checking** to keep track of the remaining legal values for the unassigned variables, and prune the domains that are inconsistent with the assignment.



- The algorithm starts with an empty assignment and selects the first variable to assign.
- According to the **MRV heuristic**, the **variable with the most constraints is SA, as it has six neighbors**. The algorithm then tries to assign a value to SA, and according to the **LCV heuristic**, it **chooses red**, as it is the least constraining value for the neighboring variables. The algorithm then uses forward checking to prune the domains of the neighboring variables and updates the domains as follows:
 - WA: {green, blue}
 - NT: {green, blue}
 - Q: {green, blue}
 - NSW: {green, blue}
 - V: {green, blue}
 - T: {red, green, blue}

- According to the MRV heuristic, the next variable with the most constraints is **NT**, as it has three neighbors. The algorithm then tries to assign a value to NT, and according to the LCV heuristic, it chooses **green**. The algorithm then uses **forward checking** to prune the domains of the neighbouring variables and updates the domains as follows:
 - WA: {blue}
 - Q: {blue}
 - SA: {red}
 - NSW: {green, blue}
 - V: {green, blue}
 - T: {red, green, blue}
- According to the MRV heuristic, the variable with the most constraints is **WA**, as it has two neighbors. The algorithm then tries to assign a value to WA, and according to the LCV heuristic, it chooses **blue**

- NT: {green}
- SA: {red}
- Q: {blue}
- NSW: {green}
- V: {green, blue}
- T: {red, green, blue}

Solving 8-Queens Problem using Local Search-CSP

- Local search algorithms are optimization techniques that start with an initial solution and iteratively make small, incremental changes to it in pursuit of a better solution.
- Variables (X): we have eight variables: $X = \{X_1, X_2, X_3, X_4, X_5, X_6, X_7, X_8\}$.
- $D = \{1, 2, 3, 4, 5, 6, 7, 8\}$ for all X_i .
- In the 8-Queen problem, we have three primary types of constraints: Row Constraint, Column Constraint, Diagonal Constraint.

- In the 8-queens problem, the initial state might be a random configuration of 8 queens in 8 columns, and each step moves a single queen to a new position in its column.
- Typically, the initial guess violates several constraints.
- The point of local search is to eliminate the violated constraints.
- In choosing a new value for a variable, the most obvious heuristic is to select the value **MIN-CONFLICTS** that results in the minimum number of conflicts with other variables—the min-conflicts heuristic.

function MIN-CONFLICTS(*csp*, *max_steps*) **returns** a solution or failure

inputs: *csp*, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current $\leftarrow$ an initial complete assignment for *csp*

for *i* = 1 to *max_steps* **do**

if *current* is a solution for *csp* **then return** *current*

var $\leftarrow$ a randomly chosen conflicted variable from *csp*.VARIABLES

value $\leftarrow$ the value *v* for *var* that minimizes CONFLICTS(*var*, *v*, *current*, *csp*)

 set *var* = *value* in *current*

return *failure*

Figure 6.8 The MIN-CONFLICTS algorithm for solving CSPs by local search. The initial state may be chosen randomly or by a greedy assignment process that chooses a minimal-conflict value for each variable in turn. The CONFLICTS function counts the number of constraints violated by a particular value, given the rest of the current assignment.

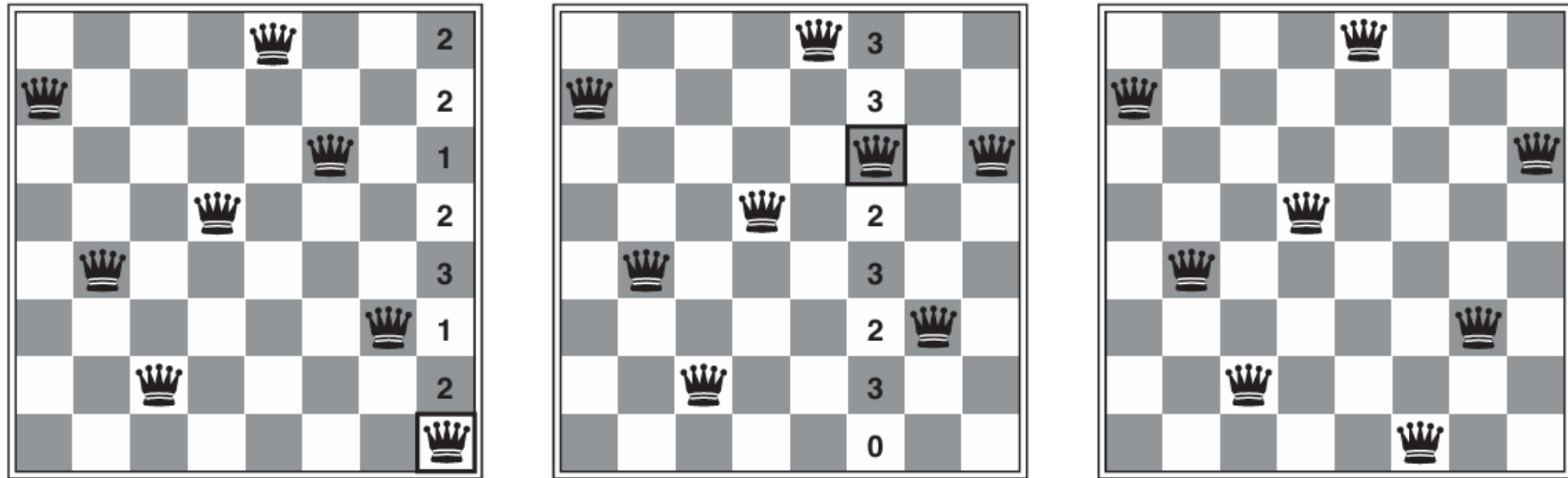


Figure 6.9 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Summary

- CSPs are a special kind of search problems:
 - states defined by values of a fixed set of variables
 - goal test defined by constraints on variable values
- Backtracking = depth-first search with one variable assigned per node
- Variable ordering and value selection heuristics help
- Forward checking prevents assignments that guarantee later failure
- Constraint propagation (e.g., arc consistency) is an important mechanism in CSPs.
- It does additional work to constrain values and detect inconsistencies.
- Tree-structured CSPs can be solved in linear time
- Further exploration: How can local search be used for CSPs?
- **The power of CSPs: domain-independent, that is you only need to define the problem and then use a solver that implements CSPs mechanisms.**